F-16 İçin Otonom Yer Çarpışma Önleyici Sistem Geliştirilmesi

Celal Aksoy[1]

Uçak Mühendisliği Bölümü, İstanbul Teknik Üniversitesi

I. Özet

Bu çalışmada, F-16 savaş uçağı için geliştirilmiş yer çarpışma önleyici sistem (GCAS) kapsamında, klasik tekil kaçış manevrasına dayalı yaklaşımlar yerine çoklu yörünge (multi-trajectory) değerlendirmesine dayalı bir sistem önerilmiştir. Çalışmanın temel amacı, düşük irtifada ve karmaşık arazi koşullarında uçağın önceden tanımlanmış altı farklı kaçış manevrasını (sağ, sol ve yukarı yönlü 3.5g ve 4.5g manevralar) aynı anda analiz ederek, araziyle çakışma riski en az olan rotayı gerçek zamanlı olarak seçmesini sağlamaktır.

Geliştirilen sistem üç ana bölümden oluşmaktadır: (1) Yönlendirme algoritması, (2) GCAS karar mekanizması ve (3) geçiş koşullarını yöneten "Stateflow" yapısı. Yönlendirme algoritması, uçak takip noktalarını (waypoint) takip ederken "carrot-chasing" yöntemiyle yönelimi kontrol eder. GCAS algoritması ise uçağın kaçış yolunda sanal noktalar belirleyerek her bir G kuvveti için farklı yönlerde olası kaçış yörüngeleri oluşturur ve bu yörüngeleri araziyle çakışma açısından değerlendirir. Çakışma tespiti, MATLAB ortamında bir yüzey haritası (terrain surface) üzerinde yapılan kesişim analizleriyle gerçekleştirilmiştir. En son kalan güvenli yörünge, sistem tarafından otomatik olarak seçilerek uçuş kontrolüne iletilir. Tüm geçiş mantığı, Stateflow diagramı ile modellenmiş ve "güdüm" ile "kurtarma" modları arasında dinamik geçişler senkronize edilmiştir.

Sistem uçuş simülasyonlarıyla test edilmiş, uçağın seyri ve arazi yapısı 3B olarak görselleştirilmiştir. Görselleştirme için MATLAB'in FlightGear ve FlyPath3D paketleri entegre edilmiş, geliştirilen sistemin hem gerçek zamanlı izlenmesi hem de animasyonla sunulması sağlanmıştır. Bu yapı sayesinde sadece çarpışmadan kaçınmak değil, aynı zamanda gereksiz manevralardan kaçınılarak operasyonel verimlilik de artırılmıştır.

II. Giriş ve Amaçlar

A. Giriş

Modern savaş uçaklarının yüksek hız ve manevra kabiliyeti, beraberinde yer çarpışma riski gibi ciddi tehlikeleri de getirmiştir. Bu tehlikelerin başında, pilotun durumsal farkındalığını kaybettiği veya reaksiyon süresinin yetersiz kaldığı durumlarda meydana gelen kontrol kayıpları gelmektedir. Bu gibi durumların önüne geçmek amacıyla geliştirilen "Ground Collision Avoidance System (GCAS)", bir uçağın mevcut uçuş durumunu ve çevresel bilgileri analiz ederek potansiyel çarpışma tehlikelerini önceden tespit eden ve gerektiğinde otonom müdahale eden bir güvenlik sistemidir.

İlk GCAS sistemleri, esas olarak pilot uyarısına dayalı olup, pilotun karar verme sürecine müdahale etmeksizin yalnızca tehlikeyi bildiren yapılardı. Ancak zamanla bu sistemler, otomatik manevra başlatabilen daha gelişmiş sürümlere evrilmiştir. Bu evrimin en önemli adımlarından biri, "Automatic GCAS (Auto-GCAS)" sistemidir. Lockheed Martin ve USAF tarafından geliştirilen bu sistem, gerçek zamanlı olarak uçuş dinamiklerini ve arazi verilerini karşılaştırarak, kaçınılmaz çarpışma senaryolarında müdahale etme yetisine sahiptir [1]. Bu sistemlerin ilk sürümlerinde kullanılan yöntemlerden biri, yalnızca tek bir kaçış yörüngesinin (örneğin, yukarıya doğru 5g'lik bir pull-up manevrası) hesaplandığı ve bu yörüngenin çarpışıp çarpışmadığının kontrol edildiği "single-trajectory" yaklaşımdır. Bu yaklaşım, işlem yükünü minimal tutsa da, birçok durumda yetersiz kalabilmektedir. Özellikle dar vadiler veya

[1] E-mail: aksoyc20@itu.edu.tr

Danışman: Emre Koyuncu, Uçak Mühendisliği, İstanbul Teknik Üniversitesi, koyuncuem@itu.edu.tr

karmaşık topoğrafyalarda tek bir kaçış opsiyonunun değerlendirilmesi, ya gereksiz manevralara sebep olmakta ya da açık rotaların gözden kaçırılmasına neden olmaktadır [2]. Bu bağlamda son yıllarda geliştirilen yeni yaklaşımlar, "multi-trajectory GCAS" sistemlerine evrilmiştir [3]. Çoklu yörünge yaklaşımlarında, her bir andaki uçak durumu için farklı yönlerde (sağ, sol, yukarı) farklı kaçış manevrası hesaplanmakta ve her birinin araziyle çakışma durumu ayrı ayrı değerlendirilmektedir.[4] Bu yöntem, hem gereksiz kaçışları önlemekte hem de daha geç müdahale olanağı sağlayarak pilotun kontrolü yeniden kazanma ihtimalini artırmaktadır [5]. Wang ve arkadaşları tarafından önerilen çoklu yörünge tabanlı GCAS mimarisi, risk değerlendirme ve karar fonksiyonu gibi iki katmanlı bir yapıyla donatılmıştır. Bu sistem, her manevra rotası için çarpışma riskini hesaplayarak en güvenli ve en az maliyetli kaçışı seçebilmektedir. Ayrıca, klasik GCAS sistemlerinde sıkça karşılaşılan "nuisance alarm" problemlerinin de bu yaklaşım ile önemli ölçüde azaldığı görülmüştür [5]. Literatürde yapılan bazı simülasyon çalışmalarında, çoklu yörünge sistemlerinin özellikle düşük irtifada ve yüksek hızlarda uçuş senaryolarında geleneksel sistemlere kıyasla daha iyi performans gösterdiği bildirilmiştir [2]. Ayrıca, gerçek uçuş testlerinde elde edilen veriler, Auto-GCAS sistemlerinin yüzlerce kazayı önleyebilecek potansiyele sahip olduğunu ortaya koymuştur [1].

Sonuç olarak, GCAS sistemleri özellikle yeni nesil savaş platformlarında bir gereklilik haline gelmiş ve sürekli gelişim göstermiştir. Bu çalışmada geliştirilen sistem de, mevcut literatürdeki multi-trajectory temelli yaklaşımların bir örneği olarak tasarlanmış ve MATLAB/Simulink ortamında otonom kaçış mantığı ile desteklenmiştir. Sistemin temel amacı, en uygun kaçış manevrasını seçerek çarpışmaları önlemek ve pilotun müdahalesine gerek kalmaksızın güvenli bir uçuş profili sağlamaktır.

B. Amaç

Bu çalışmanın temel amacı, F-16 tipi savaş uçakları için geliştirilen mevcut yer çarpışma önleyici sistemlerin (Ground Collision Avoidance System, GCAS) eksikliklerini giderecek şekilde, çoklu manevra (multi-trajectory) analizi temelli otonom bir yer çarpışma önleyici sistem tasarlamak ve bu sistemi MATLAB/Simulink ortamında modelleyerek sentetik arazi üzerinde simüle etmektir. Günümüzde kullanılan GCAS sistemleri çoğunlukla tekil kaçış manevraları üzerine kurgulanmış olup, uçağın mevcut yönelimine göre önceden tanımlanmış bir “pull-up” rotası ile çarpışma tespiti yapılmaktadır. Ancak bu yaklaşım, özellikle dar vadilerde, asimetrik arazilerde veya dinamik uçuş senaryolarında kaçınılmaz yanlış pozitif uyarılar veya gereksiz kaçış manevralarına neden olabilmektedir. Bu bağlamda çalışmada, her anlık durumda uçağın arka merkezine yerleştirilen referans bir noktadan başlayan ve uçağın burnuna doğru uygulanan belirli çekiş kuvvetlerine ($3.5g$ ve $4.5g$) göre yukarı (pull-up), sağ ve sol yönlerde toplam altı farklı kaçış manevrası (trajectory) hesaplanmaktadır. Bu çoklu yörünge seti, önceden oluşturulmuş sentetik bir arazi haritası ile çarpışma açısından anlık olarak kontrol edilmekte ve herhangi bir çarpışma tehdidi algılandığında, mevcut koşullara en uygun kaçış manevrası seçilmektedir.Yalnızca çarpışma algılama değil, aynı zamanda karar verme (decision-making) mekanizmasının da sistem mimarisi içerisine entegre edilmesi hedeflenmiştir. Bu karar yapısı, her bir manevra için potansiyel çarpışma zamanını (time-to-collision) hesaplayarak, en son geçerli kalacak manevrayı otomatik olarak tespit eder. Böylece sistem, hem gereksiz manevralardan kaçınmakta hem de kaçınılmaz durumlarda en güvenli rota ile müdahaleyi gerçekleştirmektedir.

Sonuç olarak bu çalışmanın amacı:

- Geleneksel GCAS sistemlerinin sınırlılıklarını analiz etmek,
- Çoklu manevra tabanlı yeni bir GCAS algoritması geliştirmek,
- Bu algoritmayı gerçek zamanlı simülasyon ortamında test etmek,
- Geliştirilen sistemin karar alma doğruluğunu ve uygulama başarımını görselleştirmek,
- Gelecekte gerçek uçaklara entegre edilebilir modüler ve genişletilebilir bir algoritmik altyapı sunmaktır.

Bu kapsamda geliştirilen sistemin, hem teorik olarak hem de uygulamalı olarak mevcut literatüre katkı sunmasını; aynı zamanda insan-makine etkileşimini minimize eden tam otonom karar alma mekanizmalarının savunma havacılığına entegrasyonu konusunda örnek teşkil etmesi amaçlanmaktadır.

C. Motivasyon ve Yaygın Etki

Hava araçları için geliştirilen yer çarpışma önleyici sistemlerin (GCAS) temel amacı, pilot farkındalığı azaldığında ya da kontrol kaybı yaşandığında, hava aracının araziyle istemsiz çarpışmasını engellemektir. Özellikle yüksek

manevra kabiliyetine sahip savaş uçaklarında bu tür kazalar, genellikle operasyonel stres, düşük irtifada uçuş veya fizyolojik faktörler (örneğin G-force kaynaklı bilinç kaybı, G-LOC) nedeniyle meydana gelmektedir. ABD Hava Kuvvetleri'nin 2010'lu yıllarda açıkladığı verilere göre, sadece 2000–2010 yılları arasında yaşanan kazalarda, GCAS sistemi ile önlenebilecek en az 22 ölümcül olay tespit edilmiştir. Bu kazaların önemli bir kısmı, pilotun farkında olmadan kontrollü bir şekilde yere doğru yöneldiği ve müdahale edemediği durumlarda meydana gelmiştir.

Bu bağlamda 2014 yılında ABD Hava Kuvvetleri, Lockheed Martin ve NASA ortaklığında geliştirilen ve F-16 uçaklarına entegre edilen ilk gerçek zamanlı otomatik GCAS sistemiyle (Automatic GCAS – Auto GCAS) birçok can kaybı önlenmiştir. Örneğin, 2016 yılında yaşanan bir olayda, F-16 pilotunun G kuvvetinden dolayı bilincini kaybettiği sırada uçak yere yaklaşmış ve sistem devreye girerek zamanında bir kaçış manevrası başlatmıştır. Bu olay, sadece sistemin teknik başarısını değil, aynı zamanda savaş pilotlarının hayatını kurtaran kritik rolünü de gözler önüne sermiştir.[1](Deets ve arkadaşları, 2014) Ancak mevcut sistemlerin büyük bir çoğunluğu yalnızca tekil bir kaçış manevrasını (tipik olarak 5g'lik bir "pull-up") temel alır. Bu yöntem, özellikle vadiler, dağlık bölgeler veya sınırlı hava sahalarında gereksiz manevralara ve operasyonel risklere yol açabilmektedir. Bu nedenle son yıllarda çoklu manevra (multi-trajectory) değerlendirmesi içeren yeni nesil algoritmalar üzerine çalışmalar yapılmaktadır. Wang ve arkadaşları (2020), bu yaklaşımın hem operasyonel esneklik sağladığını hem de araziye uygun karar üretme başarımını artırdığını göstermiştir [5].

Bu çalışma, yukarıda belirtilen eksiklikleri giderme motivasyonuyla ortaya çıkmış; farklı G seviyelerinde ve yönlerde çoklu kaçış manevralarının eş zamanlı değerlendirilmesi ile hem daha gerçekçi hem de daha güvenli bir sistem geliştirmeyi hedeflemiştir.

Bu sistemin olası yaygın etkileri şu şekilde özetlenebilir:

- Özellikle modern savaş uçaklarına entegre edilecek bu tip sistemler sayesinde, pilot hatasından veya kontrol kaybından kaynaklı ölümcül kazaların önüne geçilebilir.
- Tam otonom çalışabilen ve insan müdahalesi olmadan doğru manevrayı seçebilen bu yapılar, yüksek irtifada değil, alçak irtifada ve dar arazi yapılarında dahi etkili olarak çalışabilir.
- Geliştirilen algoritmalar sadece F-16 gibi avcı uçaklarına değil, İHA (İnsansız Hava Aracı) sistemlerine de uyarlanarak sivil havacılıkta da çarpışma önleyici yazılımların yaygınlaşmasına katkı sağlayabilir.
- Eğitim simülatörlerinde kullanılmak üzere oluşturulan bu tip algoritmalar, yeni nesil pilot eğitimlerinde karar verme reflekslerini ölçmek amacıyla da kullanılabilir.

Sonuç olarak, bu çalışmanın motivasyonu yalnızca akademik bir algoritma geliştirmek değil, aynı zamanda operasyonel güvenliği artıracak, yaşam kurtaracak ve sistemin gelecekteki savunma ve sivil uygulamalarına katkı sağlayacak bir yapı sunmaktır.

III. Yöntem ve Kaynak Analizi

A. Projede Kullanınlan Yöntemler

Bu projede, F-16 uçağı için geliştirilen Otonom Yer Çarpışma Önleyici Sistem (Auto-GCAS), çok yörüngeli çarpışma risk analizi yaklaşımına dayanmaktadır. Geleneksel sistemlerin aksine yalnızca bir adet kaçış rotasını değerlendirmek yerine, sistem her kontrol döngüsünde altı farklı kaçış manevrası üretir ve bu manevraların her birini gerçek zamanlı olarak sentetik arazi verisiyle karşılaştırarak çarpışma riskini analiz eder. Bu yaklaşım, karar ağacı temelli bir kontrol mimarisi ile gerçekleştirilmiş ve her adımda en uygun manevranın seçilmesini sağlamıştır.

Uygulama düzeyinde sistem üç ana bileşene ayrılmıştır: güdüm algoritması, GCAS karar sistemi ve stateflow mantığı. Güdüm algoritması, "carrot-chasing" temelli basit bir rota takip mantığına dayanmakta; "waypoint" takibi ile uçağın normal görev profili sağlanmaktadır. Bu algoritma, MATLAB/Simulink ortamında gerçekleştirilmiştir.

GCAS bloğu ise her 0.1 saniyede bir, yükseklik, yatay ve yönelme bilgilerine dayanarak altı farklı kaçış yörüngesini hesaplamakta ve her biri için arazide çarpışma kontrolü gerçekleştirmektedir.

Çarpışma kontrolü, önceden oluşturulmuş sentetik bir arazi matrisi ile yapılmıştır. Arazi verisi, rastgele dağ ve tepe yapıları ile şekillendirilmiş olup, Gauss dağılım fonksiyonlarıyla modellenmiştir. Böylece gerçek dünyadaki

çeşitli coğrafi durumların benzetimi sağlanmıştır.

Karar yapısı ise Stateflow mantığı ile modellenmiş, sistemin "güvenli rota" ya da "kurtarma modu" arasında geçiş yapabilmesi sağlanmıştır. GCAS, çarpışma tespiti yapılır yapılmaz güdüm sistemini devre dışı bırakır ve uygun kaçış manevrasını aktif hale getirir. Manevranın başarıyla tamamlanmasının ardından sistem yeniden waypoint moduna döner.

Görsel doğrulama için FlightGear simülatörü kullanılmış, uçuş yolu ise "flypath3d" eklentisiyle 3 boyutlu olarak animasyonlaştırılmıştır. Elde edilen çıktılar, algoritmanın işlevselliğini hem görsel hem sayısal olarak değerlendirmek için kullanılmıştır.

B. Kaynak Analizi

Projede geliştirilen yer çarpışma önleyici sistemin başarıyla simüle edilebilmesi ve algoritmaların doğrulanabilmesi için çeşitli kaynaklara ihtiyaç duyulmuştur. Bu kaynaklar; insan gücü, yazılım araçları, hesaplama altyapısı ve görsel simülasyon bileşenleri şeklinde sınıflandırılmıştır.

1. İnsan Gücü

Çalışma, iki kişilik mühendislik öğrencisi ekibi tarafından yürütülmüştür. Geliştirme süreci boyunca uçuş dinamiği, kontrol teorisi, MATLAB-Simulink ortamı ve havacılık elektroniği konularında ileri düzey bilgi gerektiren uygulamalar gerçekleştirilmiştir.

2. Yazılım ve Hesaplama Altyapısı

Simülasyon ve algoritma geliştirme çalışmaları MATLAB R2024b ve Simulink ortamında gerçekleştirilmiştir. Ara yüzey modellemesi için Stateflow, 3D animasyon için "flypath3d" eklentisi ve FlightGear uçuş simülatörü kullanılmıştır. Hesaplamaların gerçekleştirildiği bilgisayar, 8 çekirdekli bir Intel i7 işlemciye, 16 GB RAM'e ve 524 GB SSD tabanlı bir depolama birimine sahiptir. Bu yapı, simülasyonların akıcı ve eşzamanlı olarak yürütülmesini mümkün kılmıştır.

3. Fiziksel ve Sanal Ortamlar

Sistemin gerçek uçuş testleri yerine sentetik ortamda denenmesi için sanal simülasyon ortamları kullanılmıştır. Bu amaçla:

- FlightGear: Uçağın görsel olarak modellenmesi ve manevraların gerçek zamanlı görsel testleri için kullanılmıştır.
- MATLAB Terrain Modeling: Yükseklik haritalarının oluşturulması ve analiz edilmesi için özel olarak geliştirilmiş terrain generation scriptleri uygulanmıştır.
- Simulink: GCAS, guidance ve stateflow algoritmalarının blok tabanlı modellenmesi için temel ortam sağlanmıştır.

4. Veri ve Donanım Gereksinimi

Proje "f16.mat" datası hariç nerdeyse tamamen sentetik verilere dayanarak tasarlanmış olup, herhangi bir gerçek uçuş verisi kullanılmamıştır. Bu sayede geliştirme süreci hem ekonomik hem de hızlı olmuştur. Donanım ihtiyaçları yalnızca kişisel bilgisayarlarla sınırlı kalmış, özel laboratuvar veya donanımsal simülatör ihtiyacı doğmamıştır.

Sonuç olarak, proje düşük maliyetli ve tamamen yazılım temelli bir ortamda geliştirilmiş, kaynak etkin bir mühendislik yaklaşımı sergilemiştir.

IV. F-16 İçin Otonom Yer Çarpışma Önleyici Sistem Geliştirilmesi

F-16 savaş uçağına otonom yer çarpışma önleyici sistem (Auto-GCAS) geliştirmek amacıyla yürütülen bu çalışmada, tasarım süreci üç temel aşamada ele alınmıştır. İlk olarak, uçağın uçuş karakteristiklerini fiziksel olarak temsil eden altı serbestlik dereceli (6DoF) dinamik model oluşturulmuştur. İkinci aşamada, bu modele entegre çalışacak şekilde alt seviye kontrolcü yapıları tasarlanarak uçuş kararlılığı ve temel komut takip kabiliyeti sağlanmıştır. Son aşamada ise, hem görev bölgesine yönelim sağlayan güdüm algoritmalarını hem de çarpışma riskine karşı kaçış manevralarını yöneten otonom üst seviye kontrolcü yapısı geliştirilmiştir. Bu yapının genel blok diyagramı Proje Ekleri B bölümünde Şekil 27'de sunulmuştur.

A. Dinamik Model

Tablo 1, bu bölümde tanımlanan dinamik modelde kullanılan sembolleri ve kısaltmaları içermektedir. Söz konusu dinamik model, Nguyen ve arkadaşlarının [6] yayınladığı çalışmada sunulan F-16'ya ait gerçek uçuş denklemleri temel alınarak oluşturulmuştur. Söz konusu dinamik model bloğunun blok diagramı Proje EkleriB bölümünde Şekil 4"de sunulmuştur.

Tablo 1 Aerodinamik Denklemlerde Kullanılan Semboller ve Kısaltmalar

Sembol	Açıklama	Birim / Değer
Uçuş Durumu Değişkenleri		
α	Hücum açısı (Angle of Attack)	rad
β	Yan kayma açısı (Sideslip Angle)	rad
p	Yuvarlanma (roll) hızı	rad/s
q	Yunuslama (pitch) hızı	rad/s
r	Sapma (yaw) hızı	rad/s
V	Uçağın toplam hızı	m/s
ρ	Hava yoğunluğu	kg/m³
$\bar{q}$	Dinamik basınç, $\frac{1}{2}\rho V^2$	N/m²
P_S	Statik basınç	N/m²
Kontrol Yüzeyi Sapmaları		
δ_h	Yükseliş dümeni sapması	rad
δ_a	Kanatçık sapması	rad
δ_r	İstikamet dümeni sapması	rad
δ_{lef}	Hücum kenarı flapı sapması	rad
δ_{sb}	Hız freni sapması	rad
Dinamik Türevler		
$C_{X_q}, C_{Z_q}, C_{m_q}$	Yunuslama hızına (q) bağlı türevler	–
$C_{Y_p}, C_{l_p}, C_{n_p}$	Yuvarlanma hızına (p) bağlı türevler	–
$C_{Y_r}, C_{l_r}, C_{n_r}$	Sapma hızına (r) bağlı türevler	–
Uçak Geometrisi ve Referans Değerler		
S	Kanat referans alanı	27.88 m²
b	Kanat açıklığı	9.144 m
$\bar{c}$	Ortalama aerodinamik kord uzunluğu	3.45 m
x_{cg}	Ağırlık merkezinin anlık konumu	0.35 (oran)
$x_{\text{cg,ref}}$	Referans ağırlık merkezi konumu	0.35 (oran)
η_{δ_h}	Yükseliş dümeni etkinlik faktörü	–

Dinamik modeli besleyen veri yapısı, `f16.mat` dosyasında tanımlı olan aerodinamik ve motor bağımlı katsayıların giriş parametrelerine bağlı olarak çıkarılmasını sağlar. Yapı, MATLAB/Simulink ortamında birden fazla boyuta sahip look-up tabloları ile birlikte kullanılır. Giriş değişkenleri önce "Prelookup" bloğu aracılığıyla indekslenir. Bu indeks değerleri daha sonra "Interpolation Using Prelookup" bloklarına iletilerek, ilgili katsayının doğrusal veya parçalı doğrusal (piecewise linear) interpolasyon yöntemiyle hesaplanması sağlanır. Bu sayede model, sürekli giriş değerlerine karşılık gelen doğru katsayıyı hızlı ve hassas şekilde üretir. Bu yapı, Proje Ekleri bölümünde Şekil 6 olarak gösterilmiştir.

1. Aerodinamik Kuvvet Katsayısı Denklemleri

X eksenindeki toplam kuvvet katsayısı Denklem 1"de, Simulink diagramı ise Proje Ekleri B bölümünde Şekil 7"de verilmiştir.

$$\begin{aligned} C_{X,t} = C_X(\alpha,\beta,\delta_h) &+ \Delta C_{X,\text{lef}}\left(1-\frac{\delta_{\text{lef}}}{25}\right) + \Delta C_{X,\text{sb}}(\alpha)\left(\frac{\delta_{\text{sb}}}{60}\right) \\ &+ \frac{\bar{c}q}{2V}\left[C_{X_q}(\alpha) + \Delta C_{X_q,\text{lef}}(\alpha)\left(1-\frac{\delta_{\text{lef}}}{25}\right)\right] \end{aligned} \tag{1}$$

burada

$$\Delta C_{X,\text{lef}} = C_{X,\text{lef}}(\alpha,\beta) - C_X(\alpha,\beta,\delta_h = 0^\circ) \tag{2}$$

Y eksenindeki toplam kuvvet katsayısı Denklem 3"de, Simulink diagramı ise Proje Ekleri B bölümünde Şekil 8"de verilmiştir.

$$\begin{aligned} C_{Y,t} = C_Y(\alpha,\beta) &+ \Delta C_{Y,\text{lef}}\left(1-\frac{\delta_{\text{lef}}}{25}\right) + \left[\Delta C_{Y,\delta_a=20^\circ} + \Delta C_{Y,\delta_a=20^\circ,\text{lef}}\left(1-\frac{\delta_{\text{lef}}}{25}\right)\right]\left(\frac{\delta_a}{20}\right) \\ &+ \Delta C_{Y,\delta_r=30^\circ}\left(\frac{\delta_r}{30}\right) + \frac{b}{2V}\left\{\left[C_{Y_r}(\alpha) + \Delta C_{Y_r,\text{lef}}(\alpha)\left(1-\frac{\delta_{\text{lef}}}{25}\right)\right] r \right. \\ &\left. + \left[C_{Y_p}(\alpha) + \Delta C_{Y_p,\text{lef}}(\alpha)\left(1-\frac{\delta_{\text{lef}}}{25}\right)\right] p\right\} \end{aligned} \tag{3}$$

burada

$$\Delta C_{Y,\text{lef}} = C_{Y,\text{lef}}(\alpha,\beta) - C_Y(\alpha,\beta) \tag{4}$$

$$\Delta C_{Y,\delta_a=20^\circ} = C_{Y,\delta_a=20^\circ}(\alpha,\beta) - C_Y(\alpha,\beta) \tag{5}$$

$$\Delta C_{Y,\delta_a=20^\circ,\text{lef}} = C_{Y,\delta_a=20^\circ,\text{lef}}(\alpha,\beta) - C_{Y,\text{lef}}(\alpha,\beta) \tag{6}$$

$$- \left[C_{Y,\delta_a=20^\circ}(\alpha,\beta) - C_Y(\alpha,\beta)\right] \tag{7}$$

$$\Delta C_{Y,\delta_r=30^\circ} = C_{Y,\delta_r=30^\circ}(\alpha,\beta) - C_Y(\alpha,\beta) \tag{8}$$

Z eksenindeki toplam kuvvet katsayısı Denklem 9"de, Simulink diagramı ise Proje Ekleri B bölümünde Şekil 9"de verilmiştir.

$$\begin{aligned} C_{Z,t} = C_Z(\alpha,\beta,\delta_h) &+ \Delta C_{Z,\text{lef}}\left(1-\frac{\delta_{\text{lef}}}{25}\right) + \Delta C_{Z,\text{sb}}(\alpha)\left(\frac{\delta_{\text{sb}}}{60}\right) \\ &+ \frac{\bar{c}q}{2V}\left[C_{Z_q}(\alpha) + \Delta C_{Z_q,\text{lef}}(\alpha)\left(1-\frac{\delta_{\text{lef}}}{25}\right)\right] \end{aligned} \tag{9}$$

burada

$$\Delta C_{Z,\text{lef}} = C_{Z,\text{lef}}(\alpha,\beta) - C_Z(\alpha,\beta,\delta_h = 0^\circ) \tag{10}$$

2. Aerodinamik Moment Katsayısı Denklemleri

X eksenindeki toplam moment katsayısı (yuvarlanma) Denklem 11"de, Simulink diagramı ise Proje Ekleri B bölümünde Şekil 11"de verilmiştir.

$$\begin{aligned} C_{l,t} = {} & C_l(\alpha,\beta,\delta_h) + \Delta C_{l,\text{lef}}\left(1-\frac{\delta_{\text{lef}}}{25}\right) + \left[\Delta C_{l,\delta_a=20^\circ} + \Delta C_{l,\delta_a=20^\circ,\text{lef}}\left(1-\frac{\delta_{\text{lef}}}{25}\right)\right]\left(\frac{\delta_a}{20}\right) \\ & + \Delta C_{l,\delta_r=30^\circ}\left(\frac{\delta_r}{30}\right) + \frac{b}{2V}\left\{\left[C_{l_r}(\alpha) + \Delta C_{l_r,\text{lef}}(\alpha)\left(1-\frac{\delta_{\text{lef}}}{25}\right)\right]r + \left[C_{l_p}(\alpha) + \Delta C_{l_p,\text{lef}}(\alpha)\left(1-\frac{\delta_{\text{lef}}}{25}\right)\right]p\right\} \\ & + \Delta C_{l_\beta}(\alpha)\beta \end{aligned} \tag{11}$$

burada

$$\Delta C_{l,\text{lef}} = C_{l,\text{lef}}(\alpha,\beta) - C_l(\alpha,\beta,\delta_h=0^\circ) \tag{12}$$
$$\Delta C_{l,\delta_a=20^\circ} = C_{l,\delta_a=20^\circ}(\alpha,\beta) - C_l(\alpha,\beta,\delta_h=0^\circ) \tag{13}$$
$$\Delta C_{l,\delta_a=20^\circ,\text{lef}} = C_{l,\delta_a=20^\circ,\text{lef}}(\alpha,\beta) - C_{l,\text{lef}}(\alpha,\beta) \tag{14}$$
$$- \left[C_{l,\delta_a=20^\circ}(\alpha,\beta) - C_l(\alpha,\beta,\delta_h=0^\circ)\right] \tag{15}$$
$$\Delta C_{l,\delta_r=30^\circ} = C_{l,\delta_r=30^\circ}(\alpha,\beta) - C_l(\alpha,\beta,\delta_h=0^\circ) \tag{16}$$

Y eksenindeki toplam moment katsayısı (yunuslama) Denklem 17"de, Simulink diagramı ise Proje Ekleri B bölümünde Şekil 12"de verilmiştir.

$$\begin{aligned} C_{m,t} = {} & C_m(\alpha,\beta,\delta_h)\eta_{\delta_h}(\delta_h) + C_{Z,t}(x_{\text{cg,ref}} - x_{\text{cg}}) + \Delta C_{m,\text{lef}}\left(1-\frac{\delta_{\text{lef}}}{25}\right) + \Delta C_{m,\text{sb}}(\alpha)\left(\frac{\delta_{\text{sb}}}{60}\right) \\ & + \frac{\bar{c}q}{2V}\left[C_{m_q}(\alpha) + \Delta C_{m_q,\text{lef}}(\alpha)\left(1-\frac{\delta_{\text{lef}}}{25}\right)\right] + \Delta C_m(\alpha) + \Delta C_{m,\text{ds}}(\alpha,\delta_h) \end{aligned} \tag{17}$$

burada

$$\Delta C_{m,\text{lef}} = C_{m,\text{lef}}(\alpha,\beta) - C_m(\alpha,\beta,\delta_h=0^\circ) \tag{18}$$

Z eksenindeki toplam moment katsayısı (sapma) Denklem 19"de, Simulink diagramı ise Proje Ekleri B bölümünde Şekil 13"de verilmiştir.

$$\begin{aligned} C_{n,t} = {} & C_n(\alpha,\beta,\delta_h) + \Delta C_{n,\text{lef}}\left(1-\frac{\delta_{\text{lef}}}{25}\right) - C_{Y,t}(x_{\text{cg,ref}} - x_{\text{cg}})\frac{\bar{c}}{b} + \left[\Delta C_{n,\delta_a=20^\circ} + \Delta C_{n,\delta_a=20^\circ,\text{lef}}\left(1-\frac{\delta_{\text{lef}}}{25}\right)\right]\left(\frac{\delta_a}{20}\right) \\ & + \Delta C_{n,\delta_r=30^\circ}\left(\frac{\delta_r}{30}\right) + \frac{b}{2V}\left\{\left[C_{n_r}(\alpha) + \Delta C_{n_r,\text{lef}}(\alpha)\left(1-\frac{\delta_{\text{lef}}}{25}\right)\right]r + \left[C_{n_p}(\alpha) + \Delta C_{n_p,\text{lef}}(\alpha)\left(1-\frac{\delta_{\text{lef}}}{25}\right)\right]p\right\} \\ & + \Delta C_{n_\beta}(\alpha)\beta \end{aligned} \tag{19}$$

burada

$$\Delta C_{n,\text{lef}} = C_{n,\text{lef}}(\alpha,\beta) - C_n(\alpha,\beta,\delta_h=0^\circ) \tag{20}$$
$$\Delta C_{n,\delta_a=20^\circ} = C_{n,\delta_a=20^\circ}(\alpha,\beta) - C_n(\alpha,\beta,\delta_h=0^\circ) \tag{21}$$
$$\Delta C_{n,\delta_a=20^\circ,\text{lef}} = C_{n,\delta_a=20^\circ,\text{lef}}(\alpha,\beta) - C_{n,\text{lef}}(\alpha,\beta) \tag{22}$$
$$- \left[C_{n,\delta_a=20^\circ}(\alpha,\beta) - C_n(\alpha,\beta,\delta_h=0^\circ)\right] \tag{23}$$
$$\Delta C_{n,\delta_r=30^\circ} = C_{n,\delta_r=30^\circ}(\alpha,\beta) - C_n(\alpha,\beta,\delta_h=0^\circ) \tag{24}$$

3. Hücum Kenarı Flap Sapması Denklemi

$$\delta_{\text{lef}} = 1.38 \cdot \frac{2S + 7.25}{S + 7.25} \cdot \alpha - 9.05 \cdot \frac{\bar{q}}{P_{\text{S}}} + 1.45 \tag{25}$$

4. Toplam Aerodinamik Kuvvet ve Momementler

$$F_X = \frac{1}{2}\rho V^2 S C_{X,t} \tag{26}$$

$$F_Y = \frac{1}{2}\rho V^2 S C_{Y,t} \tag{27}$$

$$F_Z = \frac{1}{2}\rho V^2 S C_{Z,t} \tag{28}$$

$$M_X = \frac{1}{2}\rho V^2 S b C_{l,t} \tag{29}$$

$$M_Y = \frac{1}{2}\rho V^2 S \bar{c} C_{m,t} \tag{30}$$

$$M_Z = \frac{1}{2}\rho V^2 S b C_{n,t} \tag{31}$$

5. Motor Kuvvet Denklemi

Proje EkleriB bölümündeki Şekil14"de görüldüğü üzere uçağın motor itki kuvveti T, throttle komutu olan güç oranı P_c ile doğrusal parçalara ayrılmış bir fonksiyon olarak modellenmiştir. Bu modelde:

- T_{idle}: motorun boşta çalıştığı durumda ürettiği minimum itki,
- T_{mil}: askeri güç seviyesinde (military power) elde edilen sabit itki,
- T_{max}: art yakıcı açıkken ulaşılan maksimum itki

olarak tanımlanmıştır. İtki değeri, throttle komutunun büyüklüğüne göre aşağıda bulunan Denklem 32 gibi hesaplanır:

$$T = \begin{cases} T_{\text{idle}} + (T_{\text{mil}} - T_{\text{idle}}) \cdot \dfrac{P_c}{50}, & \text{if } P_c < 50 \\ T_{\text{mil}} + (T_{\text{max}} - T_{\text{mil}}) \cdot \dfrac{P_c - 50}{50}, & \text{if } 50 \leq P_c \end{cases} \tag{32}$$

Bu model, throttle komutunun $0 \leq P_c \leq 100$ aralığında olduğu varsayımıyla çalışmaktadır. İlk parça, idle ile military güç seviyeleri arasında doğrusal bir geçiş tanımlar. İkinci parça ise military güçten maximum power seviyesine geçişi ifade eder. Bu sayede motor davranışı sadeleştirilmiş fakat fiziksel olarak anlamlı bir şekilde modellenmiştir. İlgili iki rejimi gösteren blok diyagramlar Proje Ekleri B bölümünde sırasıyla Şekil 16 ve Şekil 15 olarak verilmiştir.

6. Yerçekimi Kuvveti Hesabı

Simülasyon ortamında, yerçekimi kuvvetinin uçak gövdesine göre etkisini doğru biçimde modelleyebilmek için, yerçekimi vektörünün dünya (inertial) koordinatlarından gövde koordinat sistemine dönüştürülmesi gerekmektedir. Bu işlemde yön kosinüs matrisi (Direction Cosine Matrix – DCM) kullanılmıştır. Şekil 17'te gösterilen yapıda, $[0 \quad 0 \quad + \text{Mass} \cdot 9.81]^\top$ vektörü, dünya koordinatlarında aşağı yönlü (negatif z) etki eden yerçekimi kuvvetini temsil eder. Bu vektör, DCM ile çarpılarak gövde koordinat sistemine dönüştürülür. Sonuçta elde edilen çıktı, gövde eksenlerinde hissedilen yerçekimi kuvveti bileşenlerini içerir.

7. Standart Atmosfer Modeli ve Dinamik Basınç Hesabı

Uçuş ortamı parametrelerini hesaplamak amacıyla, yükseklik ve hız bilgilerine bağlı olarak Uluslararası Standart Atmosfer (ISA) modeli kullanılmıştır. Proje Ekleri B"ndeki, Şekil 18'de gösterilen yapıda, verilen irtifa değerinden ortam sıcaklığı (T), ses hızı (a), hava yoğunluğu (ρ) ve statik basınç (P_{static}) hesaplanır. Bu değerler kullanılarak aşağıdaki Denklem 33ile dinamik basınç (P_{dynamic}) elde edilir:

$$P_{\text{dynamic}} = \frac{1}{2}\rho V^2 \tag{33}$$

6 Serbestlik Dereceli Uçuş Dinamiği Modeli

Uçağın altı serbestlik dereceli (6DoF) hareketi, gövde koordinat sisteminde tanımlanan net kuvvet ve moment girdilerine dayalı olarak hesaplanmaktadır. Şekil 19'de gösterilen bu blok, uçağın konum, hız, oryantasyon ve açısal hareketlerini zamana bağlı olarak çıkış olarak üretmektedir.

Girişler şu şekilde gruplanabilir:

- Thrust: Motorlardan gelen itki kuvveti (gövde ekseninde ve X yönünde),
- T_A_F: Toplam aerodinamik kuvvet vektörü (F_{aero}),
- G_Force: Yerçekimi kuvveti vektörü (DCM ile dönüştürülmüş),
- T_A_M: Toplam aerodinamik moment vektörü (M_{aero})

Kuvvet girdileri, vektörel olarak toplanıp toplam net kuvvet F_{xyz} olarak sistemin doğrusal dinamiklerine uygulanır. Moment girdisi M_{xyz} ise açısal hareket denklemlerinde kullanılır.

Çıkışlar şu şekilde gruplanabilir:

- X_e ve V_e – Eylemsizlik Koordinatlarında Konum ve Hız

  X_e vektörü, uçağın dünya (eylemsizlik) koordinat sistemindeki konumunu üç boyutlu olarak verir. Genellikle Doğu-Kuzey-Yukarı (East-North-Up – ENU) eksen takımı kullanılır ve bu koordinatlar metre cinsindendir. V_e ise aynı eksen sisteminde tanımlı olan mutlak hız vektörüdür. Bu hız, gövdeye göre değil, Dünya'ya göre tanımlanmıştır. Uçağın görev bölgesine yönelimi, navigasyon hesapları ve görev planlaması bu çıktılar üzerinden yürütülür.

- V_b ve ω_b – Gövde Koordinatlarında Doğrusal ve Açısal Hız

  V_b, uçağın gövde ekseni sistemindeki (body axes) doğrusal hız bileşenlerini içerir: ileri yöndeki (u), yan yöndeki (v) ve düşey yöndeki (w) hızlar. Bu hızlar, aerodinamik kuvvetlerin hesaplanmasında doğrudan kullanılır. ω_b vektörü ise uçağın gövdesi etrafındaki açısal hızlarını temsil eder: yuvarlanma hızı (p), yunuslama hızı (q) ve sapma hızı (r). Bu büyüklükler, moment denklemlerinin girdileri olup, kontrol sistemlerinin (örneğin LQR, PID) geri beslemelerinde önemli yer tutar.

- ϕ, θ, ψ – Euler Açıları (Roll, Pitch, Yaw) Euler açıları, uçağın yönelimini (attitude) ifade eden üç temel açıdır. ϕ (roll): Gövdenin kendi ekseni etrafında dönmesi (kanatların yatıklığı), θ (pitch): Uçağın burun yukarı-aşağı hareketi, ψ (yaw): Uçağın yönü, yani burunun Kuzey'e göre yönelimi. Bu açılar, DCM"nin hesaplanmasında ve yönelim geri bildirimi gerektiren tüm kontrol algoritmalarında kullanılır. Uçuşun stabilitesi ve yön kontrolü Euler açıları üzerinden tanımlanır.

- $\dot{\omega}_b$ ve A_{bb} – Açısal İvme ve Doğrusal İvme

  $\dot{\omega}_b$, gövde eksen takımı içinde açısal hızın türevidir. Uçağın momentlerine bağlı olarak oluşan açısal ivmeyi temsil eder ve genellikle moment denklemlerinin sağ tarafı üzerinden elde edilir. Kontrol girdilerinin anlık etkilerini gözlemlemek için kullanılır. A_{bb} ise gövde ekseninde tanımlı doğrusal ivme vektörüdür. $F = m \cdot a$ prensibi doğrultusunda toplam kuvvetin kütleye bölünmesiyle elde edilir. Uçuş mekaniği analizinde, özellikle ivmeye dayalı kontrol sistemlerinde (örneğin hız PID döngüleri) önemli rol oynar fakat bu çalışmada kullanılmamıştır.

- DCM – Yön Kosinüs Matrisi

  DCM (Direction Cosine Matrix), gövde koordinat sistemi ile eylemsizlik koordinat sistemi arasındaki dönüşüm matrisidir. Bu matris, birim vektörlerin bir koordinat sisteminden diğerine yöneltilmesini sağlar. Örneğin yerçekimi vektörünü gövde eksenine taşımak için DCM kullanılır. Euler açılarına bağlı olarak sürekli güncellenir ve yönelim bilgisi içeren tüm fiziksel büyüklüklerin dönüşümünde kritik rol oynar. Denklemsel olarak 3×3 boyutlu bir ortonormal matristir.

Bu blok, hem aerodinamik kuvvet/moment girdileriyle hem de dış etkilerle (itki, yerçekimi) birlikte uçağın uçuş halini zamana bağlı olarak hesaplamak için kullanılmaktadır.

B. Modelin Doğrusal Analizi

Otonom bir yer çarpışma önleyici sistemin etkin biçimde çalışabilmesi için uçak modelinin kontrol edilebilir bir yapıda olması gereklidir. Bu nedenle sistemin kontrol tasarımına geçmeden önce, doğrusal kontrol yöntemlerinin uygulanabilmesine olanak tanıyan iki temel aşama gerçekleştirilmiştir: trim analizi ve modelin doğrusal hale getirilmesi. Bu iki adım, dinamik sistemin belirli bir uçuş koşulunda denge noktasının bulunmasını ve bu nokta etrafındaki davranışın analizini sağlar. Bu bölümde ilk başta önce trim (uçuş dengesi) analizi, ardından ise doğrusallaştırma adımı ele alınacaktır.

1. Trim (Uçuş Dengesi) Analizi

Trim, bir uçağın belirli bir uçuş koşulunda — örneğin sabit hız ve sabit irtifada düz uçuş — denge durumunda kalabilmesi için gerekli olan durum değişkenlerini ve kontrol girdilerini bulma işlemidir. Bu işlem sırasında amaç, sistemin türevlerinin sıfırlandığı bir nokta bulmaktır. Böylece uçağın, herhangi bir dış kuvvet veya moment uygulanmadan bu durumu sürdürebileceği varsayılır.

Trim işlemi için kullanılan Simulink modelinde dört adet durum vektörü seti ve üç kontrol girdisi bulunmaktadır. Bu değişkenlere uygulanan kısıtlamalar aşağıda özetlenmiştir:

- Euler açıları: ϕ, θ, ψ bilinen (*sabit*) olarak tanımlanmıştır.
- Açısal hızlar: p, q, r sabittir ancak steady-state özelliği kapalıdır (zamanla değişmesine izin verilir).
- Gövde ekseni hızları: içinde sadece u serbest bırakılmış, v ve w bileşenleri sıfırlanmıştır.
- Konum vektörü: $[x, y, z]$'den yalnızca irtifa (z) sabitlenmiş, yatay eksenlerdeki hareket serbest bırakılmıştır.

Ayrıca kontrol girişleri için de sınırlar tanımlanmıştır:

- Gaz kolu pozisyonu P_c, %90 olarak sabitlenmiş ve $[0, 100]$ aralığına sınırlandırılmıştır.
- Elevatör sapması δ_h sabit tutulmuştur.
- Aileron sapması δ_a sınırlandırılmış ancak sabitlenmemiştir.

Trim işlemi, "Simulink Control Design" aracılığıyla "operspec()" ve "findop()" fonksiyonları ile gerçekleştirilmiş, çözüm için "sqp" algoritması kullanılmıştır. Kodun ilgili bölümü Kod 1'da gösterilmiştir. Bu algoritma, verilen kısıtlar altında sistemin türevlerinin sıfırlandığı denge noktasını ("op") bulur ve sonraki doğrusal analiz adımı için hazır hale getirir.

2. Modelin Doğrusal Hale Getirilmesi (Linearization)

Trim noktası elde edildikten sonra, uçağın doğrusal olmayan dinamik modeli bu denge noktası etrafında doğrusal hale getirilir. Bu işlem, kontrol tasarımında yaygın olarak kullanılan doğrusal kontrol yöntemlerinin (örneğin LQR) uygulanabilmesini sağlar.

Matematiksel olarak, doğrusal hale getirme işlemi Taylor serisi açılımına dayalıdır. Genel bir durum-uzay sistemi şu şekilde ifade edilir:

$$\dot{x} = f(x, u), \qquad y = h(x, u) \tag{34}$$

Burada x durum vektörü, u kontrol girdileri ve $f(x, u)$ sistemin dinamiklerini tanımlayan diferansiyel denklemlerdir. Belirli bir denge noktası (x_0, u_0) etrafında bu sistem doğrusal hale getirilerek aşağıdaki formda bir sistem elde edilir:

$$\Delta\dot{x} = A\Delta x + B\Delta u, \qquad \Delta y = C\Delta x + D\Delta u \tag{35}$$

Buradaki matrisler:

$$A = \left.\frac{\partial f}{\partial x}\right|_{x_0,u_0}, \quad B = \left.\frac{\partial f}{\partial u}\right|_{x_0,u_0}, \quad C = \left.\frac{\partial h}{\partial x}\right|_{x_0,u_0}, \quad D = \left.\frac{\partial h}{\partial u}\right|_{x_0,u_0} \tag{36}$$

Bu lineer sistem, denge noktası etrafındaki küçük sapmalar için orijinal sistemin yaklaşık bir temsilcisidir. Bu projede doğrusal hale getirme işlemi, "Simulink" ortamında tanımlı "f16_Trim" modeli kullanılarak yapılmıştır. İlk olarak sistemin giriş-çıkış (I/O) noktaları "getlinio()" komutu ile alınmış ve daha sonra "linearize()" fonksiyonu ile sistem doğrusal hale getirilmiştir. Kodun ilgili kısmı Kod 1'da verilmiştir. Doğrusal sistem daha sonra hem "yanal (roll)" hem de "boylamsal (pitch)" hareket bileşenleri açısından indirgenmiştir. Bu amaçla "modred()" fonksiyonu ile ilgisiz durum değişkenleri elenmiş ve kontrol tasarımına uygun iki ayrı alt sistem (roll ve pitch) elde edilmiştir. Son olarak, bu sistemlerin kontrol edilebilirlikleri "ctrb()" komutu yardımıyla denetlenmiştir.

C. Kontrol Sistemi Tasarımı (Alt Kontrolcü)

Uçağın istenen görevleri güvenli ve kararlı bir şekilde yerine getirebilmesi için kontrolcü tasarımı büyük önem taşır. Bu kısımda, trimlenmiş ve doğrusal hale getirilmiş sistem üzerinde, uçuş dinamiklerinin üç bileşeni için (hız, yanal ve boylamsal hareket) ayrı kontrol döngüleri tasarlanmıştır. Her kontrol döngüsü, ilgili sistem davranışını izlemek ve istenen referanslara ulaşmak için farklı kontrol tekniklerinden yararlanır.

1. Hız Kontrolü (Velocity Control)

Hız kontrolü, uçağın ileri yönlü (longitudinal) hızını sabit tutmak ya da istenen bir değere çıkarmak amacıyla yapılır. Bu kontrol döngüsü, uçağın throttle (gaz kolu) komutunu yani "pc" kontrol girişini üretir. Sistemin bu kısmı, klasik bir oransal-integral-türevsel (PID) kontrol yapısı ile modellenmiştir. PID kontrolörü, hız hatası $e(t) = V_{ref} - V$ cinsinden tanımlanarak üç farklı bileşene ayrılır:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt} \tag{37}$$

Burada:

- K_p: Oransal kazanç – sistem tepkisini artırır,
- K_i: İntegral kazanç – sürekli hatayı yok eder,
- K_d: Türevsel kazanç – ani değişimlere tepki verir.

Hız kontrol döngüsünde bu PID çıkışı, "pc" girişine uygulanır ve throttle değerini belirler. Sistem böylece istenen ileri hız seviyesini korumaya çalışır. Bu yapı, MATLAB/Simulink ortamında "velocity controller" bloğu içerisinde gerçekleştirilmiştir. İlgili yapı Şekil 22'de görülmektedir.

2. Boylamsal Kontrol (Longitudinal Control)

Boylamsal eksende kontrol, uçağın "pitch" hareketinin ve irtifasının kararlı biçimde istenen referanslara ulaşmasını sağlar. Bu eksende kontrol iki kademeli olarak yapılandırılmıştır: "iç kontrol döngüsü", uçağın pitch açısını kontrol ederek "elevator" (dh) kontrol girişini üretirken; "dış kontrol döngüsü", irtifa (h) referansını sağlayarak iç kontrol için gerekli pitch referansını oluşturur.

İç Kontrol Döngüsü: LQR ile Pitch Kontrolü

İç kontrol döngüsü, pitch açısını (θ) referansa göre denetlemek amacıyla LQR (Linear-Quadratic Regulator) tabanlı bir kontrolcü kullanır. Trimlenmiş ve doğrusal hale getirilmiş boylamsal sistem için LQR yöntemiyle optimal bir geri besleme kazancı K elde edilmiştir. LQR kontrol yapısında maliyet fonksiyonu şu şekilde tanımlanır:

$$J = \int_0^\infty \left(x^\top Q x + u^\top R u\right) dt \tag{38}$$

Burada:

- x: sistem durumu vektörü (örneğin, pitch açısı ve pitch hızı),
- u: kontrol girdisi (elevator),

- Q, R: durum ve kontrol ağırlık matrisleridir.

Bu yöntemle elde edilen K kazanç matrisi, sistem durumları üzerinden hesaplanan hataya göre kontrol girişi üretir:

$$u = -Kx \tag{39}$$

İlgili kazançlar ve sistem yapısı, doğrusal hale getirilmiş "linsys_pitch" modeli kullanılarak hesaplanmıştır. Bu süreç Kod 1'de gerçekleştirilmiştir ve sistem içindeki "lqr_pitch" bloğu ile uygulanmaktadır. İlgili blok şeması Şekil 23'de gösterilmiştir.

Dış Kontrol Döngüsü: İrtifa Takibi ve Rehberlik Entegrasyonu

Boylamsal eksende dış kontrol döngüsü, uçağın hedef irtifasını koruyabilmesi amacıyla Şekil 24'de gösterildiği üzere PID tabanlı bir kontrolcü ile gerçekleştirilmiştir. Bu kontrolör, mevcut irtifa ile istenen irtifa arasındaki farkı hesaplayarak uygun bir pitch referansı üretir. Elde edilen bu referans, iç LQR kontrol döngüsüne iletilerek sistemin elevatör (dh) kontrol girişi ayarlanır.

Ancak bu yapı, sistemin rehberlik (guidance) bloğuna bağlı olarak dinamik şekilde değiştirilebilir. Tehlikeli bir durum algılandığında (örneğin, çarpışma riski), PID kontrolcü devre dışı bırakılır ve rehberlik sistemi tarafından üretilen acil pitch komutu doğrudan LQR kontrol döngüsüne aktarılır. Bu sayede sistem, hem normal uçuşta irtifa koruması sağlar hem de acil durumlarda hızlı tepki verebilir. Bu yapı, Simulink modelinde "altitude_hold" bloğu ve "guidance_pitch_override" mantığı ile sağlanmıştır. "Stateflow" tabanlı rehberlik yapısı, tehlike algıladığında doğrudan LQR pitch girişine sinyal yönlendirerek sistemi override eder. Bu kontrol stratejisi sayesinde sistem, normal koşullarda yüksek hassasiyetli irtifa kontrolü sağlarken, kritik durumlarda rehberlik temelli kaçış manevralarına anında geçiş yapabilir.

3. Yanal Kontrol (Roll Control)

Roll ekseninde yapılan kontrol, uçağın yönelmesini belirlemek ve dengede tutmak için gereklidir. Bu kontrol yapısı da tıpkı boylamsal eksende olduğu gibi iki katmanlıdır. İç döngü doğrudan aileron (kanatçık) kontrolünü sağlarken, dış döngü ise sistemin yönlenme komutlarını oluşturur.

İç Kontrol Döngüsü: LQR ile Roll Kontrolü

İç kontrol döngüsünde, doğrusal hale getirilmiş sistem üzerine uygulanan LQR yapısı ile roll açısı "ϕ" doğrudan kontrol edilir. LQR kontrolcüsü, phi açısını sıfıra yaklaştırmak amacıyla sistemin aileron girişini düzenler. Bu yapı, Kod 1'de oluşturulmuş ve Simulink modelinde Şekil 25'deki bloğu ile uygulanmıştır. Bu sayede sistem, rehberlik veya dış referanslardan gelen phi komutuna karşılık stabil ve hızlı bir şekilde roll ekseninde tepki verebilir. Bu kontrolcünün referans girdisi, rehberlik sistemi tarafından oluşturulmuş bir phi komutudur. Sistem, örneğin bir tehlike durumunda sola veya sağa kaçma gibi manevralar gerçekleştirmek istediğinde, rehberlik bloğunda hesaplanan phi referans açısı doğrudan iç LQR kontrol döngüsüne gönderilir.

Dış Kontrol Döngüsü: Yönelim Takibi

Alternatif olarak sistemde manuel olarak aktive edilebilen bir dış kontrol döngüsü de mevcuttur. Bu döngüde Şekil 26'de de görülebileceği üzere, PID kontrolcü kullanılarak heading (baş yönü) takibi gerçekleştirilir. PID kontrolcü, hedef heading değerini alarak uygun bir ϕ açısı üretir ve bu ϕ referansı LQR bloğuna iletilir. Bu yapı, Simulink modelinde yer alan manuel switch bloğu ile aktif veya pasif hale getirilebilir. Böylece kullanıcı, sistemin rehberlik temelli kaçış modunda mı yoksa standart yönelme kontrolünde mi çalışacağını dilediği zaman belirleyebilir.

İç ve dış döngüye ait kontrol şemaları ve sinyal akışları, ekler bölümünde yer alan ilgili şekillerde gösterilmiştir. Bu yapılar, sistemin hem otonom kontrol modlarında hem de kullanıcıya bırakılmış esnek kontrol yapılarında sağlıklı çalışmasını mümkün kılar.

D. Otopilot Sistemi Tasarımı (Üst Kontrolcü)

Modern savaş uçaklarında görev yükünü azaltmak ve emniyetli uçuşu sağlamak amacıyla geliştirilen otopilot sistemleri, pilot müdahalesine gerek kalmaksızın uçağın belirli uçuş modlarını yerine getirmesine olanak tanır. Bu projede geliştirilen F-16 otopilot sistemi, görev odaklı yönlendirme (guidance) davranışı ile yer çarpışmalarını önleyici kaçış manevralarını (GCAS) aynı yapı altında birleştiren hibrit bir yaklaşıma dayanmaktadır. Sistemin yüksek seviyeli kontrol mantığı ise "Stateflow" tabanlı bir karar yapısıyla yürütülmektedir. Otopilot mimarisinin genel yapısı Şekil 28'de sunulmaktadır. Tasarlanan otopilot sistemi iki ana kontrol mekanizması içerir:

- Güdüm (Guidance): Uçağın belirli yol noktalarını (waypoints) takip ederek görev sahasında otomatik olarak ilerlemesini sağlayan güdüm sistemi. Normal operasyon sırasında aktif olan bu mod, "carrot chasing" algoritmasına dayalı yönelim kontrolü sağlar.
- GCAS (Ground Collision Avoidance System): Uçuş sırasında çarpışma riski oluştuğunda devreye giren kaçış sistemi. Uçak önceden tanımlı kaçış manevralarını simüle ederek araziyle potansiyel çakışmaları tespit eder ve güvenli yönü belirleyerek acil kaçış komutu üretir..

"Stateflow" modülü ise yukarıdaki iki kontrol yapısı arasında geçişi sağlar. Yani sistemin hangi anda "güdüm" ve hangi anda "kaçış" davranışı sergileyeceği, çarpışma analizine bağlı olarak bu karar mekanizması tarafından belirlenir. Dolayısıyla "Stateflow" yapısı doğrudan bir kontrolcü değil, koşul tabanlı mod seçiminde görev alan mantıksal geçiş düzenleyicisidir. Bu yapı sayesinde sistem, normal şartlarda rota takip eden bir guidance davranışı sergilerken, araziyle çarpışma riski belirdiğinde GCAS moduna geçer; tehlike ortadan kalktığında ise guidance moduna geri döner. Bu yapının ilk bileşeni olan *Guidance* sistemi, bir sonraki alt bölümde matematiksel temeli ve algoritmasıyla birlikte detaylandırılacaktır.

1. Guidance Alt Sistemi

Güdüm sisteminde kullanılan "Havuç Takibi (Carrot Chasing)" algoritması, takip edilecek yol noktaları (waypoints) arasında sanal bir hedef belirleyerek hava aracının yönelmesini sağlayan basit ama etkili bir güdüm yöntemidir. Bu algoritmada uçak, önceden tanımlı yol noktaları arasındaki düz segmentleri izlerken, her zaman belirli bir mesafe ötesinde yer alan "havuç" (carrot) noktasına yöneltilir. Böylece uçağın yumuşak ve kararlı bir takip davranışı sergilemesi sağlanır. Algoritmanın temel amacı, uçuş halindeki uçağın $P = (x, y)$ konumundan itibaren, $W_1 = (x_1, y_1)$ ve $W_2 = (x_2, y_2)$ yol noktalarını bağlayan düz bir hatta ait olan bir hedef noktasına yönelim açısı üretmektir. Bu hedef nokta, W_1 başlangıç noktasından itibaren $R + \delta$ kadar ilerde konumlandırılır. Burada R, uçağın bu hatta olan izdüşüm mesafesidir ve δ ise sistem parametresi olan sabit hedef mesafesidir.

Yol segmentine ait yön açısı θ aşağıdaki şekilde hesaplanır:

$$\theta = \tan^{-1}\left(\frac{y_2 - y_1}{x_2 - x_1}\right) \tag{40}$$

Uçağın mevcut konumunun, segment üzerindeki izdüşümünün W_1'e olan mesafesi R olarak tanımlanır. Bu değer, trigonometrik ilişki ile şu şekilde yazılır:

$$R = \sqrt{R_u^2 - (R_u \sin\beta)^2} \tag{41}$$

Burada R_u uçağın W_1 noktasına olan Öklidyen mesafesidir ve β sapma açısıdır:

$$R_u = \sqrt{(x - x_1)^2 + (y - y_1)^2} \tag{42}$$

$$\beta = \theta - \theta_u = \theta - \tan^{-1}\left(\frac{y - y_1}{x - x_1}\right) \tag{43}$$

Sanal hedefin (carrot) koordinatları, segment yönü boyunca $R + \delta$ kadar ilerdeki nokta olarak hesaplanır:

$$x_{\text{target}} = x_1 + (R + \delta)\cos\theta \tag{44}$$

$$y_{\text{target}} = y_1 + (R + \delta)\sin\theta \tag{45}$$

Uçak bu noktaya yönelmek için gerekli olan yönelme açısını (hedef yaw) şu şekilde hesaplar:

$$\psi_d = \tan^{-1}\left(\frac{y_{\text{target}} - y}{x_{\text{target}} - x}\right) \tag{46}$$

Yönelme hatası $\Delta\psi$ bu hedef açı ile mevcut yaw (ψ) arasındaki farktan elde edilir. Nihayetinde kontrol komutu olan yaw dönüş hızı şu şekilde oransal kontrol ile tanımlanır:

$$r_{\text{cmd}} = k(\psi_d - \psi) \tag{47}$$

Burada k sistemin tepki kazancıdır ve tipik olarak $k \in [0.01, 0.1]$ aralığında seçilir.

Algoritma, uçağın aktif segment üzerinde ilerleyip yeni bir waypoint'e yaklaştığını tespit ettiğinde segmenti günceller ve bir sonraki yol parçasına geçer. Bu sayede uçak görev boyunca tüm rotayı takip eder. Bu algoritmanın MATLAB uygulaması Kod 4'de sunulmuştur. Simulink blok yapısı ise de Şekil 30'de sunulmuştur.

2. *Yer Çarpışma Önleyici Sistem*

Yer çarpışma önleyici sistem (Ground Collision Avoidance System - GCAS), uçağın mevcut uçuş durumuna göre çevresindeki araziden kaçış rotalarını önceden hesaplayarak, olası bir çarpışma durumunda otonom manevra gerçekleştirmesini amaçlar. Bu sistem, özellikle pilotun fark edemeyeceği riskli anlarda devreye girerek güvenliği maksimize eder.

Kaçış Yörüngelerinin Çizilmesi

GCAS kapsamında geliştirilen yaklaşımda, uçağın sabit bir mesafede arka alt kısmından tanımlanan bir referans noktasından itibaren, sabit bir n-g yüklemesi ile belirli pitch (θ) ve yaw (ψ) açılarında manevra yapılacağı varsayılır. Bu bilgilerden yararlanarak üç boyutlu kaçış yörüngeleri oluşturulur. Uçak, belirli bir n-g yüklemesi ile pull-up manevrası yaparken dairesel bir yay izler. Bu yay, aşağıdaki gibi tanımlanır:

$$\begin{aligned} R &= \frac{V^2}{g \cdot n} \\ X_c &= X_0 - R \cdot \sin(\theta) \\ Y_c &= Y_0 \\ Z_c &= Z_0 + R \cdot \cos(\theta) \end{aligned} \tag{48}$$

Burada R manevra yarıçapıdır, X_0, Y_0, Z_0 uçağın mevcut konumudur ve (X_c, Y_c, Z_c) kaçış manevrasının merkez noktasıdır. Uçuş düzlemine dik bir vektör kümesi bulunarak, dairesel yay parametreleri hesaplanır. Parametrik olarak:

$$\mathbf{p}(t) = \mathbf{c} + R \cdot (\mathbf{v}_1 \cdot \cos(t) + \mathbf{v}_2 \cdot \sin(t)), \quad t \in \left[-\theta - \frac{\pi}{2}, -\pi\right] \tag{49}$$

Burada $\mathbf{v}_1, \mathbf{v}_2$ düzleme dik yönlendirme vektörleridir. Bu yay, uçuş yönüne (yaw açısı ψ) göre hizalanmak üzere Z ekseni etrafında döndürülür. Döndürme matrisi:

$$D_\psi = \begin{bmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{50}$$

Bu dönüş, Z eksenine paralel olup, yay noktaları bu matrisle çarpılarak ileri yöne hizalanır. Son aşamada, yaw'a göre hizalanmış yaylar ileri yön vektörü etrafında $-90°$, $0°$, $+90°$ olacak şekilde döndürülerek, farklı yönlerde üç boyutlu kaçış yörüngeleri üretilir. Bu dönüş, Rodrigues'in döndürme formülü ile gerçekleştirilir:

$$R_\phi = \mathbf{I} \cdot \cos\phi + (1 - \cos\phi) \cdot \mathbf{u}\mathbf{u}^T + \sin\phi \cdot [\mathbf{u}]_\times \tag{51}$$

Burada $\mathbf{u}$ ileri yön eksenini temsil eder ve $[\mathbf{u}]_\times$ çapraz çarpım operatör matrisidir. Bu hesaplamalar sonucunda her bir G limiti için üç ayrı yön (up, right, left) olmak üzere toplam altı adet kaçış yörüngesi elde edilir. Hesaplamalar, Kod 5 üzerinden gerçekleşmiştir.

Arazi Datasının Oluşturulması ve Modele Entegre Edilmesi

Yer çarpışma önleyici sistemin (GCAS) etkin şekilde çalışabilmesi için gerçek veya sentetik bir arazi haritası (terrain map) üzerinde çarpışma kontrolü yapılması gerekmektedir. Bu çalışma kapsamında, MATLAB ortamında üç boyutlu sentetik bir arazi haritası oluşturulmuştur. Arazi modeli, hem ana topografik yapıyı tanımlayan dağ sıralarını hem de rastgele dağıtılmış küçük tepeleri içerecek şekilde tasarlanmıştır. Oluşturulan yüzey, GCAS sistemi tarafından hesaplanan kaçış yörüngeleri ile sürekli olarak çakıştırılarak olası çarpışmalar belirlenmekte ve karar algoritmasına veri sağlanmaktadır.

Arazi haritası, `meshgrid` komutu ile X ve Y düzleminde tanımlı 9000×9000 boyutlarında bir grid üzerine yerleştirilmiştir. Haritanın çözünürlüğü 1 metredir:

$$X, Y = \texttt{meshgrid}(0:1:9000,\ 0:1:9000) \tag{52}$$

Ana coğrafi yapılar, $\mathcal{N}(\mu, \sigma^2)$ benzeri iki boyutlu Gauss fonksiyonlarıyla modellenmiştir. Üç büyük dağ sırası şu şekilde tanımlanmıştır:

$$Z_{\text{main}} = A \cdot e^{-\frac{(X-\mu_x)^2}{2\sigma_x^2}} \cdot e^{-\frac{(Y-\mu_y)^2}{2\sigma_y^2}} \tag{53}$$

Bu denklemde A maksimum yükseklik, (μ_x, μ_y) dağın merkezi ve σ yayılma katsayısıdır. Üç ayrı merkezde bu formül kullanılarak dağ sıraları oluşturulmuştur. Araziyi daha gerçekçi hale getirmek için rastgele sayıda ve konumda daha küçük Gauss tipi tepeler oluşturulmuştur. Her bir küçük tepe için rastgele yükseklik h_i, konum (x_i, y_i) ve yayılma katsayısı σ_i tanımlanmıştır:

$$Z_{\text{hill}}^i = h_i \cdot e^{-\frac{(X-x_i)^2+(Y-y_i)^2}{2\sigma_i^2}}, \quad i = 1, 2, \ldots, 100 \tag{54}$$

Toplam küçük tepe verisi, bu 100 tepenin üst üste toplanması ile elde edilmiştir:

$$Z_{\text{small}} = \sum_{i=1}^{100} Z_{\text{hill}}^i \tag{55}$$

Son olarak, arazi yüksekliği Z_{map}, büyük dağlar ve küçük tepelerin toplamından oluşur:

$$Z_{\text{map}} = Z_{\text{main}} + Z_{\text{small}} \tag{56}$$

Bu harita, GCAS sistemi tarafından sürekli olarak kullanılan ve kaçış yörüngeleriyle karşılaştırılan üç boyutlu arazi verisidir. Harita istenirse "surf" komutu ile görselleştirilebilir. Ancak, simülasyon performansı açısından görselleştirme kodu yorum satırında bırakılmıştır. Kodun tamamı Kod 2 içinde verilmiştir.

Çarpışma Algılama Sistemi

Yer çarpışma önleyici sistemin (GCAS) temel amacı, uçağın mevcut uçuş rotasının önceden tanımlanmış bir arazi haritasıyla çakışıp çakışmadığını gerçek zamanlı olarak tespit etmektir. Günümüzde gelişmiş GCAS sistemleri, bu kontrolü yalnızca tek bir kaçış rotası üzerinden değil, birden fazla potansiyel kaçış manevrasının eş zamanlı analizi ile gerçekleştirmektedir.

Bu çalışmada da benzer şekilde, her anlık uçuş durumu için toplam altı farklı manevra değerlendirmeye alınmaktadır. Bu manevralar; üçü yumuşak ($3.5g$) ve üçü sert ($4.5g$) olacak şekilde sağ, sol ve yukarı (pull-up) yönlerde planlanmaktadır. Bu yaklaşımın tercih edilmesinin temel nedeni, karar verme esnekliğini artırarak gereksiz manevraların önüne geçmektir.

Örneğin, dar bir vadiden geçiş senaryosunda yalnızca sabit bir G-limitine sahip kaçış rotaları tanımlanmış olsaydı; sağ ve sol manevralar araziyle çakıştığı durumda sistem yalnızca yukarı (pull-up) manevrasına yönelmek zorunda kalacaktı. Ancak bu durumda uçak, önünde açık bir güzergâh olmasına rağmen gereksiz şekilde tırmanma yaparak stabil uçuşu bozmuş olacaktı.

Buna karşılık, sistemin farklı G limitlerinde (örneğin $3.5g$ ve $4.5g$) planlanmış aynı yönlü manevraları karşılaştırması sayesinde, ilk etapta sadece $3.5g$ manevralar değerlendirilir ve yalnızca bu rotalar tehlikeye girerse daha sert $4.5g$ manevralar dikkate alınır. Böylece kaçınılmaz durumlara yalnızca gerektiğinde müdahale edilerek, sistemin operasyonel doğruluğu ve güvenliği önemli ölçüde artırılmış olur.

Bu çoklu manevra stratejisi, sistemin gerçek zamanlı karar mekanizmasında esneklik sağlayarak gereksiz kaçışları önlerken, aynı zamanda son kalan güvenli manevranın uygulanmasını mümkün kılar. Böylece GCAS sisteminin uçuş güvenliğine katkısı maksimize edilir. Çarpışma kontrol fonksiyonu (Kod 6) temel olarak aşağıdaki üç adımı uygular:

1. Yörünge Konumlarının Harita Üzerindeki İndisleri: Yörünge veri setinden alınan (x, y) koordinatları, arazi matrisi `Z_map` içinde karşılık gelen satır ve sütun indislerine çevrilir:

$$\text{row} = \text{int}(y - \min(y)) + 1, \quad \text{col} = \text{int}(x - \min(x)) + 1 \tag{57}$$

2. Sınır Dışı Kontrolü: Yörüngenin herhangi bir noktası arazi haritasının sınırları dışına çıkarsa bu bir "stop" durumu olarak kabul edilir. Bu durumda çarpışma kontrolü yapılmadan tüm değerler `collision = 1` olarak atanır:

$$\text{if } \max(\text{row}) \geq \text{size}(Z_map) \Rightarrow \text{collision} = \mathbf{1} \tag{58}$$

3. Yörünge-Arazi Kesişimi: Harita yüksekliği ile yörünge yüksekliği karşılaştırılarak çarpışma olup olmadığı belirlenir. Her bir noktada:

$$\text{collision}_i = \begin{cases} 1 & \text{if } Z_{\text{terrain}} \geq Z_{\text{trajectory}} \\ 0 & \text{otherwise} \end{cases} \tag{59}$$

   Burada lineer indeksleme (`sub2ind`) kullanılarak hızlı veri erişimi sağlanmıştır.

Buna karşılık fonksiyonun iki temel çıktısı bulunmaktadır:

- stop – Yörünge sınır dışına çıkarsa 1 olur.
- collision – Her nokta için çarpışma bilgisi $(1/0)$ içeren vektör.

Bu veri, GCAS karar mantığında kullanılan altı ayrı manevra için ayrı ayrı hesaplanır. Simulink içerisinde bu yapı Şekil 31'ta gösterildiği gibi her manevra için ayrı bloklarda değerlendirilmiştir.

3. Durum Akışı (State Flow) Yapısı

Bu bölümde, yer çarpışma önleyici sistemin (GCAS) en kritik bileşenlerinden biri olan karar tabanlı Stateflow diyagramı detaylı biçimde açıklanmaktadır. Sistem genel olarak üç ana durum çevresinde çalışmaktadır: "Waypoint-Following", "Danger" ve "Recover". Her bir durumda uygulanacak kontrol komutları ve geçiş şartları belirlenmiştir.

WaypointFollowing durumu, sistemin normal uçuş modunu temsil eder. Bu durumda uçak, guidance modülü tarafından belirlenen rota boyunca ilerlemeye devam eder. Bu durumun girişinde "escape_route" sıfırlanır ve "pitch_command" sıfır olarak ayarlanır. Ayrıca "bank_command" değişkeni guidance tarafından belirlenen "GuidedBankCommand" sinyaline atanır. Boylamasına kontrol modu "(lon_mode)" aktif durumdadır.

Bu moddan Danger moduna geçiş şartı, toplam çarpışma riskini gösteren "Collisions" vektöründeki değerlerin toplamının 5 olmasıdır. Yani 6 farklı kaçış rotasından yalnızca biri geçerli kaldığında, sistem acil durum moduna geçer.

Danger durumunda sistem, hangi kaçış manevrasının uygulanacağını belirler. Entry satırında "escape_route = sum(find(Collisions == 0))" şeklinde tanımlanmıştır. Bu ifade, çarpışma içermeyen yörünge indekslerinin toplamını verir. Bu sayıya bağlı olarak sistem, uygun kaçış durumuna yönlendirilir:

- escape_route == 4 durumunda sistem "Pull_Up" durumuna geçer. Bu durumda "pitch_command" = pi/2.5, "bank_command = 0" olarak atanır. Yani uçak doğrudan yukarı doğru yüksek G manevrası uygular.
- "escape_route == 5" ise" Level_Right" durumuna geçilir. Bu durumda "pitch_command" sıfır, "bank_command = pi/3" olur. Uçak sağa yatış yapar.

- "escape_route == 6" durumunda sistem "Level_Left" moduna geçer. Bu modda "pitch_command" = -pi/2.5 ve "bank_command" = 0'dır. Bu durumda uçak sola yatış yaparak manevra başlatır.

Her bir kaçış durumu sonrası sistem çarpışmanın ortadan kalkıp kalkmadığını kontrol eder. Eğer çarpışma ortadan kalktıysa ve belirli bir zaman geçtiyse sistem "Recover" durumuna geçer.

Recover durumu, kaçış sonrası sistemin normale dönmeden önce stabilizasyon sürecini geçirdiği bölgedir. Burada tüm komutlar sıfırlanır ve sistem 7 saniye boyunca çarpışmasız bir şekilde ilerlediyse yeniden "WaypointFollowing" moduna geçilir. Bu geçişin şartı: "Collisions(6)" == 0 ve "Collisions(3)" == 0 ve "after(7, sec)" ifadesiyle gösterilmiştir.

Bu yapı, yalnızca ani tepki veren bir sistem değil, aynı zamanda çarpışma tehdidi ortadan kalktığında da kontrollü biçimde normale dönebilen bir sistem mantığı sunmaktadır. "WaypointFollowing" ve "Recover" arasında güvenli geçişler sağlanarak gereksiz kaçış manevralarının önüne geçilmiş olur. Tüm bu geçişlerin yaşandığı blok Şekil 29'de sunulmuştur.

E. Uçuş Görselleştirme ve Animasyon

Bu çalışmada uçuş simülasyonunun görsel takibi için iki farklı sistem kullanılmıştır: FlightGear ve MATLAB tabanlı "flypath3d" paketi. Simülasyon ortamında elde edilen konum ve Euler açı verileri kullanılarak oluşturulan veri seti, önce görselleştirme formatına dönüştürülmüş, ardından uçuş yolu, animasyon ve uçak modeli üç boyutlu ortamda canlandırılmıştır.

MATLAB ortamında geliştirilen sistemde, uçuş verisi simülasyon sonrasında işlenerek bir 3D uçak nesnesine atanmış, ardından bu nesne üzerinden uçuş yolu animasyonu gerçekleştirilmiştir. Bu animasyon, ilgili uçuş sahnesiyle birlikte harita üzerine yansıtılarak hem uçuş izi hem de manevraların analizi mümkün kılınmıştır. Kullanılan görselleştirme araçları sayesinde uçak hareketleri ve arazideki çarpışma potansiyelleri detaylı biçimde izlenebilmiştir.

Hazırlanan görsel sonuçlar, sentetik arazi haritasının üzerine bindirilen uçuş yörüngesiyle beraber sunulmuş, seçilen uçuş senaryosuna ait örnek manevralar ve kaçış yolları da aynı anda grafiksel olarak değerlendirilmiştir.

İlgili "flypath3d" kod yapısı Kod3 betiğine dayalıdır[7]. Flight Gear hakkında ayrıntı için ise eklerde verilen Şekil 32 blok diyagramına başvurulabilir.

V. Çıktılar ve Sonuç

Bu çalışmada geliştirilen otonom Yer Çarpışma Önleyici Sistem (GCAS), farklı kaçış manevralarını gerçek zamanlı olarak değerlendiren çoklu yörünge (multi-trajectory) stratejisine dayanmaktadır. Sistemin performansı, hem uçuş dinamikleri hem de karar verme süreci açısından MATLAB Simulink ortamında ve "FlightPath3d" görselleştirme araçlarıyla analiz edilmiştir. Bu bölümde, sistemin çıktıları detaylı olarak sunulmuş ve yorumlanmıştır. Ek olarak belirtmek gerekirse, "5g pull-up" manevrası grafikleri "güdüm" sisteminin başarısını kanıtlamak için başlangıç noktasına geri döndürdüğümüz uçağın hareketlerinden ötürü belli bir süre sonra aslında beklenen ama sadece "pull-up" manevrası bekleyen birini şaşırtıp sistemin başarısız olduğunu düşünmelerine sebep olabileceğinden ötürü bunun farkında olunup grafiklerin bu doğrultuda yorumlanması gerekmektedir. Buna ek olarak raporu uzatmamak adına kritik olduğu düşünülen çıktılar yorumlanmıştır.

1. Yörünge Takibi ve Görselleştirme

Öncelikle, geliştirilen sistemin kaçış manevrası üretme modülünün doğruluğu ve gerçekçiliği analiz edilmiştir. Şekil 36'de gösterilen yukarı kaçış manevrasına ait konum (X_e) çıktıları incelendiğinde, manevranın istikrarlı şekilde yukarı yönlü bir tırmanış profili izlediği görülmektedir. Bu durum, 5g'lik bir ani kaçış kararının doğru bir şekilde modellenebildiğini göstermektedir. Aynı zamanda, hem yatay hem de düşey eksenlerdeki yumuşak geçişler, sistemin ani fakat kontrol edilebilir manevralar üretebildiğini ortaya koymaktadır. Buna ek olarak Şekil 40 ve Şekil 44'deki pozisyon değişimlerini aynı şekilde sistemin tutarlılığı doğrultusunda yorumlamak hiç yanlış olmaz.

2. Euler Açıları ve Uçuş Duruşu

Şekil 37'de sunulan Euler açıları (yaw, pitch, roll), kaçış sırasında uçağın burun yukarı (pitch pozitif) ve roll açısından dengeli (yaklaşık sıfır) bir duruş koruduğunu göstermektedir. Bu veriler, sistemin bank açısız bir yukarı kaçış manevrası komutladığını ve bu komutun uçuş dinamiklerine başarıyla aktarıldığını doğrular. Aynı şekilde Şekil 41 ve Şekil 45'in de burun yukarı açısının sıfır olması manevrayı ne kadar iyi yaptığını kanıtlar nitelikte.

3. Açısal Hızlar ve G-İvmelenme Tepkisi

Şekil 38'de yer alan açısal hızlar (p, q, r) grafiklerinde, özellikle q (pitch rate) bileşeninin pozitif değerlerde keskin artışı dikkat çekmektedir. Bu durum, 5g'lik bir burun yukarı manevrasının ani ivmelenme ile uygulandığını göstermektedir. Ancak p ve r bileşenlerinin sıfıra yakın kalması, uçağın sağ-sol dengesini koruyarak simetrik bir kaçış rotası izlediğini kanıtlar niteliktedir. Bu çıktılar, kontrol komutlarının doğru biçimde aktarıldığını ve uçağın fiziksel davranışlarının modelle örtüştüğünü göstermektedir.

4. Göreli Hızlar ve Dinamik Uyum

Şekil 39'de gösterilen gövdeye göre hız bileşenleri (u, v, w) zamanla değişen profiller üretmektedir. u (ileriye doğru hız) bileşeni azalarak, yukarı yönlü çekişin uçağı yavaşlattığını göstermektedir. Benzer şekilde w bileşenindeki artış, düşey eksende yukarı yönlü hızlanmayı doğrulamaktadır. Bu sonuçlar, uçağın tırmanış dinamiğiyle uyumlu biçimde hız vektörünün yön değiştirdiğini ve sistemin buna uygun kaçış algoritması yürüttüğünü ortaya koymaktadır.

5. Sağ ve Sol Kaçış Senaryoları

Aynı simülasyon ortamında elde edilen sağ(Şekil 34) ve sol(35) düz dönüş manevralarında ise sistemin roll komutları ile birlikte pitch komutlarını hassas biçimde uyguladığı gözlemlenmiştir. Manevralarda çarpışma riski ortadan kalktıktan sonra sistem, `Recover` durumuna geçerek tekrar waypoint takibine dönebilmiş ve stabil uçuş profiline ulaşmıştır. Bu geçişler, Stateflow diyagramında modellenen karar mantığının doğru çalıştığını kanıtlamaktadır.

6. Genel Başarı Değerlendirmesi

Sonuç olarak, geliştirilen sistem aşağıdaki hedefleri başarıyla yerine getirmiştir:

- Altı farklı kaçış yörüngesinin aynı anda değerlendirilmesi sayesinde yalnızca "pull-up" manevrasına mahkûm kalınmamış, risk bazlı en geçerli kaçış seçilmiştir.
- Anlık çarpışma riski belirlenebilmiş ve sistem duruma göre manevraları devreye alarak emniyetli kaçış sağlamıştır.
- Uygulanan manevralar hem fiziksel sınırlar içinde kalmış hem de uçak dinamikleri ile tutarlı sonuçlar üretmiştir.
- FlightGear ve MATLAB Flightpath3D yardımıyla görsel animasyonlar elde edilerek doğrulama süreci tamamlanmıştır.

Bu bağlamda, çalışmada geliştirilen GCAS yapısının hem modelleme hem de karar alma açısından gerçekçi, güvenilir ve operasyonel açıdan kullanılabilir olduğu görülmektedir. Özellikle dağlık araziler gibi kompleks senaryolarda sistemin performansı gelecek çalışmalarda test edilerek daha da geliştirilebilir.

References

[1] Deets, D., Foster, J., et al., "Automatic Ground Collision Avoidance System Design, Integration, & Flight Test," *NASA Publications*, 2014.

[2] Nadtochiy, P. and Fedotov, A., "Modified Algorithm of Predictive Ground Collision Avoidance System for the Low Level Terrain Flight," *Journal of Aerospace Computing, Information, and Communication*, 2018.

[3] Suplisson, W., *Optimal Recovery Trajectories for Automatic Ground Collision Avoidance Systems (Auto GCAS)*, Ph.D. thesis, Dept. of Aeronautics and Astronautics, AFIT, Wright-Patterson AFB, OH, 2015.

[4] Swihart, D. E., Barfield, A. F., Griffin, E. M., Lehmann, R. C., Whitcomb, S. C., Flynn, B., Skoog, M. A., and Processor, K. E., "Automatic Ground Collision Avoidance System Design, Integration, & Flight Test," *IEEE Aerospace & Electronic Systems Magazine*, May 2011.

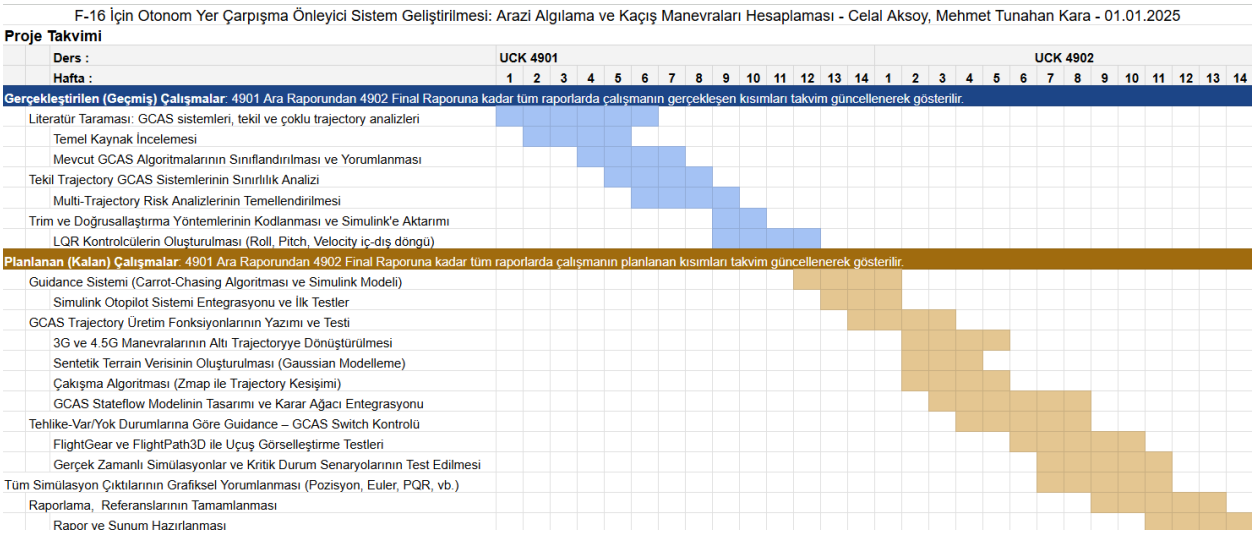
[5] Wang, X. and colleagues, "Ground collision avoidance system with multi-trajectory risk assessment and decision function," *International Journal of Aerospace Engineering*, 2020.

[6] Nguyen, L. T., Ogburn, M. E., Gilbert, W. P., Kibler, K. S., Brown, G. V., and Deal, P. L., "Simulator Study of Stall/Post-Stall Characteristics of a Fighter Airplane With Relaxed Longitudinal Static Stability," Tech. Rep. NASA-TM-1538, NASA Langley Research Center, Hampton, VA, 1979.

[7] Buśantowicz, W., "Matlab Script for 3D Visualization of Missile and Air Target Trajectories," *International Journal of Computer and Information Technology*, Vol. 5, No. 5, 2016, pp. 419–422.

I. Ek

A. Proje Takvimi



Şekil 1 Proje Takvimi

B. Takım Formu

UCK/UZB 4901(E)/4902(E) Bitirme Projesi Takım Çalışması Formu Belge Tarihi: 24.11.2024, Sürümü: 0.2			
Öğrenci Adı Soyadı:	Celal Aksoy		
Öğrenci Numarası:	110200051		
Bitirme Projesi Başlığı:	F-16 İçin Otonom Yer Çarpışma Önleyici Sistem Geliştirilmesi: Arazi Algılama ve Tehdit Analizi		
Bitirme Projesi Konusu:	F-16 dinamik modelinde arazi veri entegrasyonu, tehdit algılama algoritmaları ve çarpışma riski tespiti.		
Takım Çalışmasının Başlığı, Konusu:	F-16 İçin Otonom Yer Çarpışma Önleyici Sistem Geliştirilmesi, Projenin genel amacı, F-16 dinamik modeli ve mevcut GCAS sistemine arazi entegrasyonu sağlayarak çarpışma önleme performansını artırmaktır. Bu kapsamda, tehdit algılama ve kaçış manevrası hesaplama süreçlerini içeren bir sistem geliştirilmesi hedeflenmektedir.		
Diğer Takım Üyelerinin Bilgileri:	**Diğer Takım Üyelerinin Bitirme Proje Başlıkları**	**İlgili Takım Üyesinin Bu Projenin Gerçekleşmesi için Sağlayacağı Çıktılar**	**Çıktıların Sağlanamaması Durumunda Alternatif Çözüm**
Mehmet Tunahan Kara 110200028	F-16 İçin Otonom Yer Çarpışma Önleyici Sistem Geliştirilmesi: Kaçış Manevraları Hesaplama ve Optimizasyonu	Algılanan tehdide göre kaçış manevrası planlama algoritmasının geliştirilmesi.	Optimizasyon algoritması başarısız olursa, daha basit bir kaçış manevrası stratejisi kullanılabilir.
Danışman Onayı:	Doç. Dr. EMRE KOYUNCU, Tarih: İmza:		
Varsa Eş Danışman Onayı:			

Şekil 2 Takım Formu

C. Etik Beyanı

Bu çalışmada yer alan tüm analiz, modelleme, benzetim, yazılım geliştirme ve değerlendirme faaliyetleri akademik dürüstlük ilkelerine bağlı kalınarak gerçekleştirilmiştir.

Çalışmada kullanılan tüm görseller, algoritmalar, veriler ve alıntılar ilgili kaynaklar belirtilerek kullanılmış; üçüncü taraf yazılım paketleri veya açık kaynaklı kütüphanelerin kullanımı gerektiği durumlarda kaynaklara atıf yapılmıştır. Uçuş simülasyonları ve yer çarpışma önleyici sistem (GCAS) algoritmaları yalnızca akademik amaçla ve deneysel değerlendirme bağlamında tasarlanmış olup, herhangi bir operasyonel sistemle doğrudan ilişkili değildir.

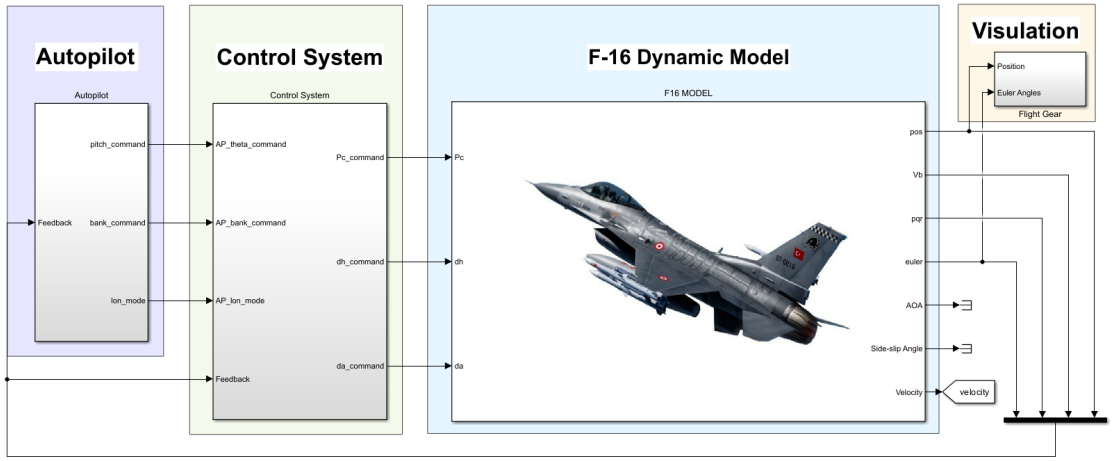
Bu çalışma kapsamında herhangi bir canlı varlık üzerinde deney yapılmamış, insan veya hayvan denekleri kullanılmamıştır. Dolayısıyla etik kurul onayı gerektirecek bir durum söz konusu değildir.

Tarafımdan sunulan bu raporun her bölümü özgün olarak tarafımca yazılmış ve etik dışı herhangi bir alıntı, kopyalama veya çarpıtma yapılmamıştır.

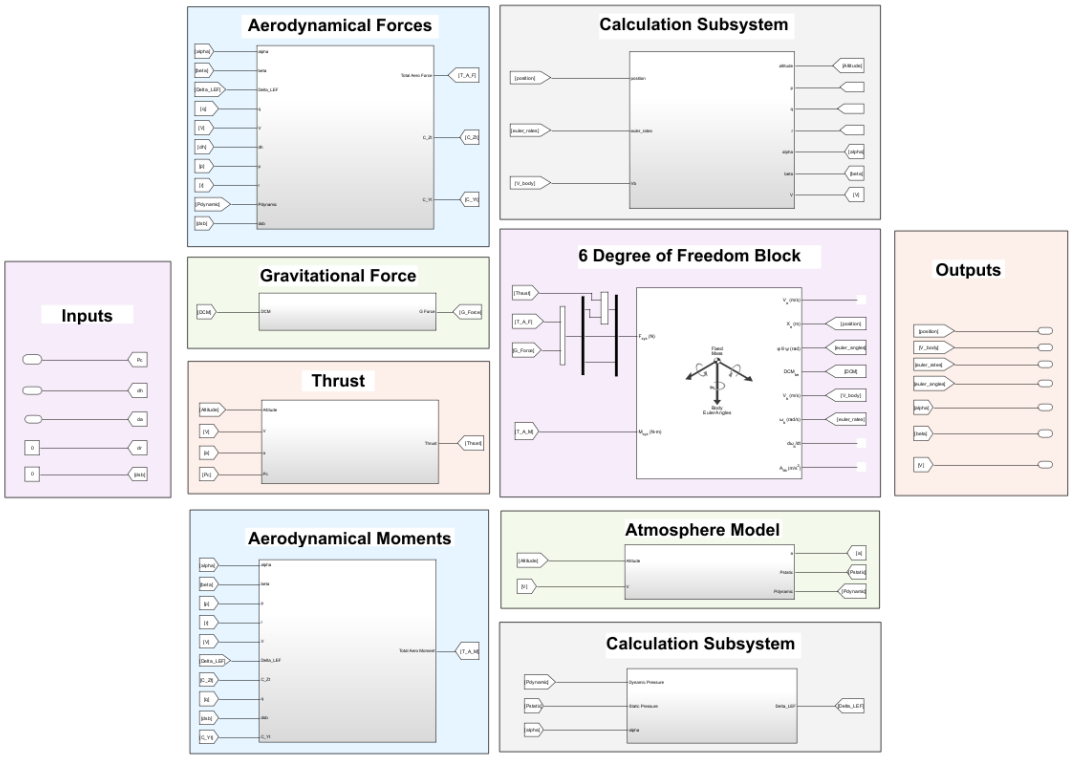
II. Proje Ekleri

A. Simulink Blokları

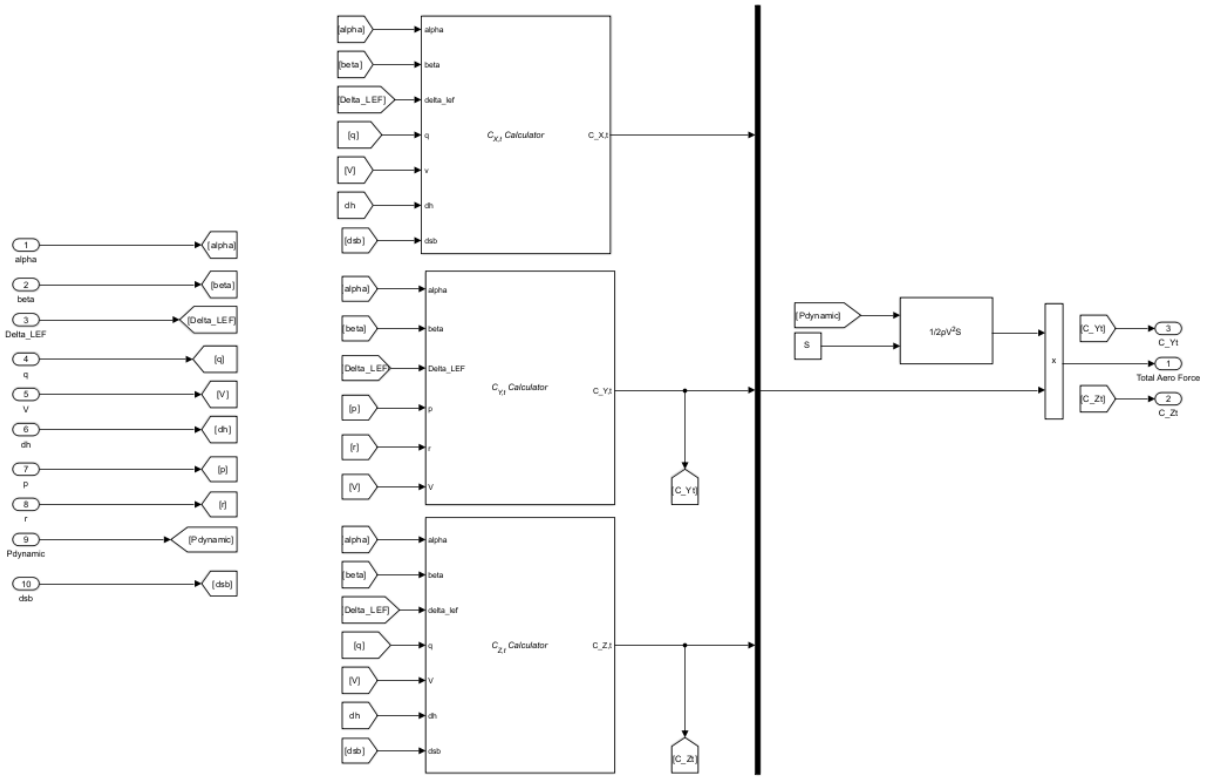
1. Dinamik Model Simulink Blokları



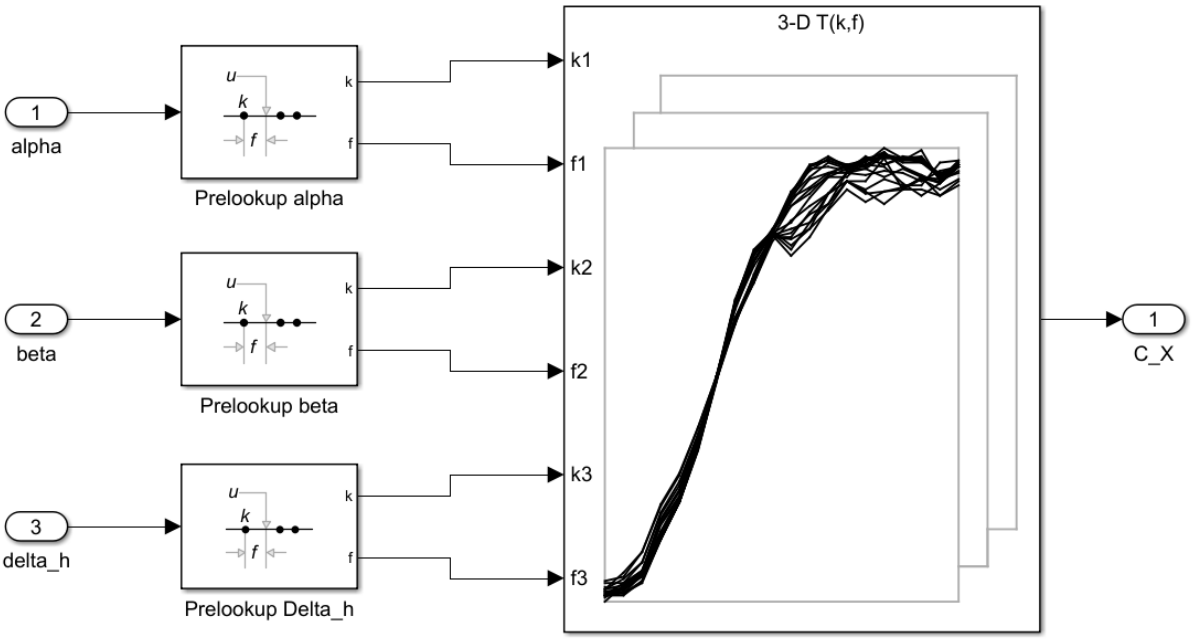
Şekil 3 Tasarlanan F-16 Modeli



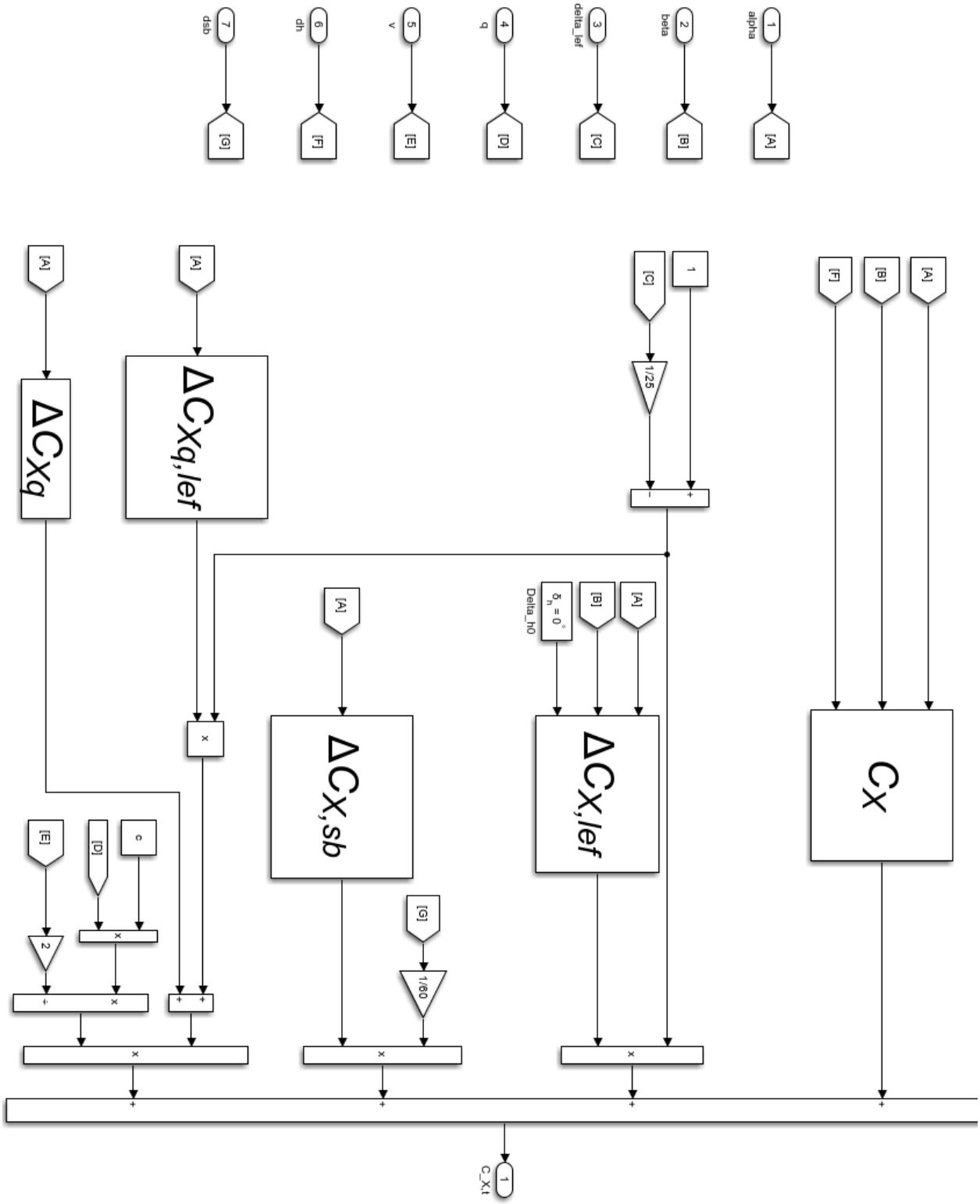
Şekil 4 F-16 Dinamik Modeli



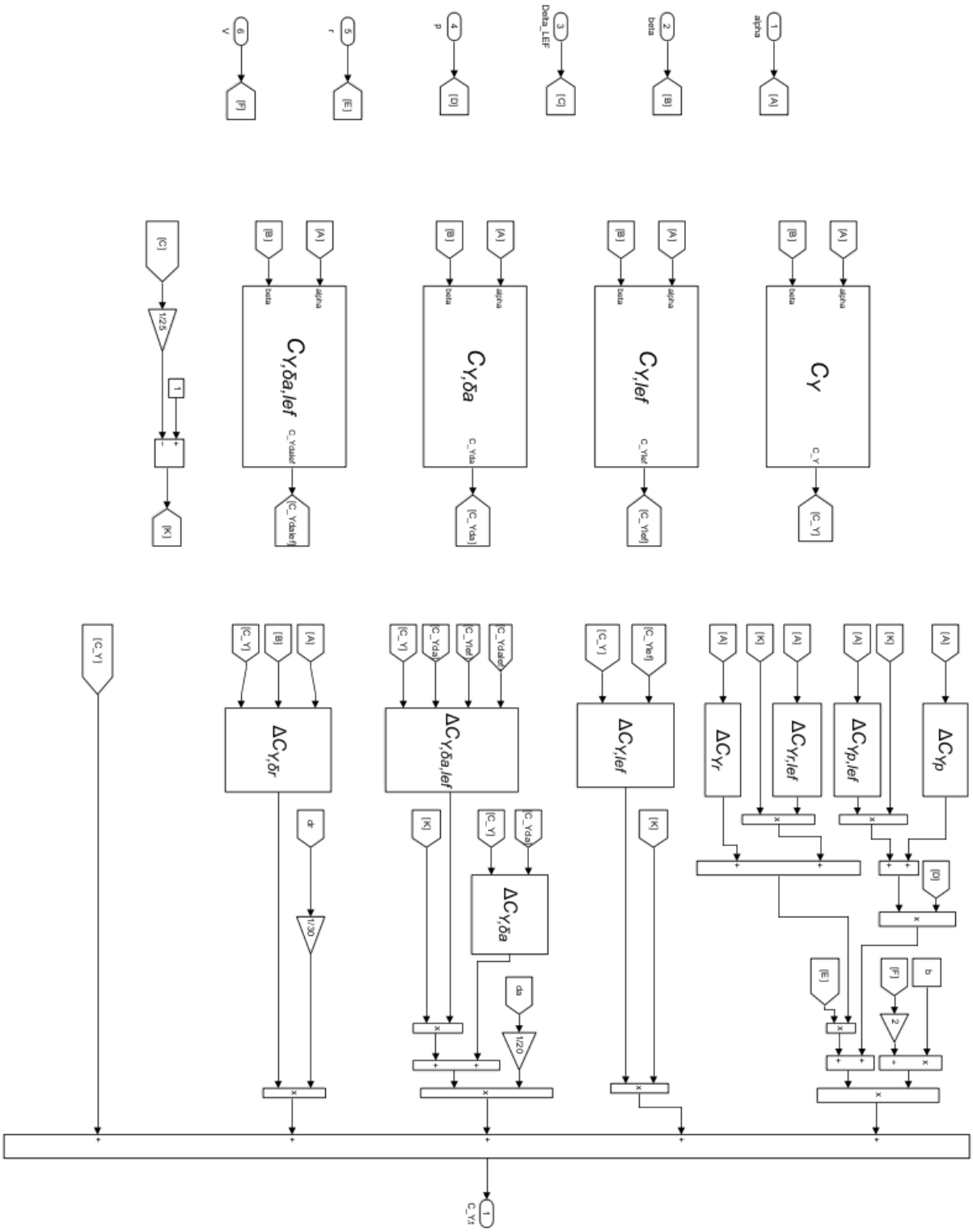
Şekil 5 Aerodinamik Kuvvet Bloğu



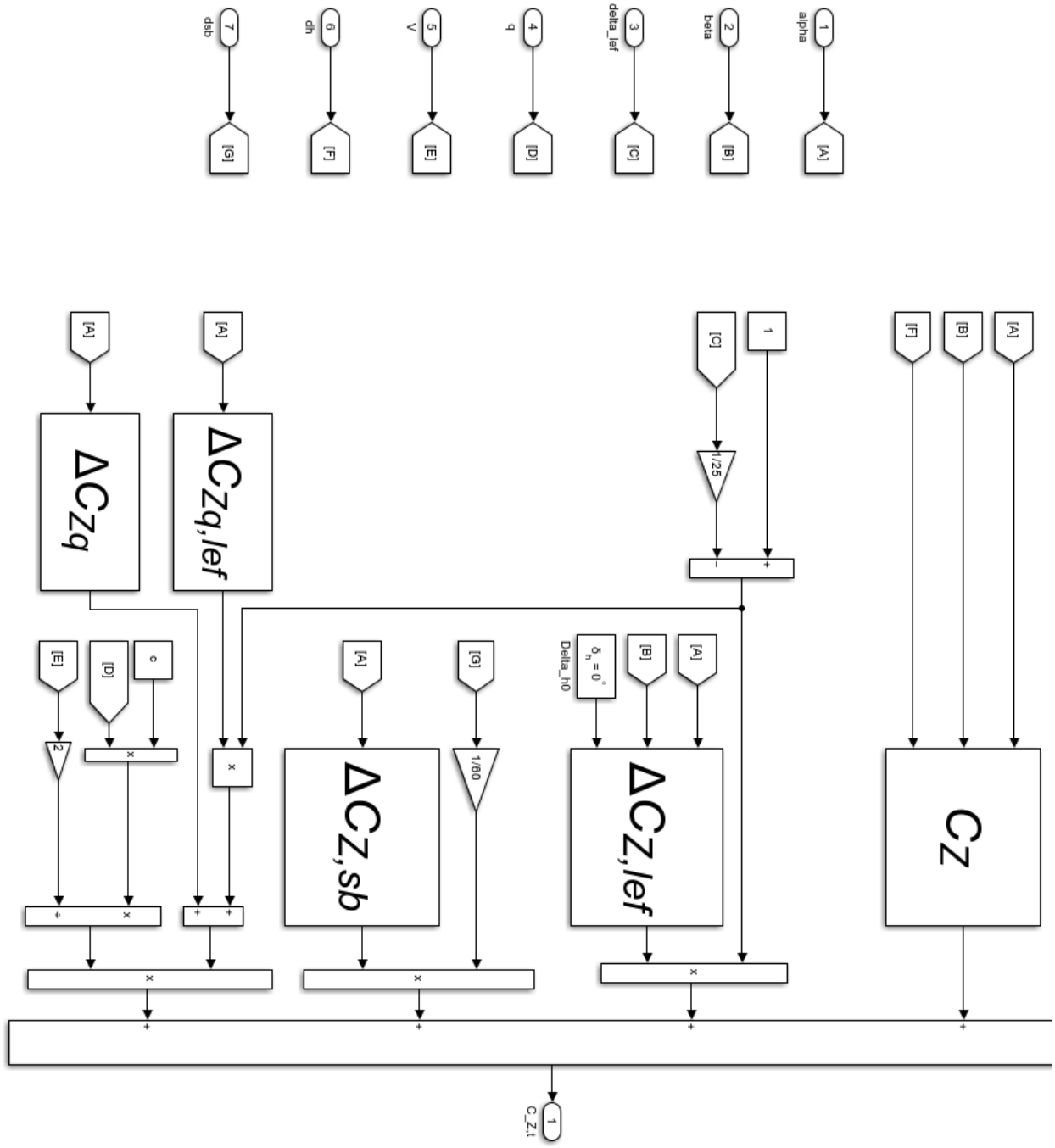
Şekil 6 Örnek PreLookup ve İnterpolasyon Blokları



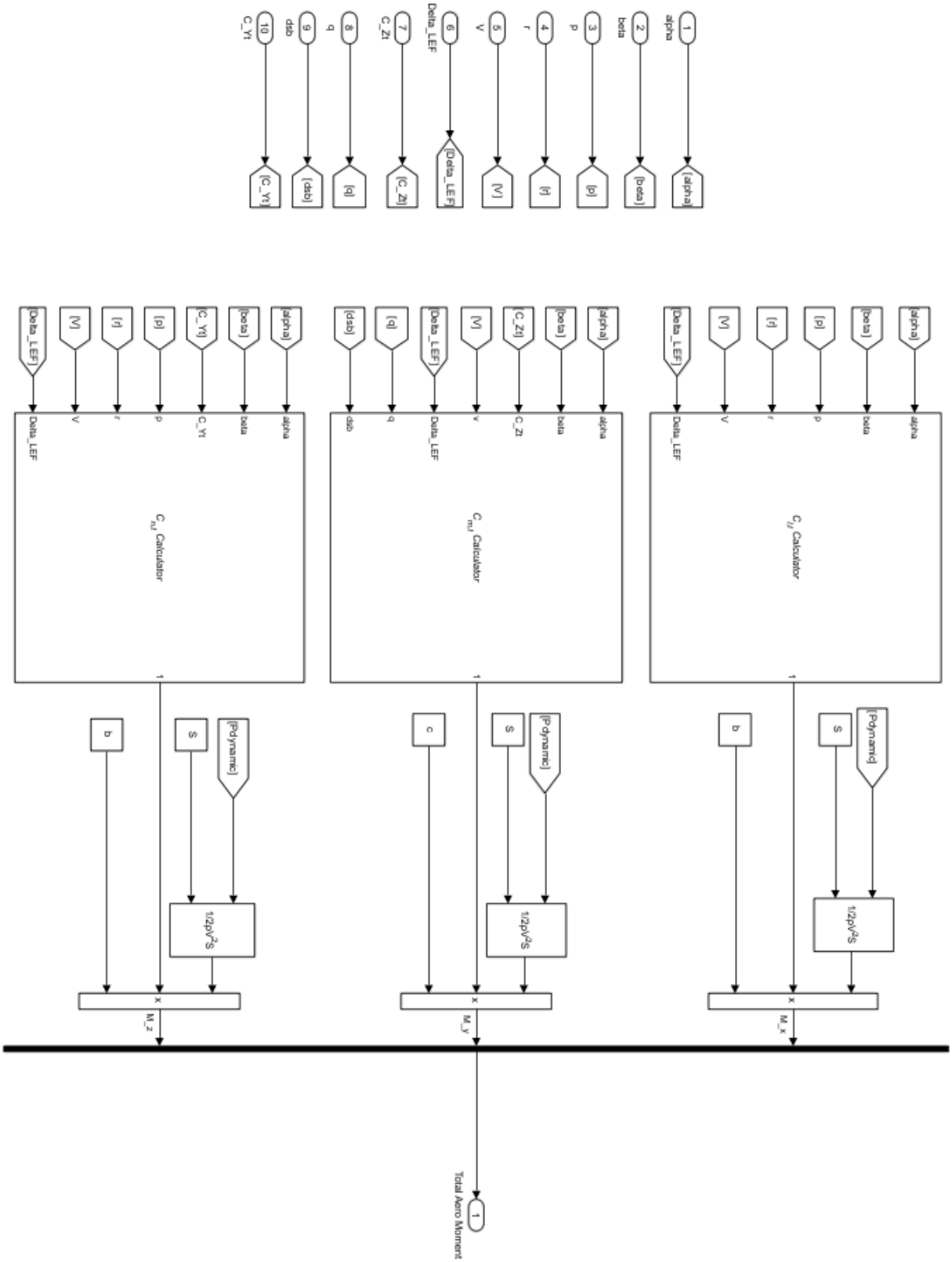
Şekil 7 X Yönündeki Aerodinamik Kuvvet Katsayısı



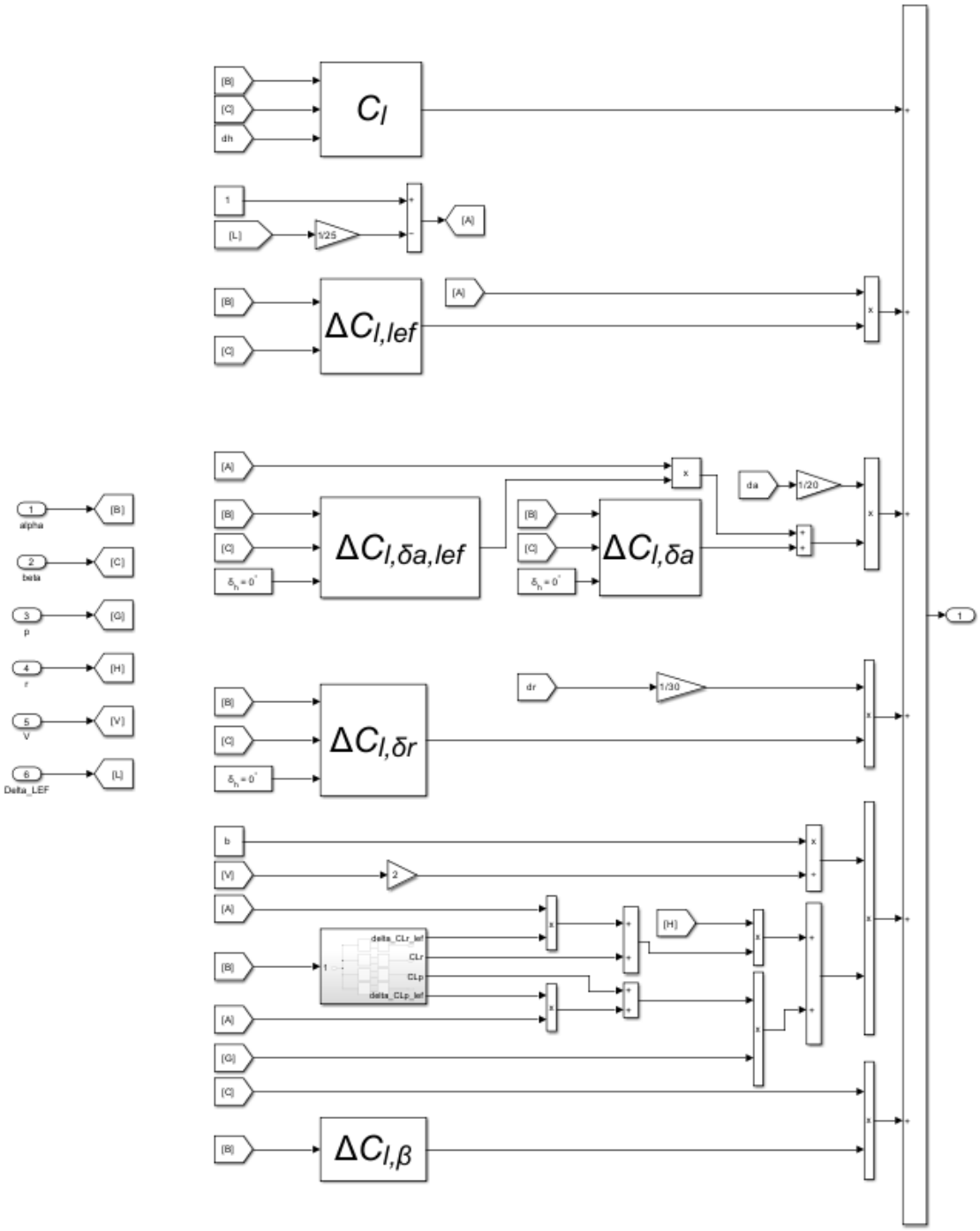
Şekil 8 Y Yönündeki Aerodinamik Kuvvet Katsayısı



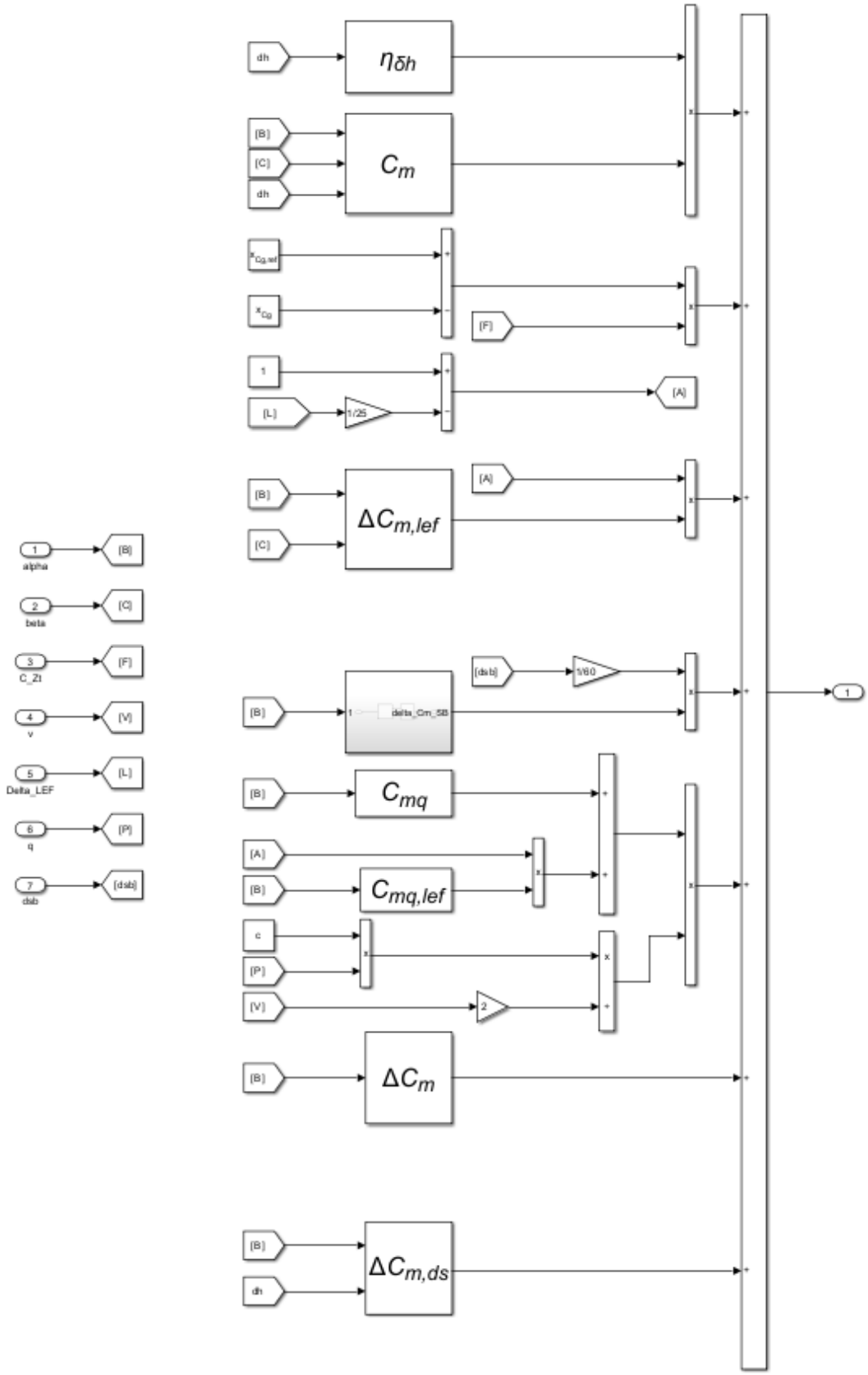
Şekil 9 Z Yönündeki Aerodinamik Kuvvet Katsayısı



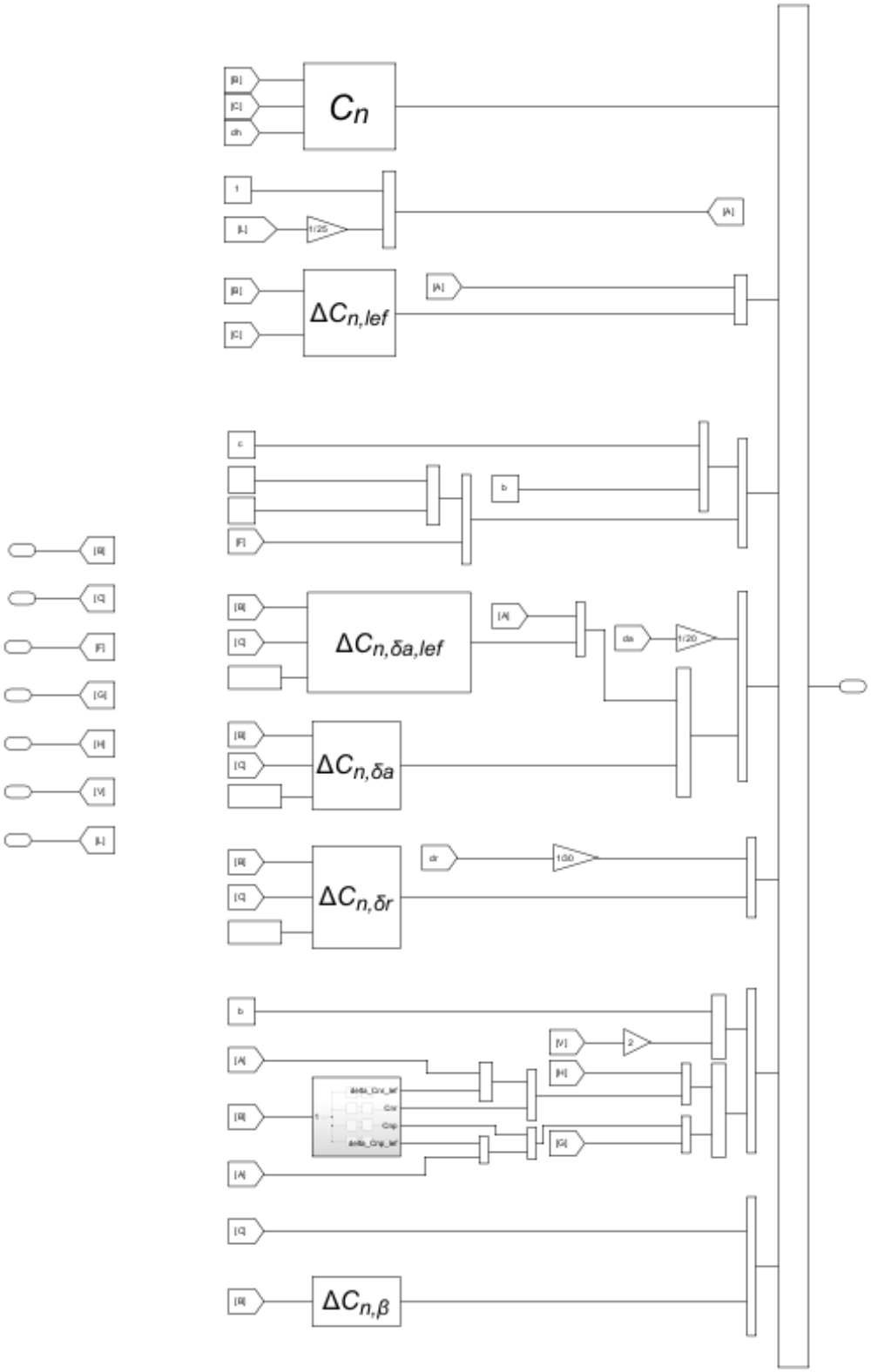
Şekil 10 Aerodinamik Moment Bloğu



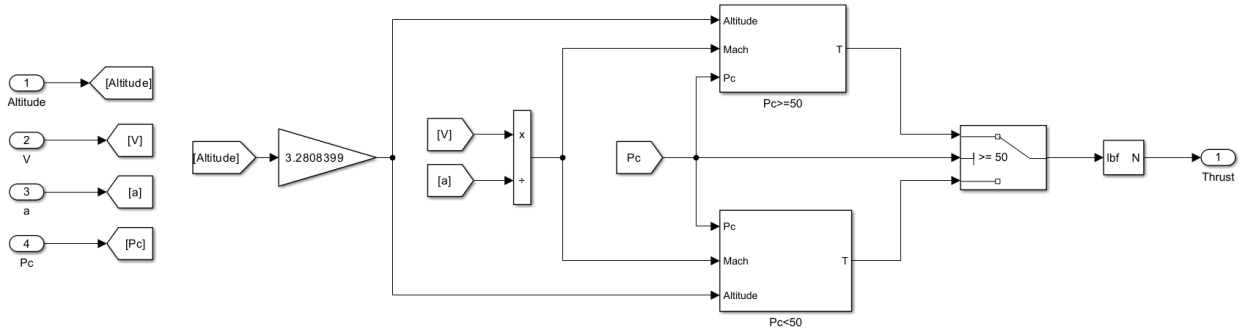
Şekil 11 X Yönündeki Aerodinamik Moment Katsayısı



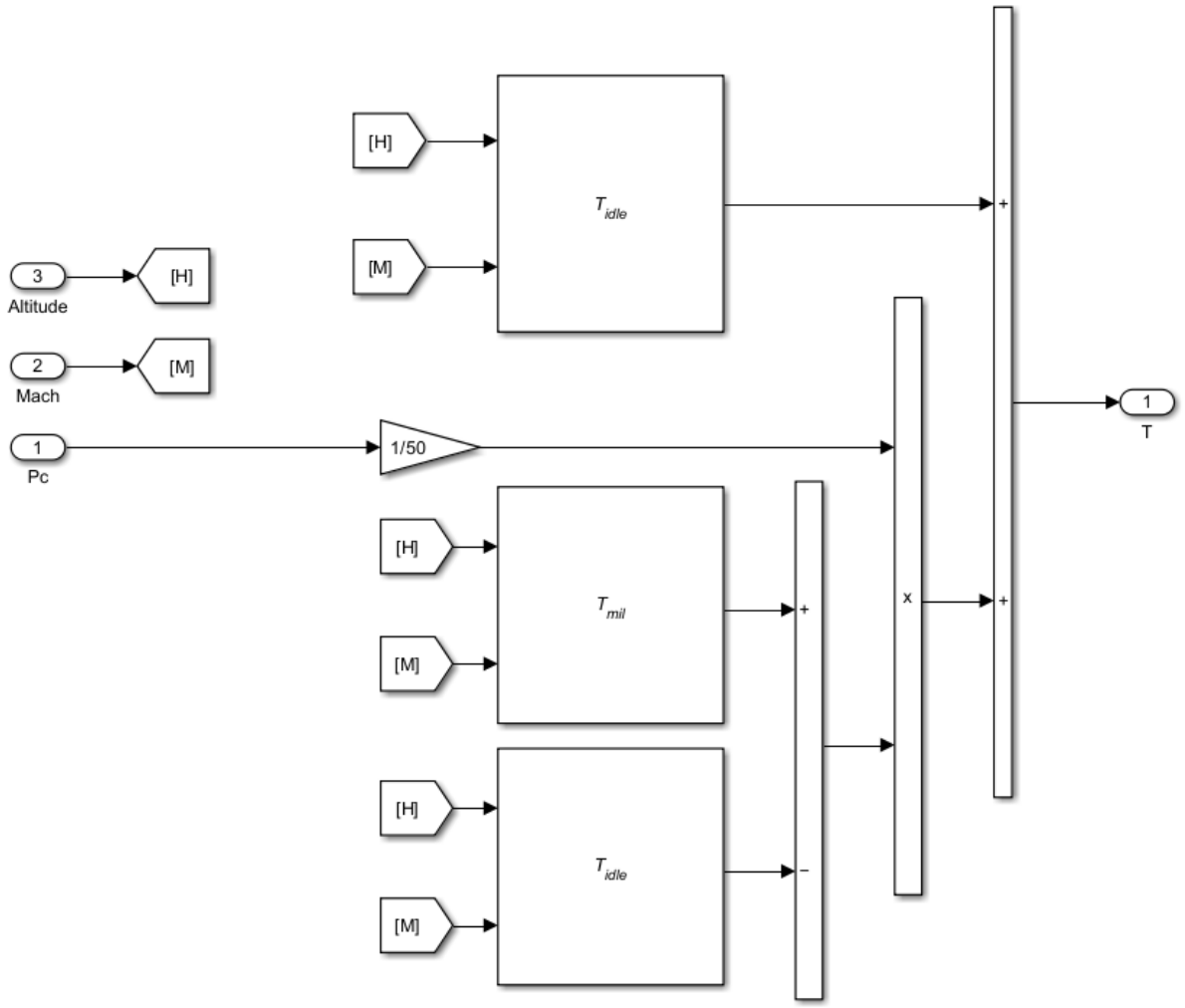
Şekil 12 Y Yönündeki Aerodinamik Moment Katsayısı



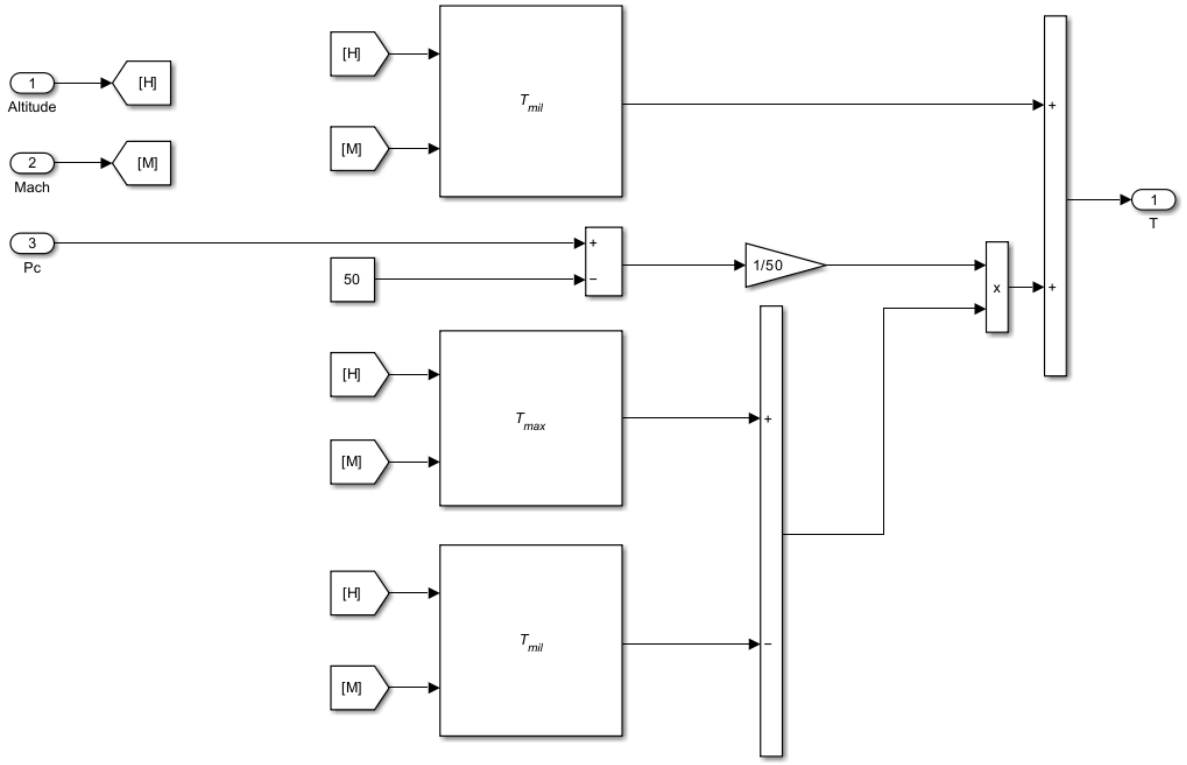
Şekil 13 Z Yönündeki Aerodinamik Moment Katsayısı



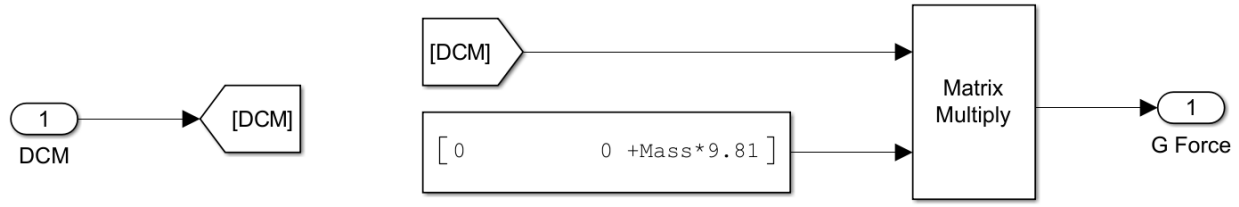
Şekil 14 Pc'ye Bağlı İtki Koşul Formülü



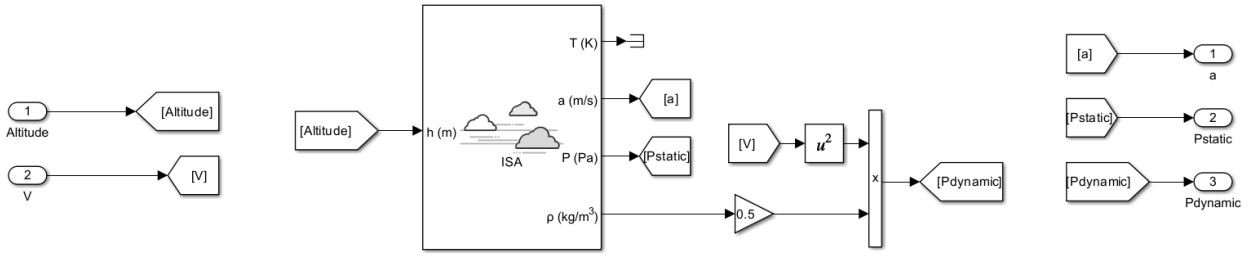
Şekil 15 Pc'ye bağlı İtki Formülü (< 50)



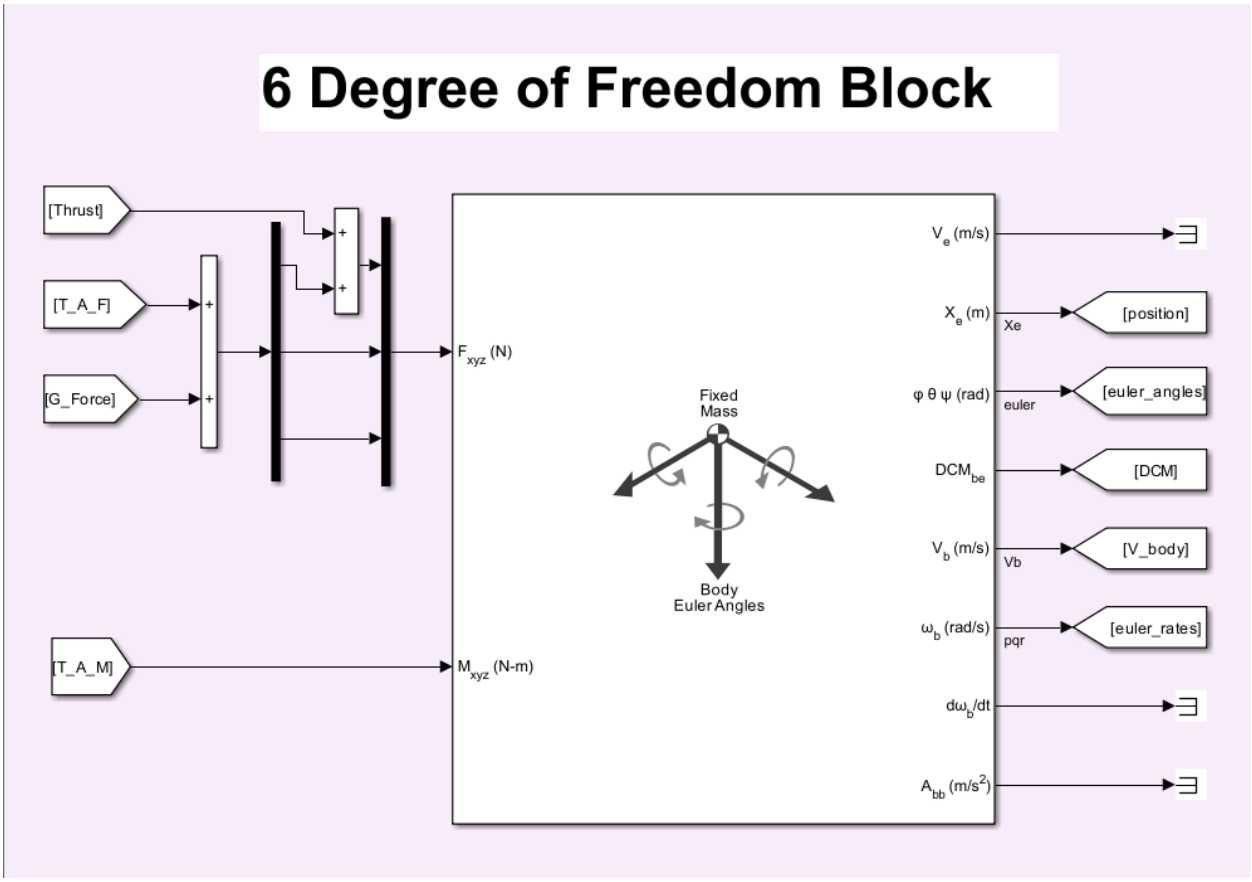
Şekil 16 Pc'ye bağlı İtki Formülü (≥ 50)



Şekil 17 Yerçekimi kuvvetinin DCM ile gövde koordinatlarına aktarılması

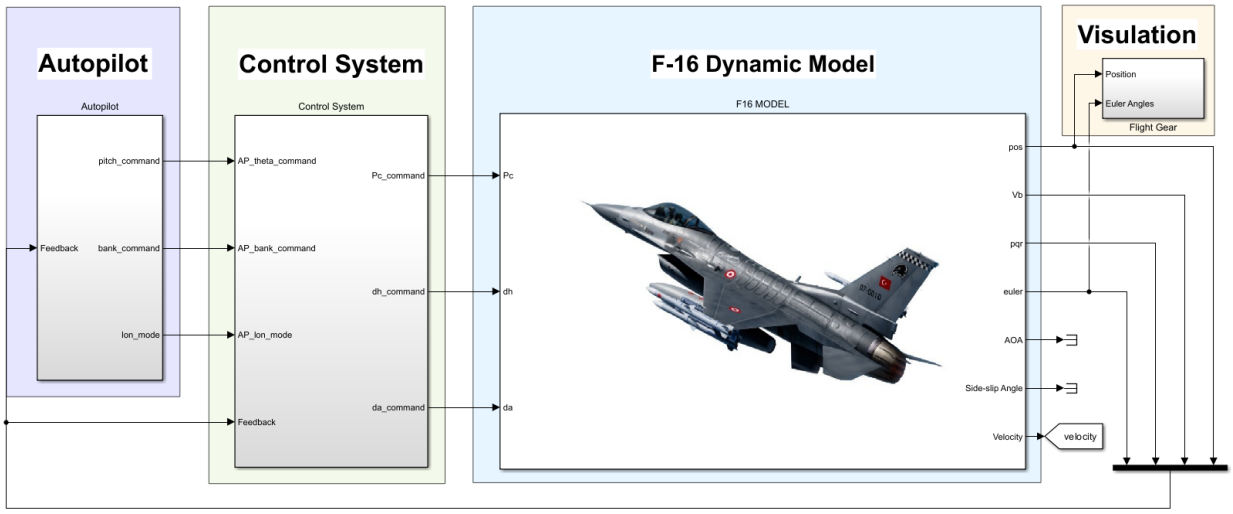


Şekil 18 ISA Tabanlı Atmosfer Modeli ve Dinamik Basınç Hesap Bloğu

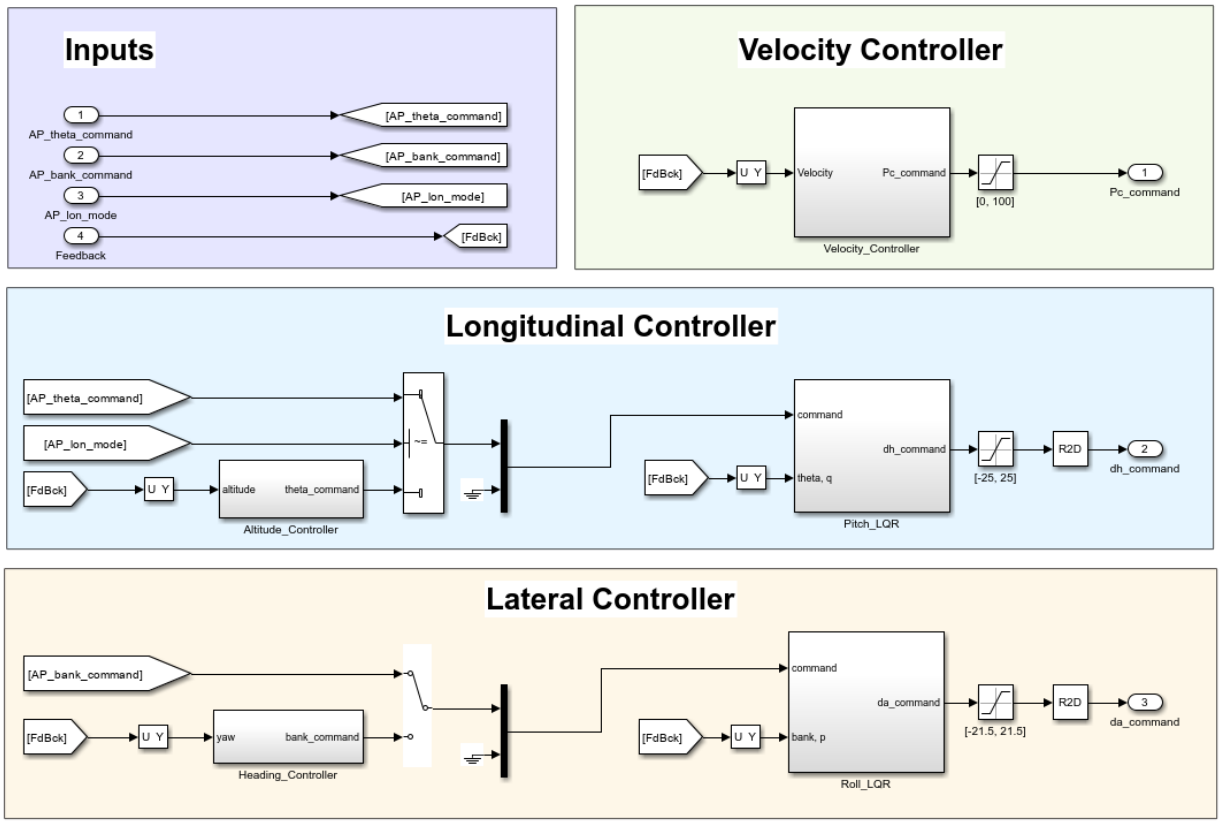


Şekil 19 6 Serbestlik Dereceli (6DoF) Uçuş Dinamiği Bloğu

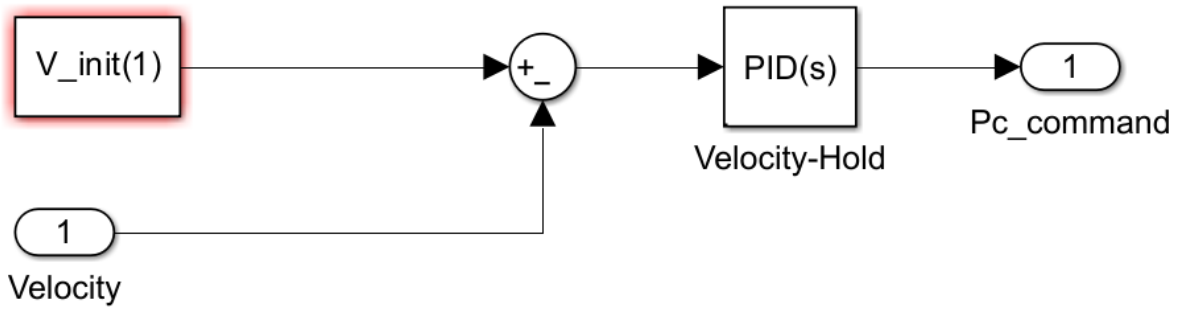
2. Kontrol Sistemi Simulink Blokları



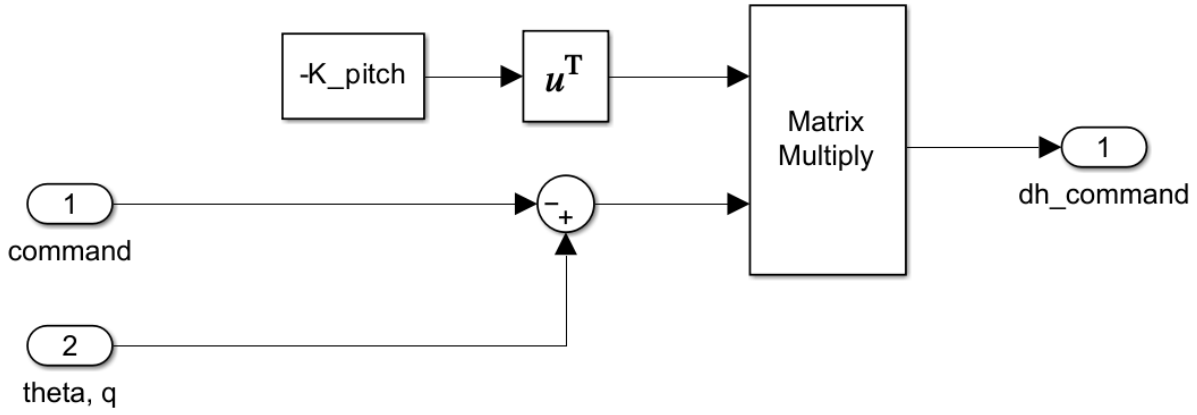
Şekil 20 Tasarlanan F-16 Modeli



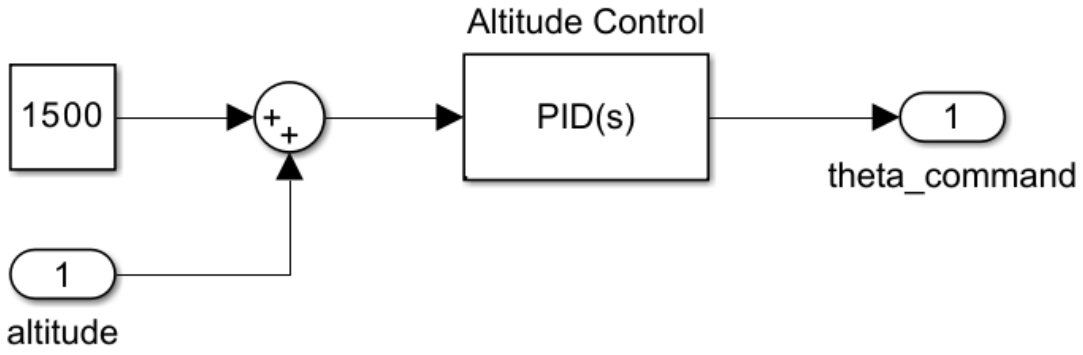
Şekil 21 Kontrol Sistem (Alt Kontrolcü) Blok Şeması



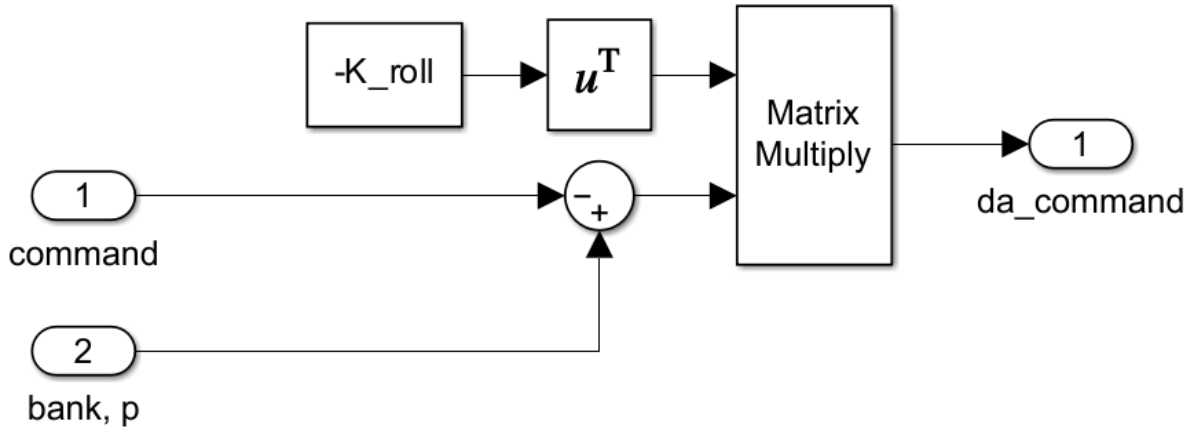
Şekil 22 Hız Sabitleyici Kontrol Bloğu



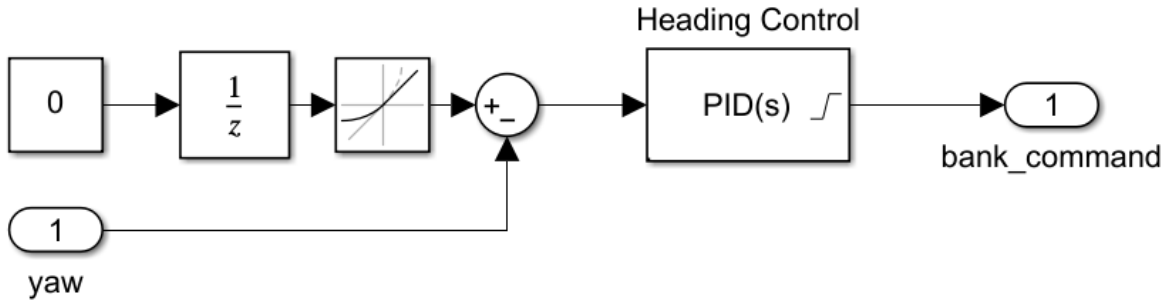
Şekil 23 Yunuslama Açısı için LQR Kontrolcü



Şekil 24 İrtifa Sabitleyici Kontrol Bloğu

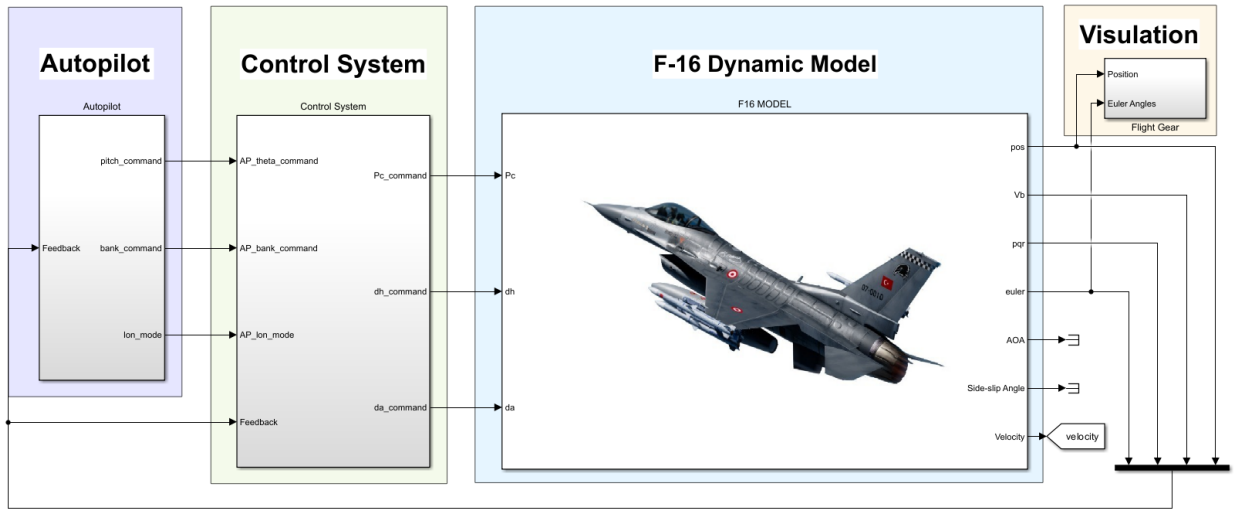


Şekil 25 Yuvarlanma Açısı için LQR Kontrolcü

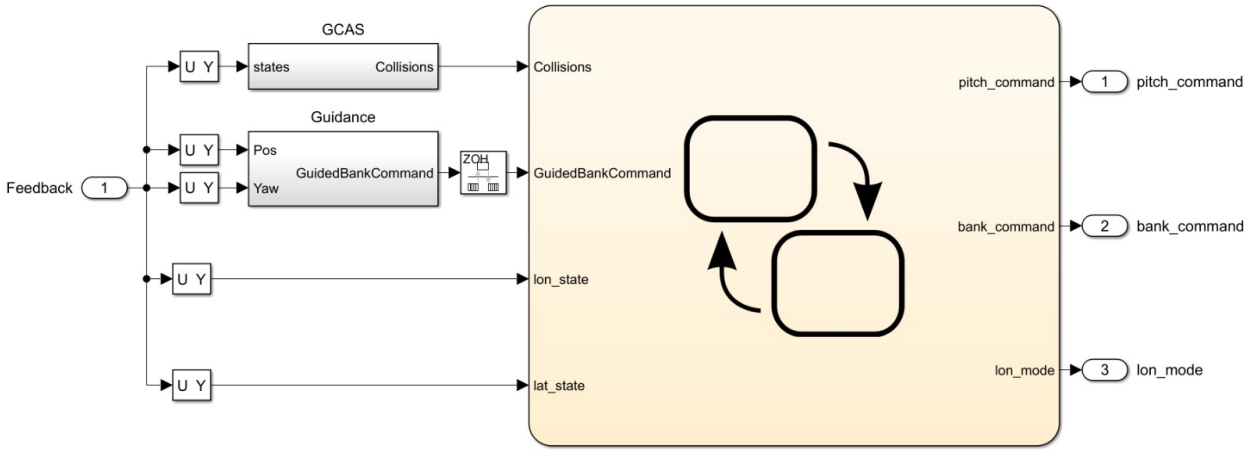


Şekil 26 Yönelim Sabitleyici Kontrol Bloğu

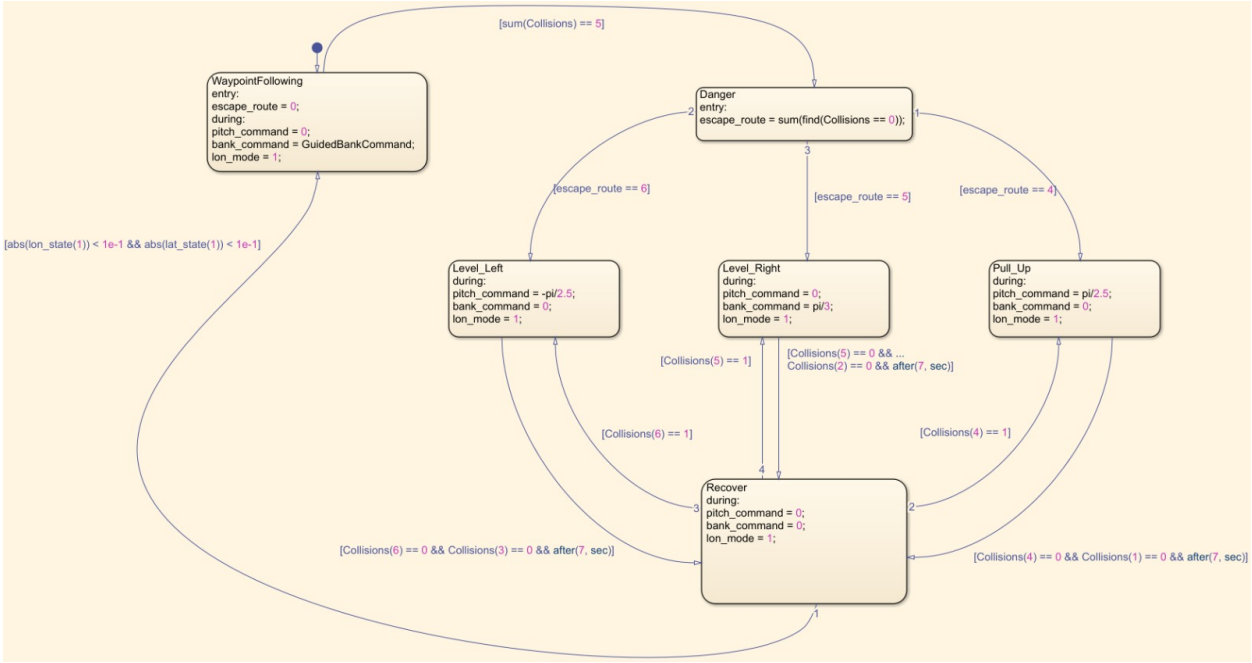
3. Otopilot Sistemi Simulink Blokları



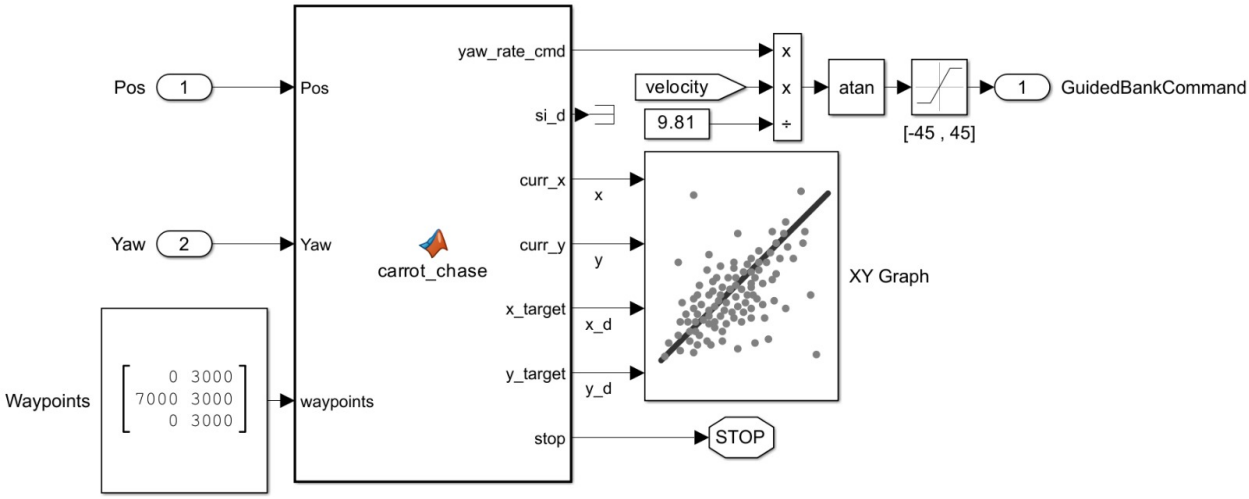
Şekil 27 Tasarlanan F-16 Modeli



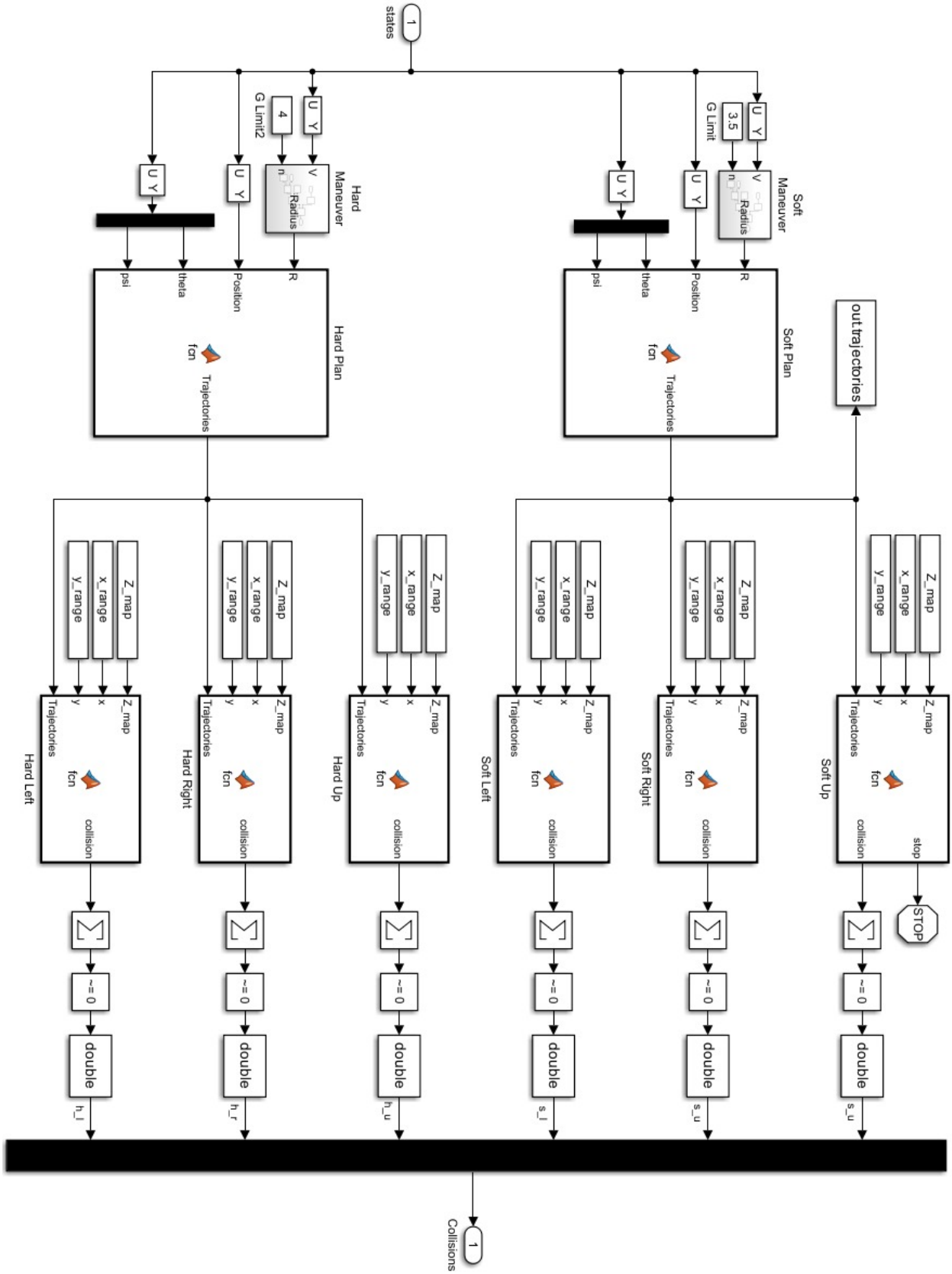
Şekil 28 Tasarlanan Otopilot Modeli



Şekil 29 Tasarlanan Durum Akışı (State Flow) Modeli

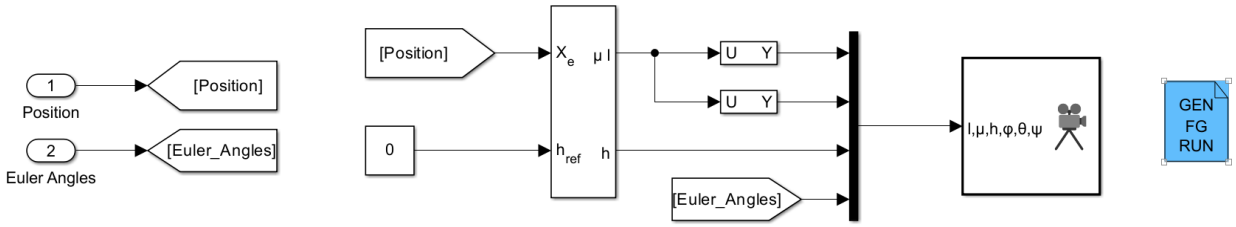


Şekil 30 Havuç Takibi Tabanlı Güdüm Sistemi



Şekil 31 Yer Çarpışma Önleyici Sistem Tasarımı

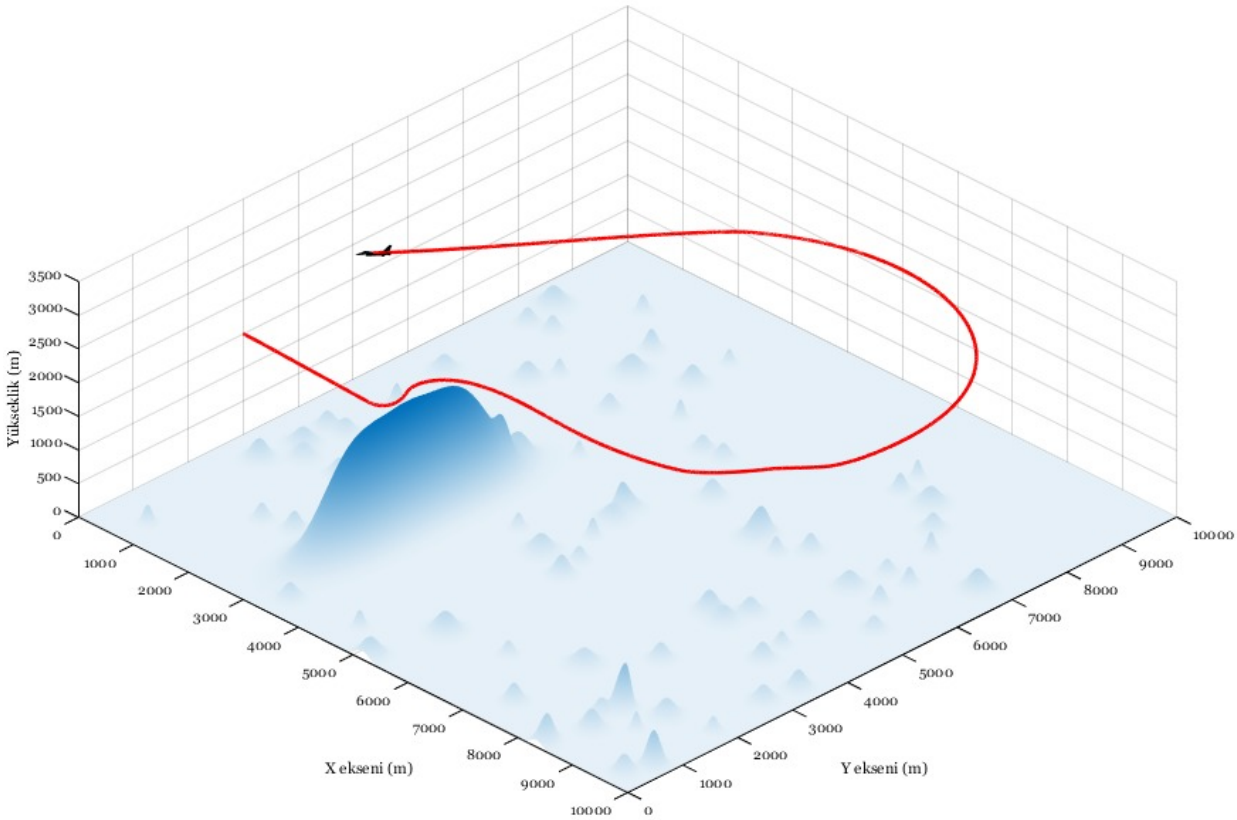
4. Görselleştirme Simulink Bloğu



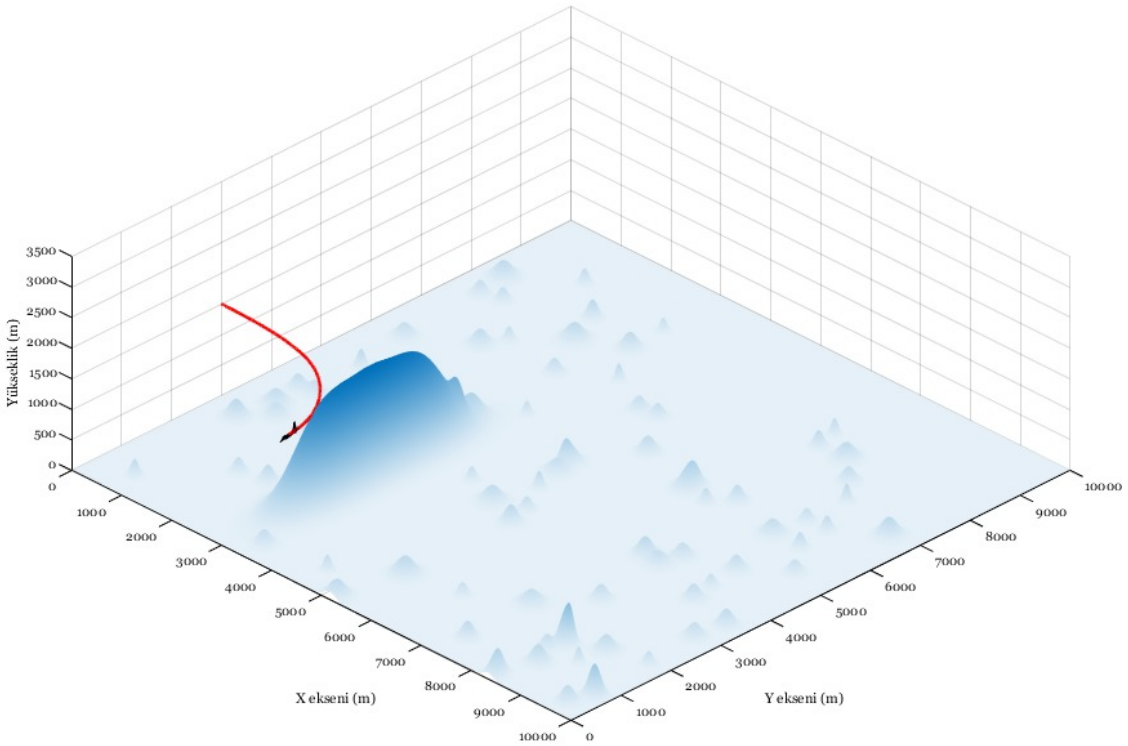
Şekil 32 Flight Gear Görselletirme Bloğu

B. Model Çıktıları

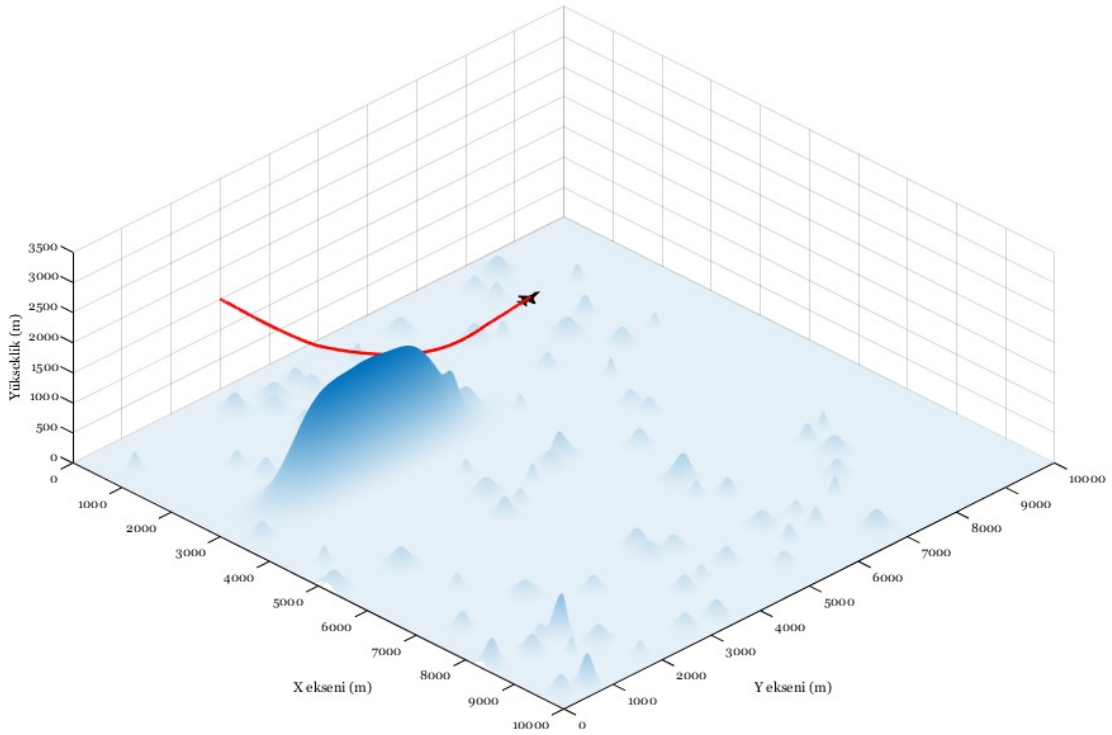
1. 3D Yörünge Grafikleri



Şekil 33 5g Yukarı Kaçış Manevrası



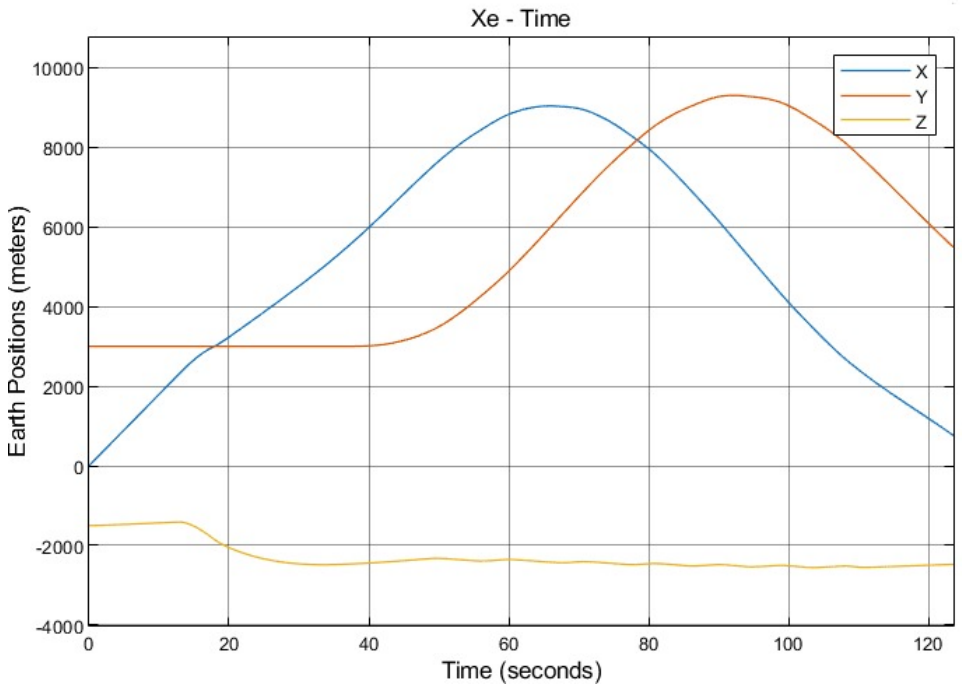
Şekil 34 Sağ Dönüş Kaçış Manevrası



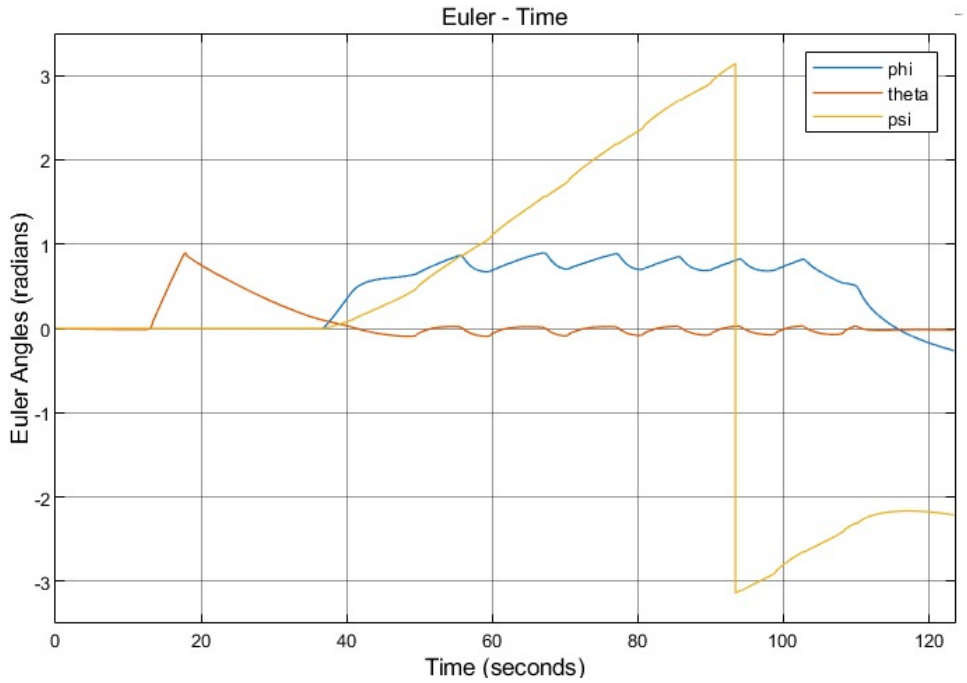
Şekil 35 Sola Dönüş Kaçış Manevrası

2. Model Durum Değerleri

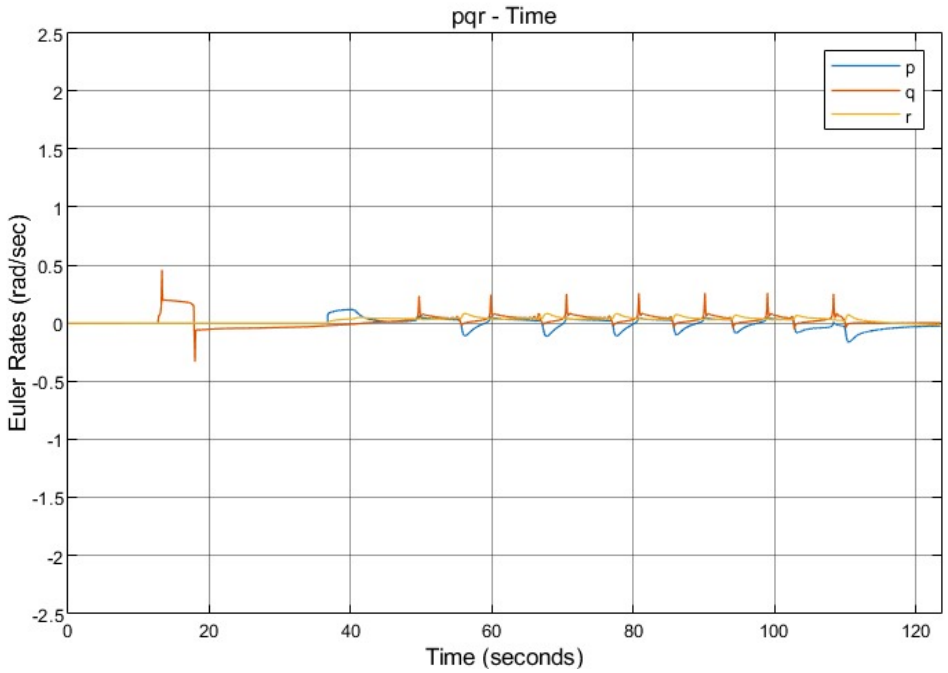
Yukarı Kaçış



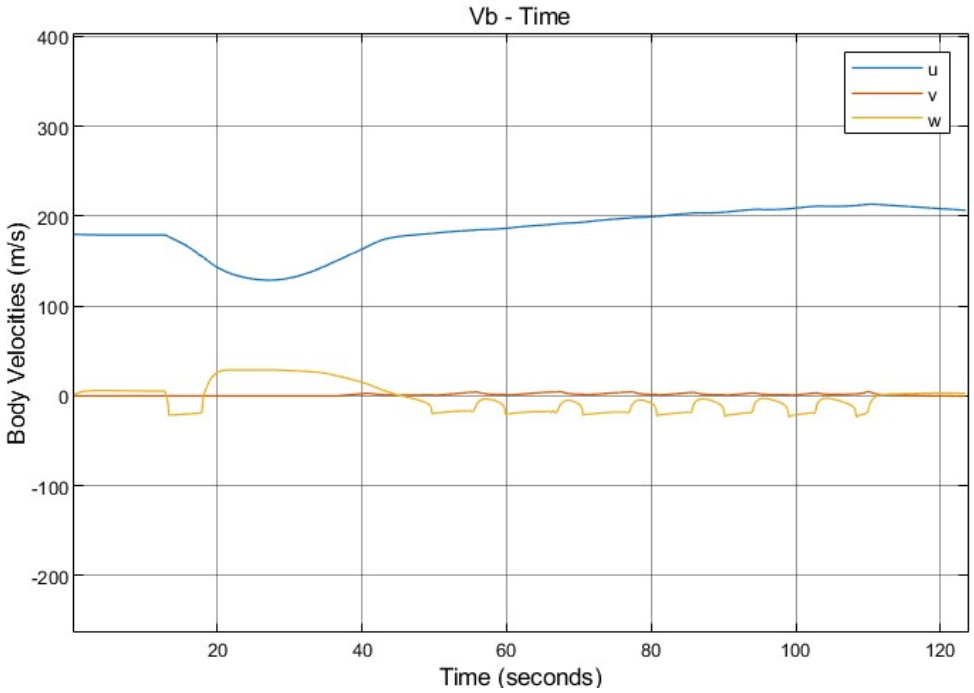
Şekil 36 Yukarı Kaçış Pozisyon Değerleri



Şekil 37 Yukarı Kaçış Euler Açı Değerleri

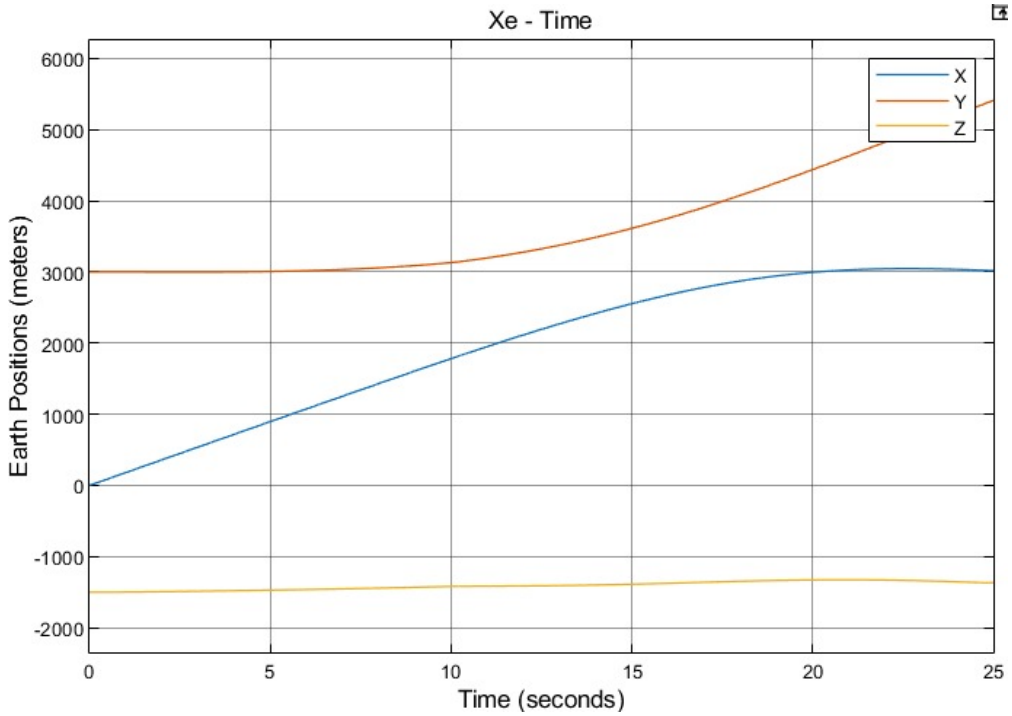


Şekil 38 Yukarı Kaçış Açısal Hız Değerleri

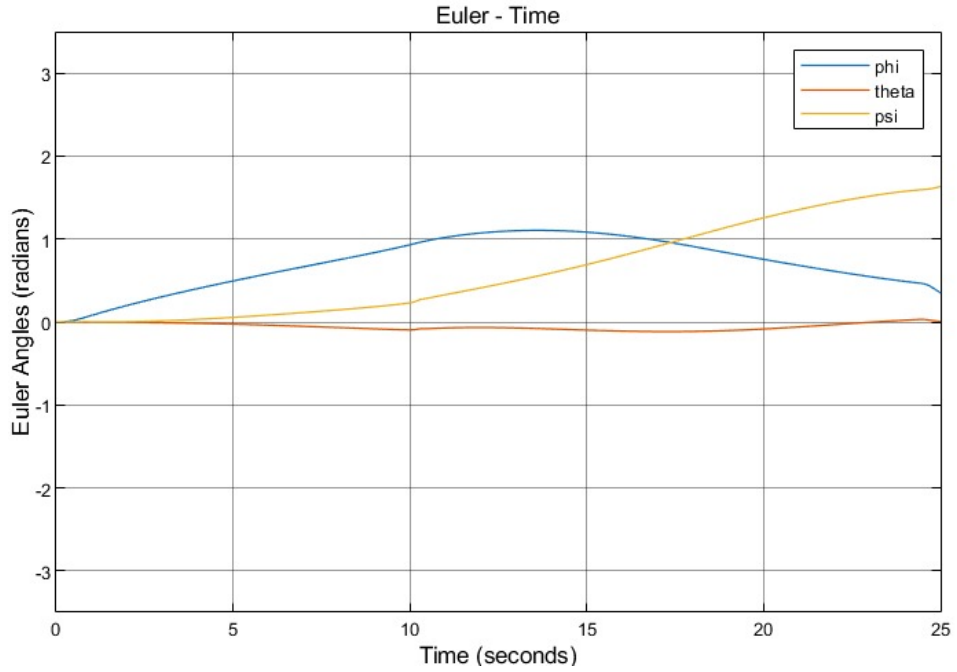


Şekil 39 Yukarı Kaçış Hız Değerleri

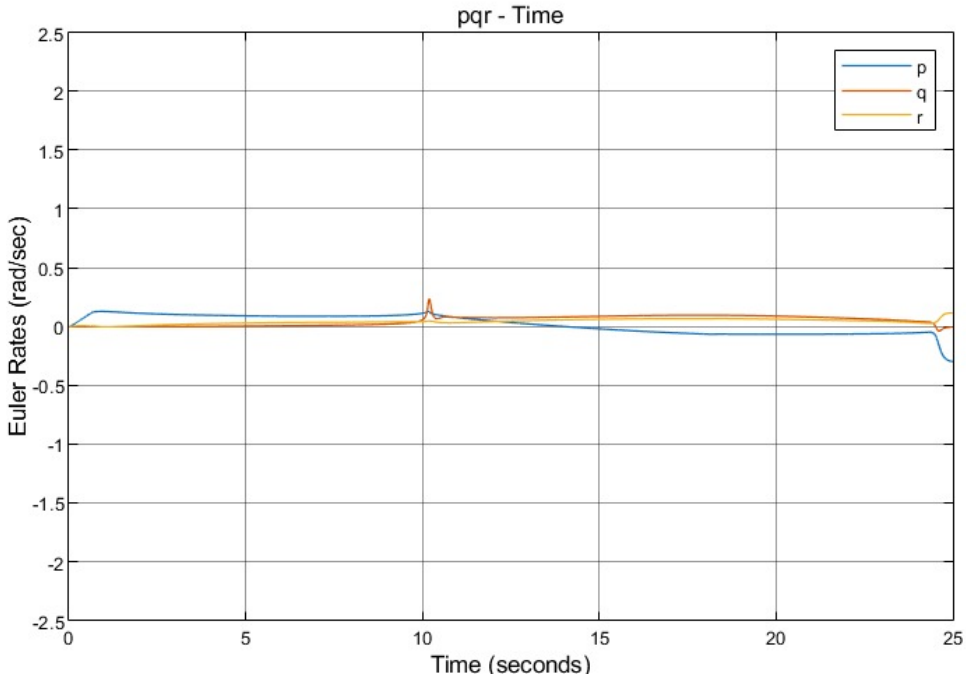
Tam Sağa Kaçış



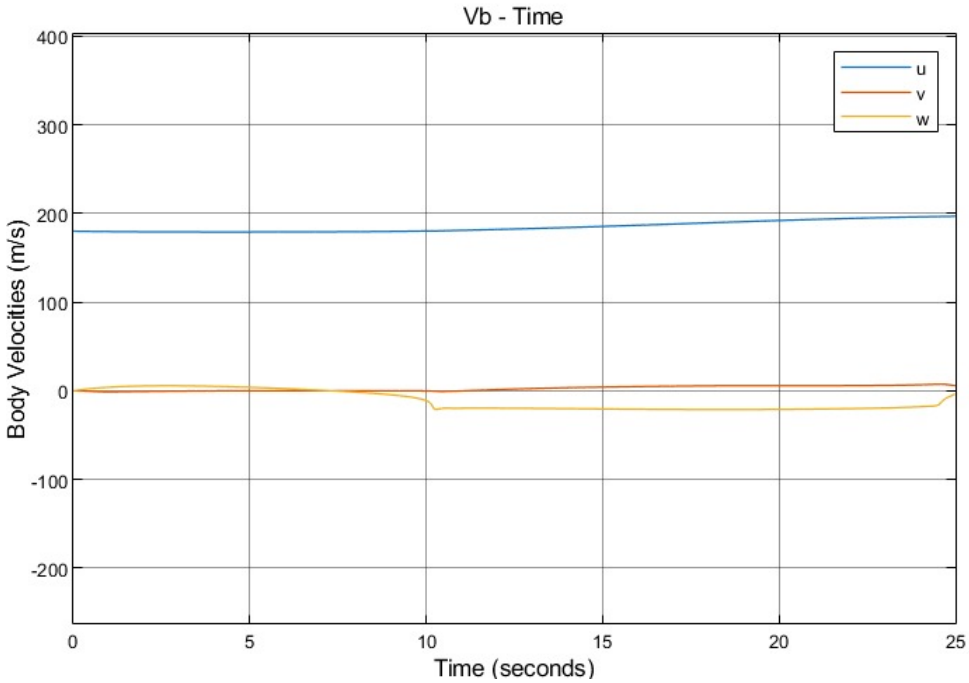
Şekil 40 Tam Sağa Kaçış Pozisyon Değerleri



Şekil 41 Tam Sağa Kaçış Euler Açı Değerleri

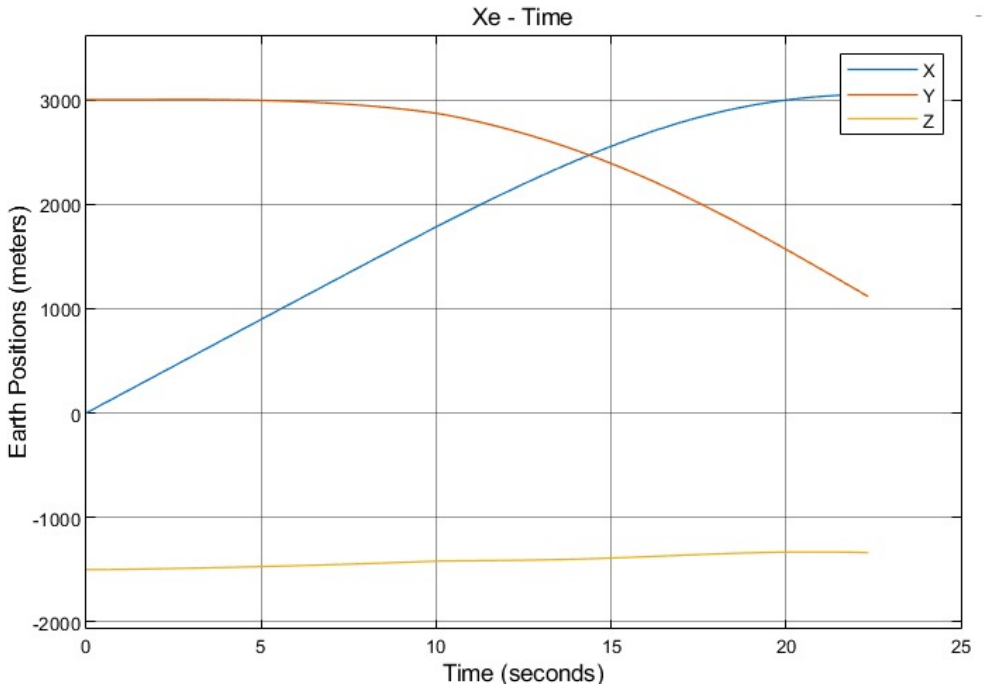


Şekil 42 Tam Sağa Kaçış Açısal Hız Değerleri

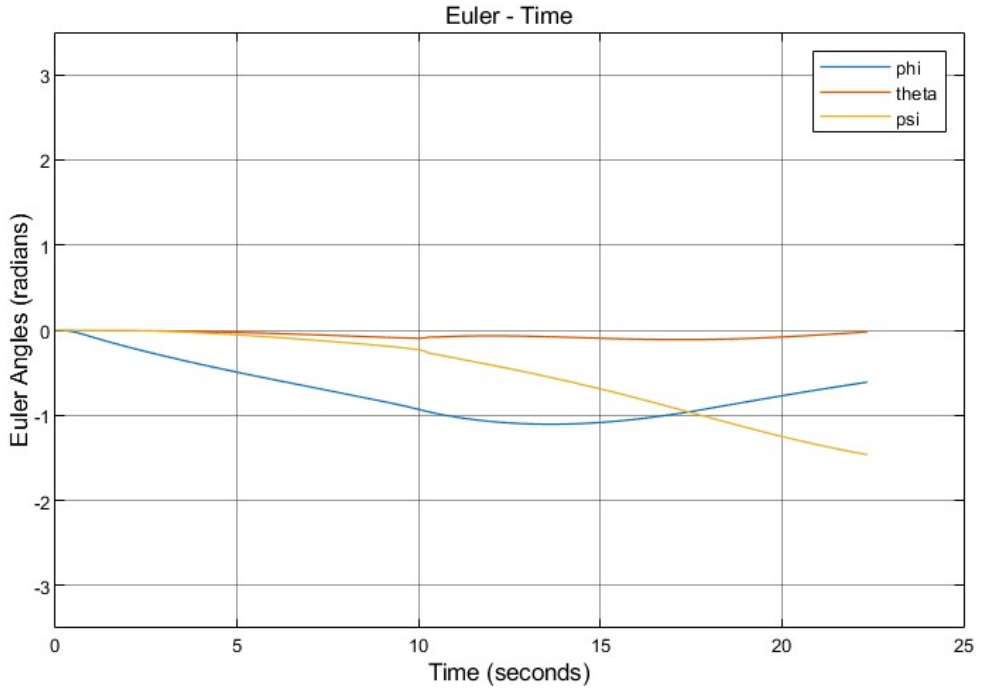


Şekil 43 Tam Sağa Kaçış Hız Değerleri

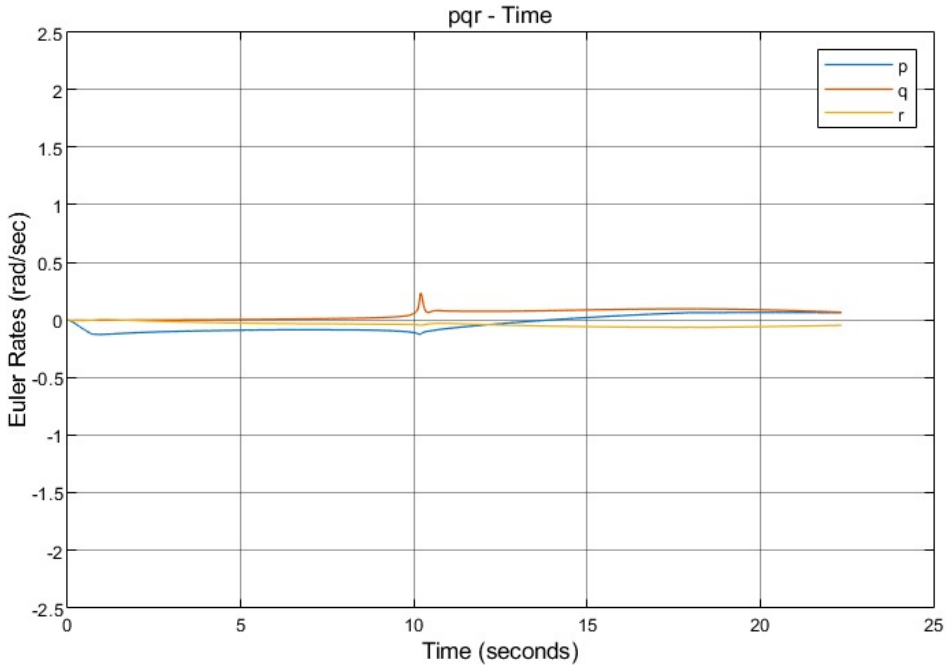
Tam Sola Kaçış



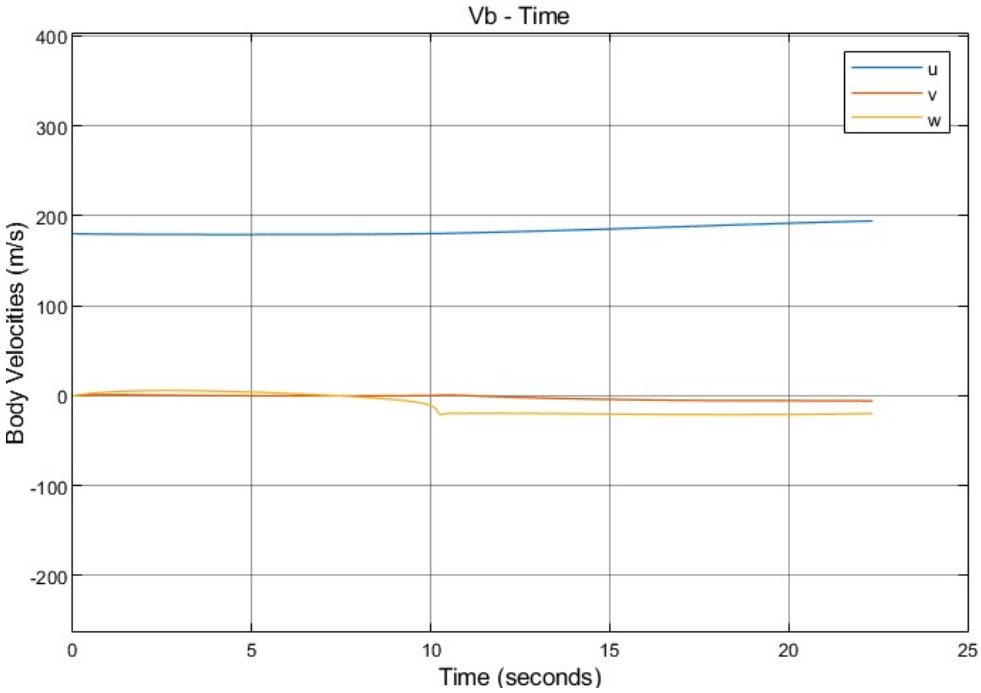
Şekil 44 Tam Sola Kaçış Pozisyon Değerleri



Şekil 45 Tam Sola Kaçış Euler Açı Değerleri



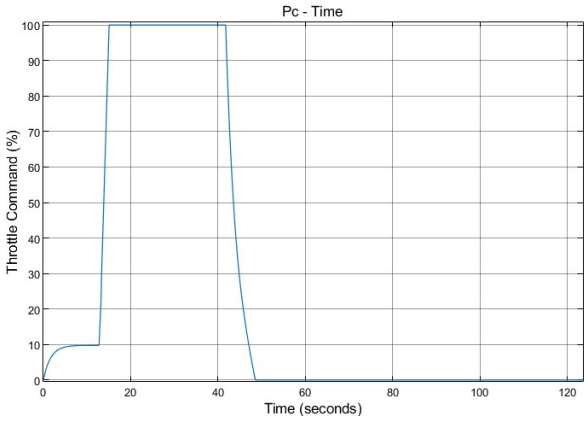
Şekil 46 Tam Sola Kaçış Açısal Hız Değerleri



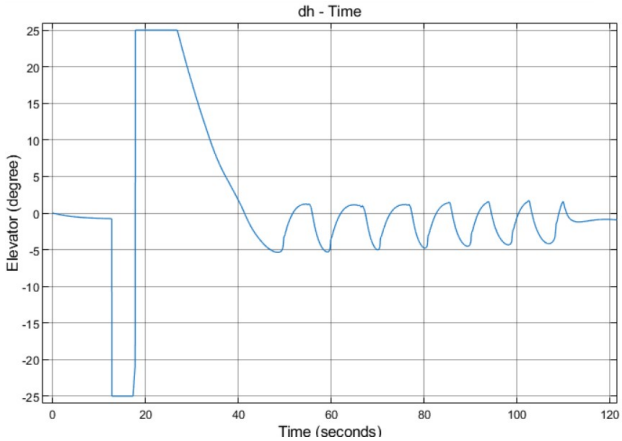
Şekil 47 Tam Sola Kaçış Hız Değerleri

3. Model Girdi Değerleri

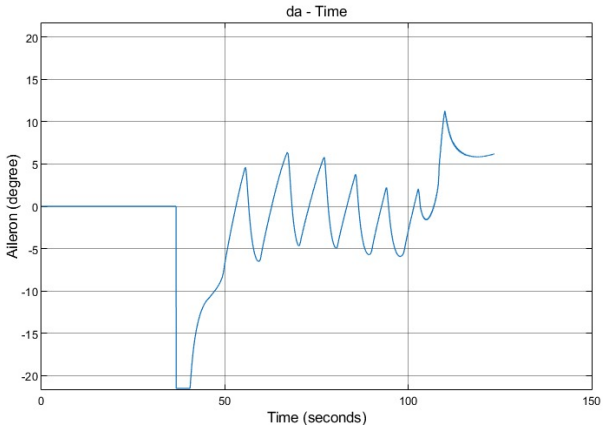
Yukarı Kaçış



Şekil 48 Yukarı Kaçış Gaz Girdisi

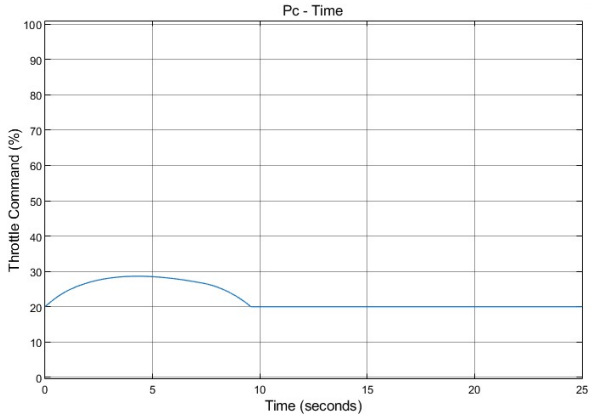


Şekil 49 Yukarı Kaçış Elevatör Girdisi

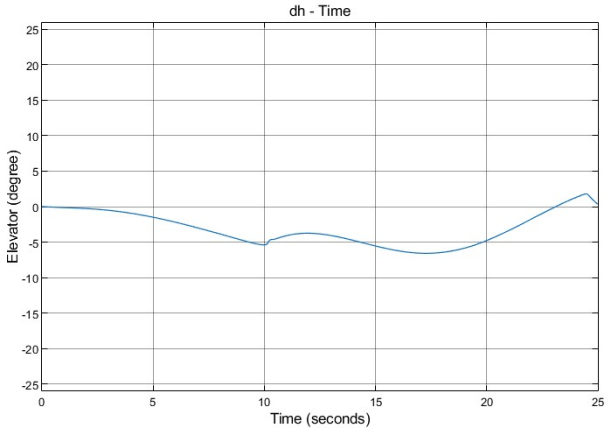


Şekil 50 Yukarı Kaçış Aleron Girdisi

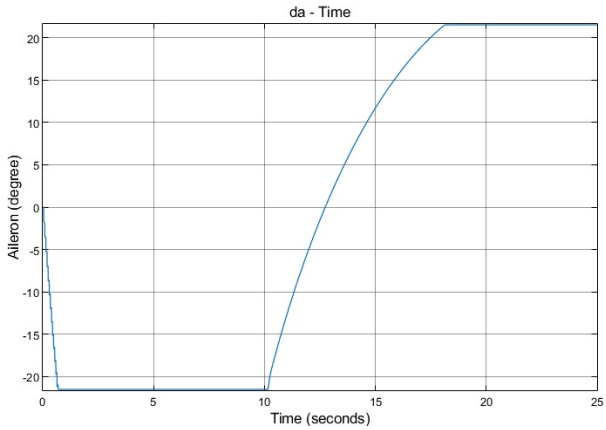
Tam Sağa Kaçış



Şekil 51 Tam Sağa Kaçış Gaz Girdisi

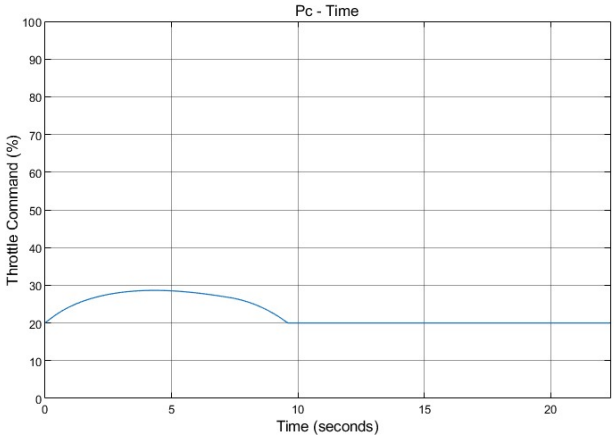


Şekil 52 Tam Sağa Kaçış Elevatör Girdisi

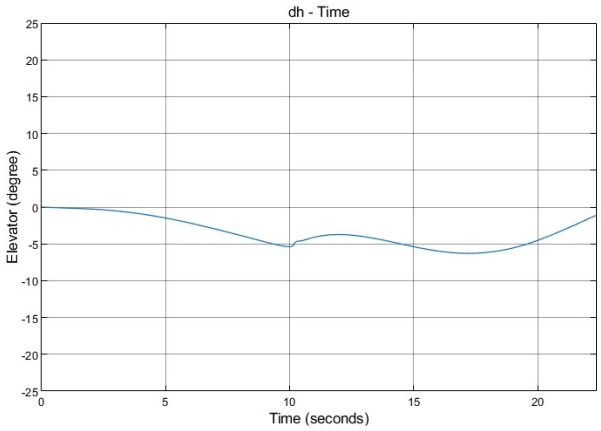


Şekil 53 Tam Sağa Kaçış Aleron Girdisi

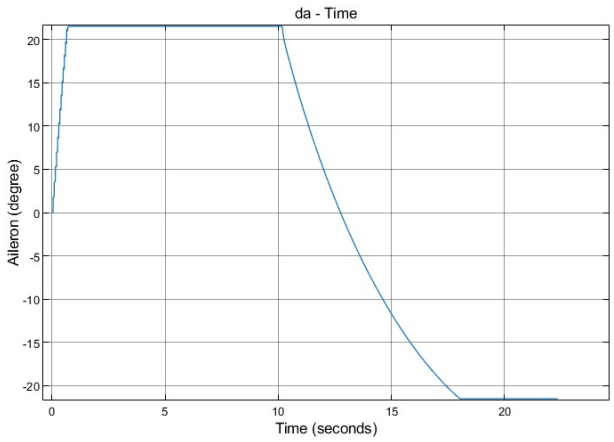
Tam Sola Kaçış



Şekil 54 Tam Sola Kaçış Gaz Girdisi



Şekil 55 Tam Sola Kaçış Elevatör Girdisi



Şekil 56 Tam Sola Kaçış Aleron Girdisi

C. MATLAB Kodları

1. *Simülasyon Dışı Kodlar*

Listing 1: F-16 Modeli için Kontrolcü Tasarımı: Trim, Doğrusallaştırma ve LQR

```
%==========================================================================
% UCK 4902 / F-16 İçin Otonom Yer Çarpışma Önleyici Sistem Geliştirilmesi
%==========================================================================

%==========================================================================
% Mehmet Tunahan Kara - 110200028
% Celal Aksoy - 110200051
%==========================================================================

%==========================================================================
% F-16 MODELİ İÇİN KONTROLCÜ TASARIMI: TRİM, DOĞRUSALLAŞTIRMA VE LQR
%==========================================================================
% Bu kod, F-16 uçağının nonlineer modelini kullanarak belirli bir uçuş
% koşulu için denge noktası (trim) bulur. Ardından, bu denge noktası
% etrafında yanal ve boylamsal dinamikler için doğrusal durum-uzay
% modelleri oluşturur. Son olarak, bu doğrusal sistemler için LQR
% (Linear-Quadratic Regulator) tabanlı optimal kontrolcü kazançlarını
% hesaplar.
%==========================================================================

%% BÖLÜM 1: UÇAK PARAMETRELERİ VE BAŞLANGIÇ KOŞULLARI

% F-16 modelinin fiziksel, ataletsel ve aerodinamik parametreleri ile
% simülasyon başlangıç koşulları bu bölümde tanımlanır.

% Fiziksel ve Aerodinamik Özellikler
I = [12875     0      -1331;      % Atalet Momenti Matrisi, [kg*m^2]
       0     75674      0;
     -1331     0      85552];
Mass = 9300;                       % Toplam Kütle, [kg]
c    = 3.45;                       % Ortalama Aerodinamik Veter, [m]
S    = 27.88;                      % Referans Kanat Alanı, [m^2]
b    = 9.144;                      % Kanat Açıklığı, [m]
xCgref = 0.35;                     % Referans ağırlık merkezi
xCg    = 0.35;                     % Anlık ağırlık merkezi

load('f16_database.mat');          % Aerodinamik katsayı veri tabanı yükler

% Başlangıç Koşulları
dsb = 0;                           % Başlangıç stabilizatör sapması
dr  = 0;                           % Başlangıç istikamet dümeni sapması
V_init        = [150 0 0];         % Başlangıç hızı [u,v,w], [m/s]
Position_init = [0 3000 -1500];    % Başlangıç konumu [x,y,z], [m]
Euler_init    = [0 0 0];           % Başlangıç Euler açıları [phi,theta,psi]


%% BÖLÜM 2: TRİM (UÇUŞ DENGELEME) ANALİZİ

% Uçağın, belirlenen bir uçuş koşulunda (örneğin, sabit hız ve irtifada
```

```
% düz uçuş) kararlı bir dengeye ulaşması için gereken durum (state) ve
% girdi (input) değerlerini bulma işlemidir.

model = 'f16_Trim';        % Trim analizi için kullanılacak Simulink modeli
opspec = operspec(model);  % Trim spesifikasyon nesnesi oluştur

% --- Durum Değişkenleri (States) İçin Denge Kısıtları ---
% Euler açıları ve açısal hızlar başlangıç değerlerinde sabitlenir.
opspec.States(1).Known = [true;true;true]; % [phi,theta,psi] biliniyor
opspec.States(2).Known = [true;true;true]; % [p,q,r] biliniyor
opspec.States(2).SteadyState = [false;false;false];

% Gövde eksen hızlarından yanal ve dikey bileşenler sıfırlanır,
% ileri hız serbest bırakılır.
opspec.States(3).Known = [false;true;true]; %[ub,vb,wb]:ub serbest,vb=wb=0

% Konum değişkenlerinden irtifa (ze) sabit tutulur.
opspec.States(4).Known = [false;false;true];      % [xe,ye,ze]: ze bilinir
opspec.States(4).SteadyState = [false;false;true];% ze değişmesin

% --- Kontrol Girdileri (Inputs) İçin Sınır Değerler ---

% Gaz kolu pozisyonu, [%]
opspec.Inputs(1).u = 90;
opspec.Inputs(1).Min = 0;
opspec.Inputs(1).Max = 100;
% Yükseliş dümeni (dh) sapması sabitlenir, [rad]
opspec.Inputs(2).Min = -0.43633;
opspec.Inputs(2).Max = -0.43633;
% Kanatçık (da) sapması için sınırlar, [rad]
opspec.Inputs(3).Min = -0.37524;
opspec.Inputs(3).Max = 0.37524;

% --- Optimizasyon ile Denge Noktasının Bulunması ---
opt = findopOptions('DisplayReport','iter', ...
    'OptimizationOptions', optimset('Algorithm','sqp'));

% Belirlenen kısıtlar altında denge noktasını (op) hesapla.
[op, opreport] = findop(model, opspec, opt);


%% BÖLÜM 3: DOĞRUSALLAŞTIRMA VE MODEL İNDİRGEME

% Nonlineer F-16 modeli, hesaplanan denge noktası (op) etrafında
% doğrusal durum-uzay (A,B,C,D) modeline dönüştürülür. Ardından yanal ve
% boylamsal dinamikler için alt sistemler oluşturulur.

io = getlinio(model);  % Modelin girdi/çıktı (I/O) kanallarını al

% --- Yanal (Roll) Dinamik Modelinin Oluşturulması ---

% Tam sistemin doğrusal modelini oluştur
linsys_full = linearize(model, io, op);
% Girdi: Kanatçık (da), Çıktılar: Tüm durumlar
```

```
linsys_lat  = linsys_full(:, 2);
% Model indirgeme: Sadece yanal hareketle ilişkili durumlar korunur.
linsys_roll = modred(linsys_lat, [2 3 5 6 7 8 9 10 11 12], 'del');

% --- Boylamsal (Pitch) Dinamik Modelinin Oluşturulması ---
% Tam sistemin doğrusal modelini oluştur
linsys_full = linearize(model, io, op);
 % Girdi: Yükseliş Dümeni (dh), Çıktılar: Tüm durumlar
linsys_long = linsys_full(:, 3);
% Model indirgeme: Sadece boylamsal hareketle ilişkili durumlar korunur.
linsys_pitch = modred(linsys_long, [1 3 4 6 7 8 9 10 11 12], 'del');

% --- Kontrol Edilebilirlik Analizi ---
% Oluşturulan doğrusal alt sistemlerin kontrol edilebilirliği test edilir.
if rank(ctrb(linsys_roll)) == length(linsys_roll.Statename) && ...
   rank(ctrb(linsys_pitch)) == length(linsys_pitch.Statename)
    disp("Yanal ve boylamsal sistemler kontrol edilebilirdir.");
end


%% BÖLÜM 4: LQR KONTROLCÜ TASARIMI

% Doğrusal sistemler için optimal durum-geri besleme kazanç matrisi (K),
% LQR metodu ile tasarlanır. Q matrisi durum hatalarının, R matrisi ise
% kontrol eforunun maliyetini temsil eder.

% --- Roll Ekseni için LQR Tasarımı ---
LQR_roll.A = linsys_roll.A;
LQR_roll.B = linsys_roll.B;
LQR_roll.Q = diag([1 .1]);  % Durum ağırlıkları: [roll_acisi, roll_hizi]
LQR_roll.R = diag(1);       % Kontrol girdi (da) ağırlığı
% LQR çözücüsü ile K_roll kazanç matrisi hesaplanır.
K_roll = lqr(LQR_roll.A, LQR_roll.B, LQR_roll.Q, LQR_roll.R);
disp("Roll ekseni için LQR kazançları hesaplandı.");

% --- Pitch Ekseni için LQR Tasarımı ---
LQR_pitch.A = linsys_pitch.A;
LQR_pitch.B = linsys_pitch.B;
LQR_pitch.Q = diag([1 .1]);  % Durum ağırlıkları: [pitch_acisi, pitch_hizi]
LQR_pitch.R = diag(1);       % Kontrol girdi (dh) ağırlığı
% LQR çözücüsü ile K_pitch kazanç matrisi hesaplanır.
K_pitch = lqr(LQR_pitch.A, LQR_pitch.B, LQR_pitch.Q, LQR_pitch.R);
disp("Pitch ekseni için LQR kazançları hesaplandı.");
```

Listing 2: F-16 Modeli için Sentetik Arazi Haritası Oluşturma

```
%==========================================================================
% UCK 4902 / F-16 İçin Otonom Yer Çarpışma Önleyici Sistem Geliştirilmesi
%==========================================================================

%==========================================================================
% Mehmet Tunahan Kara - 110200028
% Celal Aksoy - 110200051
%==========================================================================

%==========================================================================
% SENTETİK ARAZİ HARİTASI OLUŞTURMA
%==========================================================================
% Bu kod, Gauss fonksiyonları kullanarak ana dağ sıraları ve rastgele
% dağılmış küçük tepeler içeren üç boyutlu bir sentetik arazi yüzeyi
% (haritası) oluşturur.
%==========================================================================

%% BÖLÜM 1: HARİTA PARAMETRELERİNİN TANIMLANMASI

% Haritanın oluşturulacağı koordinat sistemi ve grid (ızgara) yapısı
% bu bölümde tanımlanır.

% X ve Y eksenleri için koordinat vektörleri oluşturulur.
x_range = 0:1:9000;
y_range = 0:1:9000;

% meshgrid fonksiyonu ile 2D bir koordinat matrisi (grid) oluşturulur.
[X, Y] = meshgrid(x_range, y_range);


%% BÖLÜM 2: ANA DAĞ SIRALARININ OLUŞTURULMASI

% Haritanın ana coğrafi yapısını oluşturan büyük dağlar, iki boyutlu
% Gauss fonksiyonları kullanılarak modellenir.

mountain  = 1500 * exp(-((X-3000).^2)/1e5) .* exp(-((Y-3000).^2)/.55e6);
mountain2 = 1500 * exp(-((X-3000).^2)/1e5) .* exp(-((Y-4000).^2)/.55e6);
mountain3 = 1500 * exp(-((X-3000).^2)/1e5) .* exp(-((Y-2000).^2)/.55e6);


%% BÖLÜM 3: RASTGELE KÜÇÜK TEPELERİN EKLENMESİ

% Araziye daha gerçekçi bir görünüm kazandırmak için rastgele konumlara,
% yüksekliklere ve genişliklere sahip çok sayıda küçük tepe eklenir.

rng(3); % Tekrarlanabilir rastgele sonuçlar için başlangıç değeri  atanır.

% Tepe sayısı ve özellikleri için parametreler belirlenir.
num_hills        = 100;
hill_heights     = 400 + 300*rand(1, num_hills); % Tepe yükseklikleri
hill_positions_x = 9000 * rand(1, num_hills);    % X eksenindeki konumları
hill_positions_y = 9000 * rand(1, num_hills) - 1000; % Y eksen  konumları
hill_sigmas      = 50 + 80*rand(1, num_hills); % Tepe genişlikleri (sigma)
```

```

% Tepelerin ekleneceği boş bir harita matrisi oluşturulur.
small_hills = zeros(size(X));

% Döngü içerisinde her bir tepe oluşturulur ve ana tepe haritasına eklenir.
for i = 1:num_hills
    sx     = hill_positions_x(i);
    sy     = hill_positions_y(i);
    sigma  = hill_sigmas(i);
    height = hill_heights(i);

    % Her bir tepe, bir Gauss fonksiyonu ile modellenir.
    small_hills = small_hills + ...
    height * exp(-((X - sx).^2 + (Y - sy).^2) / (2 * sigma^2));
end


%% BÖLÜM 4: TOPLAM YÜKSEKLİK VERİSİNİN HESAPLANMASI

% Nihai arazi haritası, ana dağ sıraları ile küçük tepelerin
% yükseklik verilerinin toplanmasıyla elde edilir.

Z_map = mountain + mountain2 + mountain3 + small_hills;


%% BÖLÜM 5: GÖRSELLEŞTİRME

% Bu bölüm, oluşturulan Z_map verisini 3D yüzey grafiği olarak çizer.
% İhtiyaç duyulduğunda aşağıdaki komutların yorumları kaldırılabilir.
% Ama kodun çalışma hızını negatif etkilediği göz önünde bulundurulmalıdır.

% surf(X, Y, Z_map, 'EdgeColor', 'none') % Yüzey grafiğini oluşturur.
% colormap(sky)                          % Renk haritasını ayarlar.
% xlabel('X ekseni (m)')                 % Eksen etiketleri.
% ylabel('Y ekseni (m)')
% zlabel('Yükseklik (m)')

disp("Harita oluşturma işlemi tamamlandı.");
```

Listing 3: F-16 Modeli için 3D Uçuş Animasyonu ve Görselleştirme

```
%==========================================================================
% UCK 4902 / F-16 İçin Otonom Yer Çarpışma Önleyici Sistem Geliştirilmesi
%==========================================================================

%==========================================================================
% Mehmet Tunahan Kara - 110200028
% Celal Aksoy - 110200051
%==========================================================================

%==========================================================================
% 3D UÇUŞ ANİMASYONU VE GÖRSELLEŞTİRME
%==========================================================================
% Bu kod, önceden çalıştırılmış bir simülasyonun sonuçlarını kullanarak
% bir F-16 uçağının uçuş yolunu 3 boyutlu olarak görselleştirir.
% Görselleştirme, 'flypath 3d' adlı özel bir MATLAB paketi ile
% gerçekleştirilir ve oluşturulan sentetik arazi üzerine çizdirilir.
%==========================================================================

%% BÖLÜM 1: UÇUŞ VERİSİNİN YÜKLENMESİ VE HAZIRLANMASI

% Simulink veya başka bir simülasyon ortamından elde edilen uçuş verisi,
% görselleştirme paketinin gerektirdiği formata dönüştürülür.

% Simülasyon çıktısından (out) uçak verilerini al.
sim_data = out.AircraftData.Data;
n = size(sim_data, 1);
data_aircraft = zeros(n, 6);

% Veri sütunları, animasyon fonksiyonunun beklediği sıraya göre düzenlenir.
% Bazı eksenlerde koordinat sistemi uyumluluğu için dönüşüm uygulanır.
data_aircraft(:,1) = sim_data(:,1);           % Konum x
data_aircraft(:,2) = sim_data(:,2);           % Konum y
% Konum z (yükseklik ekseni ters çevrilir)
data_aircraft(:,3) = -sim_data(:,3);
data_aircraft(:,4) = sim_data(:,5);           % Pitch açısı
% Yaw açısı (90 derece ofset eklenir)
data_aircraft(:,5) = sim_data(:,6) + pi/2;
data_aircraft(:,6) = sim_data(:,4);           % Roll açısı


%% BÖLÜM 2: UÇAK NESNESİNİN TANIMLANMASI

% Animasyonda kullanılacak olan 3D uçak nesnesi, konumu, modeli, rengi ve
% ölçeği gibi görsel özellikleriyle birlikte tanımlanır.

% Uçak verisini ve kaydedileceği dosya adını belirtir
new_object('aircraft.mat', data_aircraft, ...
    'model', 'f-16.mat', ...          % Uçağın 3D model dosyasını tanımlar
    'edge', [0.1 0.1 0.1], ...        % Modelin kenar rengi
    'face', [0.2 0.2 0.2], ...        % Modelin yüzey rengi
    'path', 'on', ...            % Uçuş yolunun (iz) çizilmesini aktive eder
    'pathcolor', [1 0 0], ...    % Yolun rengini kırmızı olarak ayarlar
    'pathwidth', 1.5, ...        % Yolun çizgi kalınlığı
```

```
    'scale', 30);               % Modelin görsel ölçeği


%% BÖLÜM 3: UÇUŞ YOLU ANİMASYONUNUN OLUŞTURULMASI

% Tanımlanan uçak nesnesi ve ayarlar kullanılarak uçuş animasyonu
% oluşturulur ve isteğe bağlı olarak bir GIF dosyası olarak kaydedilir.

flypath('aircraft.mat', ...           % Animasyonu yapılacak nesneyi seçer
    'animate', 'on', ...              % Animasyonu aktive eder
    'output', '5g_pull_up.gif', ...   % Animasyonu GIF olarak kaydeder
    'step', 50, ...         % Animasyon adım büyüklüğü (hız/akıcılık ayarı)
    'font', 'Georgia', ...            % Eksen etiketlerinin yazı tipi
    'fontsize', 6, ...                % Yazı tipi boyutu
    'view', [45 30], ...              % 3D bakış açısı [yatay, dikey]
    'xlim', [min(x_range), max(x_range)], ... % X ekseni limiti
    'ylim', [min(y_range), max(y_range)], ... % Y ekseni limiti
    'zlim', [0 3500]);                        % Z ekseni limiti


%% BÖLÜM 4: YÜZEY VE EK VERİLERİN GRAFİĞE EKLENMESİ

% Animasyonun arka planına, daha önce oluşturulmuş olan sentetik arazi
% haritası ve varsa diğer yörünge verileri eklenir.

% Arazi yüzeyini 'surf' komutu ile çizdirir.
surf(X, Y, Z_map, 'EdgeColor', 'none')
colormap(sky) % Yüzey için renk haritasını belirler
xlabel('X ekseni (m)')
ylabel('Y ekseni (m)')
zlabel('Yükseklik (m)')

% Mevcut grafiğin üzerine ek çizim yapmak için 'hold on' komutu kullanılır.
hold on;
% Çarpışma kontrolünde kullanılan yörünge de grafiğe eklenir.
plot3(out.trajectories(1,:,2,end), out.trajectories(2,:,2,end), ...
      out.trajectories(3,:,2,end));
```

2. *Simülasyon İçi Kodlar*

Listing 4: Havuç Takip (Carrot Chasing) Güdüm Kontrol Algoritması

```
%==========================================================================
% UCK 4902 / F-16 İçin Otonom Yer Çarpışma Önleyici Sistem Geliştirilmesi
%==========================================================================

%==========================================================================
% Mehmet Tunahan Kara - 110200028
% Celal Aksoy - 110200051
%==========================================================================

function [yaw_rate_cmd, si_d, curr_x, curr_y, x_target, y_target, stop]=...
 carrot_chase(Pos, Yaw, waypoints)

%==========================================================================
% CARROT CHASING (HAVUÇ TAKİP) GÜDÜM ALGORİTMASI
%==========================================================================
% Bu fonksiyon, uçağımızı belirlenmiş yol noktaları (waypoints) üzerinde
% yönlendirmek için "Carrot Chasing" algoritmasını uygular. Algoritma,
% rota üzerinde aracın ilerisinde sanal bir hedef ("havuç") oluşturur ve
% aracı sürekli olarak bu hedefe yönlendirir.
%
% Girdiler:
%   Pos        : Uçağın mevcut konumu [x, y].
%   Yaw        : Uçağın mevcut gidiş açısı (yönelimi), [rad].
%   waypoints  : Takip edilecek yol noktaları [x1 y1; x2 y2; ...].
%
% Çıktılar:
%   yaw_rate_cmd :Uçağa uygulanması gereken dönüş hızı komutu.
%   si_d         : Uçağın olması gereken anlık hedef yönelim açısı.
%   curr_x, curr_y: Uçağın mevcut X ve Y koordinatları.
%   x_target, y_target: Sanal hedefin ("havuç") koordinatları.
%   stop         : Görevin tamamlandığını belirten bayrak (1 ise bitti).
%==========================================================================

%% BÖLÜM 1: PERFORMANS PARAMETRELERİ

% Algoritmanın davranışını belirleyen temel ayarlar.

% [m] Havuç Mesafesi: Sanal hedefin ne kadar ileride olacağı.
delta     = 5000;
% [-] Kazanç Katsayısı: Yönelim hatasına verilecek tepkinin agresifliği.
k         = 0.05;
 % [m] Tolerans: Bir yol noktasına ulaşıldığını kabul etme mesafesi.
tolerance = 1500;


%% BÖLÜM 2: DURUM DEĞİŞKENLERİ VE BAŞLANGIÇ AYARLARI

% ‘persistent‘ değişkenler, fonksiyon her çalıştığında değerlerini korur.
% Bu sayede, hangi yol segmentinde olduğumuz bilgisi kaybolmaz.

stop = 0; % Her döngü başında stop bayrağını sıfırla.
```

```

% Kalıcı (Persistent) değişkenler
persistent w1_x w1_y w2_x w2_y wp

% Fonksiyon ilk kez çalıştığında bu blok çalışır,başlangıç değerlerini atar
if isempty(wp)
    % Başlangıç yol segmentini tanımla
    wp_1 = waypoints(1, :);
    w1_x = wp_1(1); w1_y = wp_1(2); % Segment başlangıcı: 1. yol noktası

    wp_2 = waypoints(2, :);
    w2_x = wp_2(1); w2_y = wp_2(2); % Segment bitişi: 2. yol noktası

    wp = 2; % Takip edilen yol noktasının indeksi
end

% Mevcut durum değişkenlerini girdilerden al.
curr_x  = Pos(1);
curr_y  = Pos(2);
curr_si = Yaw;


%% BÖLÜM 3: GEOMETRİK HESAPLAMALAR VE "HAVUÇ" TESPİTİ

%Sanal hedef noktasının ("havuç") koordinatları geometrik olarak hesaplanır.

% Uçağa en yakın hedef yol noktasına olan mesafe(waypoint değiştirmek için).
dist_wp = sqrt((w2_x - curr_x)^2 + (w2_y - curr_y)^2);

% --- Havuç Noktası Hesaplama Adımları ---
% 1) Uçağın bir önceki yol noktasına (w1) uzaklığı (R_u).
R_u = sqrt((w1_x - curr_x)^2 + (w1_y - curr_y)^2);

% 2) Takip edilen yol segmentinin (w1 -> w2) açısı (theta).
theta = atan2((w2_y - w1_y), (w2_x - w1_x));

% 3) Önceki yol noktasından (w1) uçağa olan doğrunun açısı (theta_u).
theta_u = atan2(curr_y - w1_y, curr_x - w1_x);

% 4) Uçağın yol segmentine göre sapma açısı (beta).
beta = theta - theta_u;

% 5) Uçağın yol segmenti üzerindeki izdüşümünün başlangıca olan mesafesi(R).
R = sqrt(R_u^2 - (R_u * sin(beta))^2);

% 6) Sanal Hedef ("Havuç") Koordinatları: İzdüşüm noktasından 'delta'
%  kadar ilerisi.
x_target = w1_x + (R + delta) * cos(theta);
y_target = w1_y + (R + delta) * sin(theta);


%% BÖLÜM 4: KONTROL KOMUTU ÜRETİMİ VE YOL NOKTASI GEÇİŞİ

% İstenen Yönelim (si_d): Uçaktan "havuç" noktasına olan açı.
```

```
si_d = atan2(y_target - curr_y, x_target - curr_x);

% --- Yol Noktası (Waypoint) Değiştirme Mantığı ---
if dist_wp < tolerance && wp ~= length(waypoints)
    % Eğer hedefe yeterince yaklaşıldıysa ve bu son hedef değilse,
    % bir sonraki yol segmentine geç.
    wp_1 = waypoints(wp, :);
    w1_x = wp_1(1); w1_y = wp_1(2);

    wp_2 = waypoints(wp + 1, :);
    w2_x = wp_2(1); w2_y = wp_2(2);

    wp = wp + 1; % Yol noktası indeksini bir artır.

elseif dist_wp < tolerance && wp == length(waypoints)
    % Eğer son hedefe yeterince yaklaşıldıysa, görevi bitir.
    stop = 1;
    fprintf("Görev Tamamlandı.\n");
end

% --- Dönüş Komutu Hesaplaması ---
% Yönelim Hatası: Olması gereken yön ile mevcut yön arasındaki fark.
yaw_diff = si_d - curr_si;

% Normalizasyon: Açı farkını [-pi, +pi] aralığına sıkıştırarak en kısa
% dönüşü sağla.
if yaw_diff > pi
    yaw_diff = yaw_diff - 2 * pi;
elseif yaw_diff < -pi
    yaw_diff = yaw_diff + 2 * pi;
end

% Nihai Dönüş Hızı Komutu (Oransal Kontrolcü).
yaw_rate_cmd = k * yaw_diff;

end
```

Listing 5: Çarpışma Kontrolü için Potansiyel Uçuş Yörüngeleri Üretme

```
%==========================================================================
% UCK 4902 / F-16 İçin Otonom Yer Çarpışma Önleyici Sistem Geliştirilmesi
%==========================================================================

%==========================================================================
% Mehmet Tunahan Kara - 110200028
% Celal Aksoy - 110200051
%==========================================================================

function Trajectories = fcn(R, Position, theta, psi)

%==========================================================================
% ÇARPIŞMA KONTROLÜ İÇİN POTANSİYEL UÇUŞ YÖRÜNGELERİ ÜRETME
%==========================================================================
% Bu fonksiyon, hava aracının mevcut konumundan ve yöneliminden
% yola çıkarak, farklı yatış açıları (bank angles) için gelecekteki
% potansiyel uçuş yörüngelerini (trajectories) 3 boyutlu olarak üretir.
%
% Girdiler:
%   R        : Manevra yarıçapı
%   Position : Uçağın mevcut konumu [x, y, z]
%   theta    : Uçağın yunuslama (pitch) açısı
%   psi      : Uçağın sapma (yaw) açısı
%
% Çıktı:
%   Trajectories: 3xNxM boyutunda yörünge matrisi. (3: xyz, N: nokta sayısı,
%                 M: yatış açısı sayısı)
%==========================================================================

%% BÖLÜM 1: GİRDİLERİN İŞLENMESİ

% Fonksiyon girdileri alınır ve yerel değişkenlere atanır. Z, genellikle
% kullanılan 'aşağı pozitif' (NED) sistemine uyum için ters çevrilir.

X0 = Position(1);
Y0 = Position(2);
Z0 = -Position(3);


%% BÖLÜM 2: TEMEL YÖRÜNGE GEOMETRİSİNİN OLUŞTURULMASI

% Uçağın yunuslama açısına göre bir dairesel yay geometrisi oluşturulur.
% Bu, uçağın yunuslama manevrasını temsil eden temel yörüngedir.

% Yunuslama manevrasının dairesel merkezini hesapla.
Xc = X0 - R*sin(theta);
Yc = Y0;
Zc = Z0 + R*cos(theta);
center = [Xc; Yc; Zc];

% Dairesel yayın düzlemini tanımlamak için dik vektörler bulunur.
% Normal vektörü [0,1,0] (Y ekseni) olan düzlem, X-Z düzlemidir.
v = null([0, 1, 0]);
```

```

% Parametrik denklem kullanarak dairesel yayın noktaları oluşturulur.
theta_circle = linspace(-theta - pi / 2, -pi, 20);
circle_points = center + R * (v(:,1) * cos(theta_circle) + v(:,2)...
 * sin(theta_circle));
points = circle_points(:,2:end); % Başlangıç noktası hariç tutulur.


%% BÖLÜM 3: GİDİŞ AÇISINA (YAW) GÖRE YÖRÜNGENİN DÖNDÜRÜLMESİ

% Önceki adımda X-Z düzleminde oluşturulan temel yörünge, uçağın gidiş
% yönüne (psi açısı) hizalanmak üzere Z ekseni etrafında döndürülür.

% Z ekseni etrafında döndürme matrisi oluşturulur.
D = [cos(psi), -sin(psi), 0;
     sin(psi),  cos(psi), 0;
     0,         0,        1];

% Yörünge, uçağın mevcut konumu etrafında döndürülür
% (kaydır-döndür-geri kaydır).
shifted_points = points - [X0; Y0; 0];
rotated_points = D * shifted_points;
% Aracın konumu ilk nokta olarak eklenir.
points = [[X0;Y0;Z0], (rotated_points + [X0; Y0; 0])];


%% BÖLÜM 4: YATIŞ AÇISI (BANK ANGLE) VARYASYONLARI

% Gidiş yönüne hizalanmış temel yörünge, farklı yatış açıları (phi) için
% uçağın ileri yön vektörü etrafında döndürülerek nihai yörünge seti oluşur.

% Yatış açısı dönüşü için eksen, aracın ileri yön vektörüdür.
line_point = [X0; Y0; Z0];
line_dir   = [cos(psi); sin(psi); 0];

% Farklı yatış açıları için döngü başlatılır.
k = 1;
desired_phi_angles = [-90, 0, 90]; % [derece]
% Çıktı matrisi için bellek ayır.
Trajectories = zeros(3, 21, length(desired_phi_angles));

for phi = deg2rad(desired_phi_angles)
% Belirtilen eksen (line_dir) ve açı (phi) için 3D dönüş matrisini hesapla.
    R_bank = rotationMatrix(line_dir, phi);

    % Yörüngeyi yatış açısına göre döndür.
    points_shifted = points - line_point;
    rotated_points_banked = R_bank * points_shifted;
    rotated_points_final = rotated_points_banked + line_point;

    % Döndürülmüş yörüngeyi çıktı matrisine kaydet.
    Trajectories(:,:,k) = rotated_points_final;
    k = k + 1;
end
```

```
% --- İsteğe Bağlı Görselleştirme ---
% hold on;
% for i = 1:size(Trajectories, 3)
%     plot3(Trajectories(1,:,i), Trajectories(2,:,i), Trajectories(3,:,i));
% end
% xlabel('X'); ylabel('Y'); zlabel('Z');
% grid on; axis equal; hold off;

end


function R = rotationMatrix(axis, phi)
%-----------------------------------------------------------------------
% Bu yardımcı fonksiyon, belirtilen bir 'axis' (eksen) etrafında 'phi'
% açısı kadar 3D döndürme işlemini gerçekleştiren bir rotasyon matrisi (R)
% oluşturur (Rodrigues'in Döndürme Formülü).
%-----------------------------------------------------------------------
    axis = axis / norm(axis); % Döndürme eksenini birim vektöre çevir.
    x = axis(1); y = axis(2); z = axis(3);
    c = cos(phi); s = sin(phi); t = 1 - c;

    % Rodrigues'in Döndürme Formülü'nün matris formu
    R = [
        t*x*x + c,   t*x*y - s*z, t*x*z + s*y;
        t*x*y + s*z, t*y*y + c,   t*y*z - s*x;
        t*x*z - s*y, t*y*z + s*x, t*z*z + c
    ];
end
```

Listing 6: Yörünge için Arazi Çarpışma Kontrolü

```
%==========================================================================
% UCK 4902 / F-16 İçin Otonom Yer Çarpışma Önleyici Sistem Geliştirilmesi
%==========================================================================

%==========================================================================
% Mehmet Tunahan Kara - 110200028
% Celal Aksoy - 110200051
%==========================================================================

function [stop, collision] = fcn(Z_map, x, y, Trajectories)

%==========================================================================
% YÖRÜNGE İÇİN ARAZİ ÇARPIŞMA KONTROLÜ
%==========================================================================
% Bu fonksiyon, önceki kodda belirtilen bir uçuş yörüngesinin ,
% bir arazi haritası ('Z_map') ile çarpışıp çarpışmadığını kontrol eder.
%
% Girdiler:
%   Z_map        : Arazi yükseklik verilerini içeren matris.
%   x, y         : Z_map'in koordinat vektörleri.
%   Trajectories : Kontrol edilecek potansiyel yörüngeler seti.
%
% Çıktılar:
%   stop         : Yörünge harita sınırlarının dışına çıkarsa 1 olan bayrak.
%   collision    : Her yörünge noktası için çarpışma durumunu (1/0)
%                  içeren vektör.
%==========================================================================


%% BÖLÜM 1: YÖRÜNGE KOORDİNATLARININ HARİTA İNDİSLERİNE ÇEVRİLMESİ

% Kontrol için seçilen yörüngenin (burada 2. yörünge olan yatışsız rota)
% 3D koordinatları, arazi haritası (Z_map) matrisinin satır ve sütun
% indislerine dönüştürülür.

stop = 0; % Başlangıçta stop bayrağı sıfır olarak ayarlanır.

% Yörüngenin X ve Y koordinatları, harita matrisinin sütun ve satır
% indislerine dönüştürülür.
row = int32(Trajectories(2, :, 2) - min(x)) + 1;
col = int32(Trajectories(1, :, 2) - min(y)) + 1;


%% BÖLÜM 2: HARİTA SINIRLARI KONTROLÜ

% Yörüngenin herhangi bir noktasının harita sınırları dışına çıkıp
% çıkmadığı kontrol edilir. Bu, geçersiz indislerle haritaya erişimi önler.

if max(row) >= size(Z_map, 1) || max(col) >= size(Z_map, 2)
    % Eğer yörünge sınırdaysa veya dışındaysa 'stop' bayrağını aktive et.
    stop = 1;
    fprintf("UYARI: Yörünge harita sınırları dışına çıktı!\n");
     % Sınır dışı durumu, bir çarpışma olarak kabul edilir.
```

```
    collision = ones(21, 1);
    return; % Fonksiyonun çalışmasını bu noktada durdur.
end

%% BÖLÜM 3: ÇARPIŞMA TESPİTİ

% Yörünge üzerindeki her bir noktanın irtifası ile o noktanın altındaki
% arazi yüksekliği karşılaştırılarak çarpışma durumu tespit edilir.

% Yörüngenin geçtiği (row, col) indislerindeki arazi yüksekliklerini al.
% sub2ind, satır ve sütun indislerini lineer indislere çevirerek tek seferde
% tüm Z değerlerinin alınmasını sağlar.
linear_indices = sub2ind(size(Z_map), row, col);
terrain_z = Z_map(linear_indices)'; % Arazinin yörünge altındaki irtifası

% 2. Adım: Yörüngenin kendi irtifa değerlerini al.
trajectory_z = Trajectories(3, :, 2)';

% 3. Adım: Arazi irtifasının yörünge irtifasından büyük veya eşit olduğu
% durumları bularak çarpışmayı tespit et. (Sonuç: mantıksal 1/0 vektörü)
collision = terrain_z >= trajectory_z;

end
```